Repository Audit

File: eslint.config.mjs

```
import { defineConfig, globalIgnores } from "eslint/config";
import nextVitals from "eslint-config-next/core-web-vitals";
import nextTs from "eslint-config-next/typescript";

const eslintConfig = defineConfig([
  ...nextVitals,
  ...nextTs,
  // Override default ignores of eslint-config-next.
  globalIgnores([
    // Default ignores of eslint-config-next:
    ".next/**",
    "out/**",
    "build/**",
    "next-env.d.ts",
  ]),
]);

export default eslintConfig;
```

File: next.config.ts

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  /* config options here */
  reactCompiler: true,
};

export default nextConfig;
```

File: package.json

```
{
  "name": "axiom",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint",
    "test": "node --import tsx --test src/lib/**/*.test.ts"
  },
  "dependencies": {
    "@supabase/auth-ui-react": "^0.4.7",
    "@supabase/auth-ui-shared": "^0.1.8",
    "@supabase/ssr": "^0.10.3",
    "@supabase/supabase-js": "^2.105.4",
    "next": "16.2.6",
    "react": "19.2.4",
    "react-dom": "19.2.4"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "@types/node": "^20",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "babel-plugin-react-compiler": "1.0.0",
    "eslint": "^9",
    "eslint-config-next": "16.2.6",
```

```
    "tailwindcss": "^4",
    "tsx": "^4.21.0",
    "typescript": "^5"
  }
}
```

File: postcss.config.mjs

```
const config = {
  plugins: {
    "@tailwindcss/postcss": {},
  },
};

export default config;
```

File: src/app/auth/auth-code-error/page.tsx

```
"use client";

import Link from "next/link";

export default function AuthCodeError() {
  return (
    <main className="min-h-screen flex items-center justify-center bg-background text-foreground p-6">
      <div className="w-full max-w-md bg-background border-2 border-red-500/20 rounded-[2.5rem] p-10 shadow-2xl
text-center">
        <div className="w-20 h-20 bg-red-500/10 rounded-3xl flex items-center justify-center mx-auto mb-6">
          <svg
            width="40"
            height="40"
            viewBox="0 0 24 24"
            fill="none"
            stroke="currentColor"
            strokeWidth="3"
            className="text-red-500"
          >
             <path d="M10.29 3.86L1.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0 0 0-3.42 0z"
/>
            <line x1="12" y1="9" x2="12" y2="13" />
            <line x1="12" y1="17" x2="12.01" y2="17" />
          </svg>
        </div>

        <h1 className="text-3xl font-black tracking-tighter uppercase mb-4">
          Auth Failed
        </h1>

        <p className="text-gray-500 text-sm font-bold uppercase tracking-widest mb-8 leading-relaxed">
          The secure handshake with the authentication provider was interrupted.
        </p>

        <Link
          href="/"
                 className="w-full bg-foreground text-background px-4 py-5 rounded-2xl font-black text-lg
hover:scale-[1.02] active:scale-[0.98] transition-all shadow-xl shadow-foreground/10 block uppercase
text-center"
        >
          Return to Login
        </Link>
      </div>
    </main>
```

```
  );
}
```

File: src/app/auth/callback/route.ts

```
import { NextResponse } from "next/server";
// The client you created in Step 1
import { createClient } from "@/lib/supabase/server";

export async function GET(request: Request) {
  const { searchParams, origin } = new URL(request.url);
  const code = searchParams.get("code");
  // if "next" is in search params, use it as the redirection URL
  const next = searchParams.get("next") ?? "/dashboard";

  if (code) {
    const supabase = await createClient();
    const { error } = await supabase.auth.exchangeCodeForSession(code);
    if (!error) {
      return NextResponse.redirect(`${origin}${next}`);
    }
  }

  // return the user to an error page with instructions
  return NextResponse.redirect(`${origin}/auth/auth-code-error`);
}
```

File: src/app/dashboard/AccountSection.tsx

```
"use client";

import { useState, useSyncExternalStore } from "react";
import { ACCOUNT_TYPES, type Account } from "@/lib/finance/accounts";
import { TAXONOMY_CATEGORIES } from "@/lib/finance/taxonomy";
import {
  createAccountAction,
  deleteAccountAction,
  createDeploymentAction,
  type DashboardSnapshot,
} from "./actions";
import { Deployment } from "@/lib/analytics/types";
import { formatCurrency } from "@/lib/utils/formatters";
import { AccountMap } from "@/lib/utils/taxonomy";

interface AccountSectionProps {
  accounts: Account[];
  deployments: Deployment[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

const emptySubscribe = () => () => {};

export function AccountSection({
  accounts,
  deployments,
  onSnapshot,
}: AccountSectionProps) {
  const isClient = useSyncExternalStore(
    emptySubscribe,
    () => true,
    () => false,
  );
```

```
const [activeTab, setActiveTab] = useState<"sources" | "log">("sources");
const [isAdding, setIsAdding] = useState(false);
const [isLoading, setIsLoading] = useState(false);
const [error, setError] = useState<string | null>(null);

// Source Form
const [sourceForm, setSourceForm] = useState({
  account_name: "",
  account_type: "checking",
  current_balance: "",
  institution: "",
});

// Log Form
const [logForm, setLogForm] = useState({
  title: "",
  amount: "",
  category: "Maintenance",
  accountId: "",
});

async function handleSourceSubmit(e: React.FormEvent) {
  e.preventDefault();
  setIsLoading(true);
  setError(null);
  try {
    const snapshot = await createAccountAction({
      account_name: sourceForm.account_name,
      account_type: sourceForm.account_type,
      current_balance: Number(sourceForm.current_balance),
      institution: sourceForm.institution || undefined,
    });
    setSourceForm({
      account_name: "",
      account_type: "checking",
      current_balance: "",
      institution: "",
    });
    setIsAdding(false);
    onSnapshot(snapshot);
  } catch (err: unknown) {
    setError(err instanceof Error ? err.message : "Failed to create source");
  } finally {
    setIsLoading(false);
  }
}

async function handleLogSubmit(e: React.FormEvent) {
  e.preventDefault();
  setIsLoading(true);
  setError(null);
  try {
    const snapshot = await createDeploymentAction({
      title: logForm.title,
      amount: Number(logForm.amount),
      category: logForm.category,
      accountId: logForm.accountId,
    });
    setLogForm({
      title: "",
      amount: "",
      category: "Maintenance",
      accountId: "",
```

```
      });
      setIsAdding(false);
      onSnapshot(snapshot);
    } catch (err: unknown) {
      setError(err instanceof Error ? err.message : "Failed to log spending");
    } finally {
      setIsLoading(false);
    }
  }

  async function handleDeleteSource(id: string) {
    if (!confirm("Archive this source? Historical records will be preserved."))
      return;
    setIsLoading(true);
    try {
      const snapshot = await deleteAccountAction(id);
      onSnapshot(snapshot);
    } catch (err: unknown) {
      alert("Failed to archive source");
    } finally {
      setIsLoading(false);
    }
  }

  return (
    <div className="space-y-6">
      <div className="flex items-center justify-between px-1">
        <div className="flex gap-4">
          <button
            onClick={() => {
              setActiveTab("sources");
              setIsAdding(false);
            }}
            className={`text-xl font-black tracking-tight uppercase transition-all ${
              activeTab === "sources" ? "text-foreground" : "text-foreground/20"
            }`}
          >
            Sources
          </button>
          <button
            onClick={() => {
              setActiveTab("log");
              setIsAdding(false);
            }}
            className={`text-xl font-black tracking-tight uppercase transition-all ${
              activeTab === "log" ? "text-foreground" : "text-foreground/20"
            }`}
          >
            Log Spend
          </button>
        </div>
        <button
          onClick={() => setIsAdding(!isAdding)}
             className="flex items-center gap-2 px-3 py-1.5 bg-foreground/5 hover:bg-foreground/10 rounded-xl
transition-all group"
        >
                    <span  className="text-[10px]  font-black  uppercase  tracking-widest  text-foreground/60
group-hover:text-foreground">
            {isAdding
              ? "Cancel"
              : activeTab === "sources"
                ? "+ Add source"
                : "+ New entry"}
          </span>
```

```
        </button>
      </div>

      {isAdding && activeTab === "sources" && (
        <div className="bg-background border-2 border-foreground rounded-3xl p-6 shadow-2xl animate-in fade-in
slide-in-from-top-4 duration-300">
          <form onSubmit={handleSourceSubmit} className="space-y-4">
            <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Source Name
                </label>
                <input
                  type="text"
                  required
                  placeholder="e.g. Primary Checking"
                  value={sourceForm.account_name}
                  onChange={(e) =>
                    setSourceForm({
                      ...sourceForm,
                      account_name: e.target.value,
                    })
                  }
                                  className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
                />
              </div>
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Current Balance (KSh)
                </label>
                <input
                  type="number"
                  required
                  placeholder="0.00"
                  value={sourceForm.current_balance}
                  onChange={(e) =>
                    setSourceForm({
                      ...sourceForm,
                      current_balance: e.target.value,
                    })
                  }
                                  className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
                />
              </div>
            </div>

            <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Type
                </label>
                <select
                  value={sourceForm.account_type}
                  onChange={(e) =>
                    setSourceForm({
                      ...sourceForm,
                      account_type: e.target.value,
                    })
                  }
```

```
                              className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold appearance-none"
                >
                  {ACCOUNT_TYPES.map((t) => (
                    <option key={t.value} value={t.value}>
                      {t.label}
                    </option>
                  ))}
                </select>
              </div>
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Institution (Optional)
                </label>
                <input
                  type="text"
                  placeholder="e.g. Standard Chartered"
                  value={sourceForm.institution}
                  onChange={(e) =>
                    setSourceForm({
                      ...sourceForm,
                      institution: e.target.value,
                    })
                  }
                              className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
                />
              </div>
            </div>

            {error && (
              <p className="text-red-500 text-[10px] font-black uppercase tracking-tight ml-1">
                {error}
              </p>
            )}

            <button
              type="submit"
              disabled={isLoading}
                       className="w-full  bg-foreground  text-background  py-3  rounded-xl  font-black  uppercase
tracking-widest hover:bg-foreground/90 transition-colors disabled:opacity-50"
            >
              {isLoading ? "ESTABLISHING..." : "ESTABLISH SOURCE"}
            </button>
          </form>
        </div>
      )}

      {isAdding && activeTab === "log" && (
         <div className="bg-background border-2 border-foreground rounded-3xl p-6 shadow-2xl animate-in fade-in
slide-in-from-top-4 duration-300">
           <form onSubmit={handleLogSubmit} className="space-y-4">
             <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
               <div>
                   <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                   What did you spend on?
                 </label>
                 <input
                   type="text"
                   required
                   placeholder="e.g. Groceries"
                   value={logForm.title}
```

```
                onChange={(e) =>
                  setLogForm({ ...logForm, title: e.target.value })
                }
                            className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
              />
            </div>
            <div>
                <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                Amount (KSh)
              </label>
              <input
                type="number"
                required
                placeholder="0.00"
                value={logForm.amount}
                onChange={(e) =>
                  setLogForm({ ...logForm, amount: e.target.value })
                }
                            className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
              />
            </div>
          </div>

          <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
            <div>
                <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                Category
              </label>
              <select
                value={logForm.category}
                onChange={(e) =>
                  setLogForm({ ...logForm, category: e.target.value })
                }
                            className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold appearance-none"
              >
                {TAXONOMY_CATEGORIES.map((t) => (
                  <option key={t.value} value={t.value}>
                    {t.label}
                  </option>
                ))}
              </select>
            </div>
            <div>
                <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                Source Account
              </label>
              <select
                required
                value={logForm.accountId}
                onChange={(e) =>
                  setLogForm({ ...logForm, accountId: e.target.value })
                }
                            className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold appearance-none"
              >
                <option value="">Select a source</option>
                {accounts.map((acc) => (
                  <option key={acc.id} value={acc.id}>
```

```
                    {acc.account_name} ({formatCurrency(acc.current_balance)})
                  </option>
                ))}
              </select>
            </div>
          </div>

          {error && (
            <p className="text-red-500 text-[10px] font-black uppercase tracking-tight ml-1">
              {error}
            </p>
          )}

          <button
            type="submit"
            disabled={isLoading || !logForm.accountId}
                     className="w-full bg-foreground text-background py-3 rounded-xl font-black uppercase
tracking-widest hover:bg-foreground/90 transition-colors disabled:opacity-50"
          >
            {isLoading ? "LOGGING..." : "LOG SPENDING"}
          </button>
        </form>
      </div>
    )}

    <div className="grid grid-cols-1 gap-4">
      {activeTab === "sources" ? (
        accounts.length === 0 ? (
             <div className="border-2 border-dashed border-foreground/10 rounded-3xl p-12 text-center group
hover:border-foreground/20 transition-colors">
            <p className="text-foreground/60 text-xs font-bold uppercase tracking-widest">
              No capital containers defined.
            </p>
            <p className="text-foreground/40 text-[10px] mt-2 uppercase tracking-tight opacity-60">
              Define accounts to track authoritative capital.
            </p>
          </div>
        ) : (
          accounts.map((account) => (
            <div
              key={account.id}
                           className="bg-foreground/5 border border-foreground/10 rounded-2xl p-5 group
hover:bg-foreground/10 transition-all relative overflow-hidden"
            >
              <div className="flex items-center justify-between relative z-10">
                <div className="space-y-1">
                  <div className="flex items-center gap-2">
                       <span className="text-[9px] font-black text-background bg-foreground/30 px-1.5 py-0.5
rounded uppercase tracking-tighter">
                         {AccountMap[account.account_type] ||
                           account.account_type}
                    </span>
                    <h3 className="text-sm font-black text-foreground uppercase tracking-tight">
                      {account.account_name}
                    </h3>
                  </div>
                  {account.institution && (
                    <p className="text-[10px] font-bold text-foreground/60 uppercase tracking-widest">
                      {account.institution}
                    </p>
                  )}
                </div>
                <div className="flex items-center gap-4">
                  <div className="text-right">
```

```
                    <p className="text-lg font-black tabular-nums text-foreground">
                      {formatCurrency(account.current_balance)}
                    </p>
                           <p className="text-[8px] font-black text-foreground/60 uppercase tracking-widest
opacity-60">
                      Authoritative Balance
                    </p>
                  </div>
                  <div className="flex items-center gap-2">
                    <button
                      onClick={() => {
                        setActiveTab("log");
                        setLogForm({ ...logForm, accountId: account.id });
                        setIsAdding(true);
                      }}
                      className="p-2 text-foreground/40 hover:text-foreground transition-colors"
                      title="Log spending from this source"
                    >
                      <svg
                        width="16"
                        height="16"
                        viewBox="0 0 24 24"
                        fill="none"
                        stroke="currentColor"
                        strokeWidth="3"
                      >
                        <path d="M12 5v14M5 12h14" />
                      </svg>
                    </button>
                    <button
                      onClick={() => handleDeleteSource(account.id)}
                            className="p-2 text-foreground/40 hover:text-red-500 transition-colors opacity-0
group-hover:opacity-100"
                    >
                      <svg
                        width="14"
                        height="14"
                        viewBox="0 0 24 24"
                        fill="none"
                        stroke="currentColor"
                        strokeWidth="3"
                      >
                         <path d="M3 6h18M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0 1 2-2h4a2 2 0
0 1 2 2v2" />
                      </svg>
                    </button>
                  </div>
                </div>
              </div>
            </div>
          ))
        )
      ) : (
        <div className="space-y-4">
                <div  className="border-2  border-dashed  border-foreground/10  rounded-3xl  p-8  text-center
bg-foreground/[0.02]">
            <p className="text-foreground/60 text-xs font-bold uppercase tracking-widest">
              Daily Spending Log
            </p>
                     <p  className="text-foreground/40  text-[10px]  mt-2  uppercase  tracking-tight  opacity-60
max-w-[200px] mx-auto leading-relaxed">
              Every entry recorded here will be deducted from your liquid
              sources to ensure total financial certainty.
            </p>
```

```
              {!isAdding && (
                <button
                  onClick={() => setIsAdding(true)}
                     className="mt-6 bg-foreground text-background px-6 py-2 rounded-xl text-[10px] font-black
uppercase tracking-widest hover:scale-105 active:scale-95 transition-all"
                >
                  Start New Entry
                </button>
              )}
            </div>

            {/* List of recent logs from sources */}
            {deployments.filter((d) => d.account_id).length > 0 && (
              <div className="space-y-3 mt-8">
                <h4 className="text-[10px] font-black text-foreground/40 uppercase tracking-[0.2em] ml-1">
                  Recent Verified Logs
                </h4>
                <div className="grid grid-cols-1 gap-3">
                  {deployments
                    .filter((d) => d.account_id)
                    .slice(0, 5)
                    .map((d) => (
                      <div
                        key={d.id}
                               className="bg-foreground/[0.03] border border-foreground/5 rounded-xl p-4 flex
items-center justify-between group hover:bg-foreground/[0.05] transition-all"
                      >
                        <div className="flex items-center gap-4">
                          <div className="w-10 h-10 bg-foreground/5 rounded-lg flex items-center justify-center
text-foreground/40 group-hover:bg-foreground group-hover:text-background transition-colors">
                            <svg
                              width="16"
                              height="16"
                              viewBox="0 0 24 24"
                              fill="none"
                              stroke="currentColor"
                              strokeWidth="3"
                            >
                              <path d="M12 2v20M17 5H9.5a3.5 3.5 0 0 0 0 7h5a3.5 3.5 0 0 1 0 7H6" />
                            </svg>
                          </div>
                          <div>
                                   <h5 className="text-xs font-black text-foreground uppercase tracking-tight
leading-none mb-1">
                              {d.title}
                            </h5>
                            <p className="text-[9px] font-bold text-foreground/40 uppercase tracking-widest">
                              {accounts.find((a) => a.id === d.account_id)
                                ?.account_name || "Source"}{" "}
                              ?{" "}
                              {isClient
                                ? new Date(d.created_at).toLocaleDateString()
                                : "--- --, ----"}
                            </p>
                          </div>
                        </div>
                        <div className="text-right">
                          <p className="text-sm font-black tabular-nums text-foreground">
                            {formatCurrency(d.amount)}
                          </p>
                              <span className="text-[8px] font-black text-green-600 uppercase tracking-tighter
bg-green-500/10 px-1.5 py-0.5 rounded">
                            Verified
                          </span>
```

```
                    </div>
                  </div>
                ))}
              </div>
            </div>
          )}
        </div>
      )}
    </div>
  </div>
  );
}
```

File: src/app/dashboard/BaselineSection.tsx

```
"use client";

import { useState } from "react";
import { TAXONOMY_CATEGORIES } from "@/lib/finance/taxonomy";
import {
  createOperationalBaselineAction,
  deleteOperationalBaselineAction,
  type DashboardSnapshot,
} from "./actions";
import { OperationalBaseline, BaselineCadence } from "@/lib/analytics/types";
import { formatCurrency } from "@/lib/utils/formatters";
import { DeploymentMap } from "@/lib/utils/taxonomy";

interface BaselineSectionProps {
  baseline: OperationalBaseline[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

const CADENCES = [
  { value: "daily", label: "Daily" },
  { value: "weekly", label: "Weekly" },
  { value: "biweekly", label: "Bi-weekly" },
  { value: "monthly", label: "Monthly" },
  { value: "quarterly", label: "Quarterly" },
  { value: "yearly", label: "Yearly" },
];

export function BaselineSection({
  baseline,
  onSnapshot,
}: BaselineSectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [form, setForm] = useState({
    title: "",
    amount: "",
    category: "Maintenance",
    cadence: "monthly" as BaselineCadence,
    baseline_type: "expense" as "expense" | "allocation",
    notes: "",
  });

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setIsLoading(true);
    setError(null);
```

```
    try {
      const snapshot = await createOperationalBaselineAction({
        title: form.title,
        amount: Number(form.amount),
        category: form.category,
        cadence: form.cadence,
        baseline_type: form.baseline_type,
        is_active: true,
        notes: form.notes || undefined,
      });
      setForm({
        title: "",
        amount: "",
        category: "Maintenance",
        cadence: "monthly",
        baseline_type: "expense",
        notes: "",
      });
      setIsAdding(false);
      onSnapshot(snapshot);
    } catch (err: unknown) {
      setError(
        err instanceof Error ? err.message : "Failed to create baseline entry",
      );
    } finally {
      setIsLoading(false);
    }
  }

  async function handleDelete(id: string) {
    if (
      !confirm(
        "Archive this baseline entry? It will no longer influence active liquidity projections but will remain
in historical telemetry.",
      )
    )
      return;
    setIsLoading(true);
    try {
      const snapshot = await deleteOperationalBaselineAction(id);
      onSnapshot(snapshot);
    } catch (err: unknown) {
      alert("Failed to archive entry");
    } finally {
      setIsLoading(false);
    }
  }

  return (
    <div className="space-y-6">
      <div className="flex items-center justify-between px-1">
        <h2 className="text-xl font-black text-foreground tracking-tight uppercase">
          Personal expenses
        </h2>
        <button
          onClick={() => setIsAdding(!isAdding)}
             className="flex items-center gap-2 px-3 py-1.5 bg-foreground/5 hover:bg-foreground/10 rounded-xl
transition-all group"
        >
                    <span  className="text-[10px]  font-black  uppercase  tracking-widest  text-foreground/60
group-hover:text-foreground">
            {isAdding ? "Cancel" : "+ Add recurring"}
          </span>
        </button>
```

```
      </div>

      {isAdding && (
        <div className="bg-background border-2 border-foreground rounded-3xl p-6 shadow-2xl animate-in fade-in
slide-in-from-top-4 duration-300">
          <form onSubmit={handleSubmit} className="space-y-4">
            <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Designation
                </label>
                <input
                  type="text"
                  required
                  placeholder="e.g. Rent, Crypto DCA"
                  value={form.title}
                  onChange={(e) => setForm({ ...form, title: e.target.value })}
                              className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
                />
              </div>
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Amount (KSh)
                </label>
                <input
                  type="number"
                  required
                  placeholder="0.00"
                  value={form.amount}
                  onChange={(e) => setForm({ ...form, amount: e.target.value })}
                              className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
                />
              </div>
            </div>

            <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Category
                </label>
                <select
                  value={form.category}
                  onChange={(e) =>
                    setForm({ ...form, category: e.target.value })
                  }
                              className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold appearance-none"
                >
                  {TAXONOMY_CATEGORIES.map((cat) => (
                    <option key={cat.value} value={cat.value}>
                      {cat.label}
                    </option>
                  ))}
                </select>
              </div>
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Cadence
```

```
                </label>
                <select
                  value={form.cadence}
                  onChange={(e) =>
                    setForm({
                      ...form,
                      cadence: e.target.value as BaselineCadence,
                    })
                  }
                                className="w-full border-2 border-foreground/10 bg-background rounded-xl p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold appearance-none"
                >
                  {CADENCES.map((c) => (
                    <option key={c.value} value={c.value}>
                      {c.label}
                    </option>
                  ))}
                </select>
              </div>
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Flow Type
                </label>
                <select
                  value={form.baseline_type}
                  onChange={(e) =>
                    setForm({
                      ...form,
                      baseline_type: e.target.value as "expense" | "allocation",
                    })
                  }
                                className="w-full border-2 border-foreground/10 bg-background rounded-xl p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold appearance-none"
                >
                  <option value="expense">Expense (Burn)</option>
                  <option value="allocation">Allocation (Investment)</option>
                </select>
              </div>
            </div>

            {error && (
              <p className="text-red-500 text-[10px] font-black uppercase tracking-tight ml-1">
                {error}
              </p>
            )}

            <button
              type="submit"
              disabled={isLoading}
                       className="w-full bg-foreground text-background py-3 rounded-xl font-black uppercase
tracking-widest hover:bg-foreground/90 transition-colors disabled:opacity-50"
            >
              {isLoading ? "CONFIGURING..." : "ESTABLISH FOUNDATION"}
            </button>
          </form>
        </div>
      )}

      <div className="grid grid-cols-1 gap-4">
        {baseline.length === 0 ? (
              <div className="border-2 border-dashed border-foreground/10 rounded-3xl p-12 text-center group
hover:border-foreground/20 transition-colors">
            <p className="text-foreground/60 text-xs font-bold uppercase tracking-widest">
```

```
              No structural flows defined.
            </p>
            <p className="text-foreground/40 text-[10px] mt-2 uppercase tracking-tight opacity-60">
              Define recurring expenses to track automatic burn.
            </p>
          </div>
        ) : (
          baseline.map((item) => (
            <div
              key={item.id}
                            className="bg-foreground/5  border  border-foreground/10  rounded-2xl  p-5  group
hover:bg-foreground/10 transition-all relative overflow-hidden"
            >
              <div className="flex items-center justify-between relative z-10">
                <div className="space-y-1">
                  <div className="flex items-center gap-2">
                    <span
                      className={`text-[8px] font-black px-1.5 py-0.5 rounded uppercase tracking-tighter ${
                        item.baseline_type === "expense"
                          ? "bg-orange-500/20 text-orange-600"
                          : "bg-emerald-500/20 text-emerald-600"
                      }`}
                    >
                      {item.baseline_type}
                    </span>
                         <span className="text-[9px] font-black text-background bg-foreground/30 px-1.5 py-0.5
rounded uppercase tracking-tighter">
                      {item.cadence}
                    </span>
                    <h3 className="text-sm font-black text-foreground uppercase tracking-tight">
                      {item.title}
                    </h3>
                  </div>
                  <p className="text-[10px] font-bold text-foreground/40 uppercase tracking-widest">
                    {DeploymentMap[
                      item.category as keyof typeof DeploymentMap
                    ] || item.category}
                  </p>
                </div>
                <div className="flex items-center gap-4">
                  <div className="text-right">
                    <p className="text-lg font-black tabular-nums text-foreground">
                      {formatCurrency(item.amount)}
                    </p>
                             <p  className="text-[8px]  font-black  text-foreground/60  uppercase  tracking-widest
opacity-60">
                      Amount per {item.cadence.replace("ly", "")}
                    </p>
                  </div>
                  <button
                    onClick={() => handleDelete(item.id)}
                             className="p-2  text-foreground/40  hover:text-red-500  transition-colors  opacity-0
group-hover:opacity-100"
                  >
                    <svg
                      width="14"
                      height="14"
                      viewBox="0 0 24 24"
                      fill="none"
                      stroke="currentColor"
                      strokeWidth="3"
                    >
                       <path d="M3 6h18M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0 1 2-2h4a2 2 0 0 1
2 2v2" />
```

```
                  </svg>
                </button>
              </div>
            </div>
          </div>
        ))
      )}
    </div>
  </div>
 );
}
```

File: src/app/dashboard/GoalSection.tsx

```
"use client";

import { useState } from "react";
import {
  GOAL_TYPES,
  GOAL_PRIORITIES,
  GOAL_STATUSES,
  calculateGoalProgressPercentage,
  type FinancialGoal,
} from "@/lib/finance/goals";
import {
  createGoalAction,
  deleteGoalAction,
  type DashboardSnapshot,
} from "./actions";
import { formatCurrency } from "@/lib/utils/formatters";

interface GoalSectionProps {
  goals: FinancialGoal[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function GoalSection({ goals, onSnapshot }: GoalSectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [form, setForm] = useState({
    goal_name: "",
    goal_type: "emergency_fund",
    target_amount: "",
    current_progress: "",
    priority: "medium",
    status: "active",
    notes: "",
  });

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setIsLoading(true);
    setError(null);
    try {
      const snapshot = await createGoalAction({
        goal_name: form.goal_name,
        goal_type: form.goal_type,
        target_amount: Number(form.target_amount),
        current_progress: form.current_progress
          ? Number(form.current_progress)
          : 0,
```

```
        priority: form.priority,
        status: form.status,
        notes: form.notes || undefined,
      });
      setForm({
        goal_name: "",
        goal_type: "emergency_fund",
        target_amount: "",
        current_progress: "",
        priority: "medium",
        status: "active",
        notes: "",
      });
      setIsAdding(false);
      onSnapshot(snapshot);
    } catch (err: unknown) {
      setError(
        err instanceof Error ? err.message : "Failed to record financial goal",
      );
    } finally {
      setIsLoading(false);
    }
  }

  async function handleDelete(id: string) {
    if (
      !confirm(
          "Archive this financial goal? It will be removed from active tracking but retained in historical
data.",
      )
    )
      return;
    setIsLoading(true);
    try {
      const snapshot = await deleteGoalAction(id);
      onSnapshot(snapshot);
    } catch {
      alert("Failed to archive goal");
    } finally {
      setIsLoading(false);
    }
  }

  return (
    <div className="space-y-6">
      <div className="flex items-center justify-between px-1">
        <h2 className="text-xl font-black text-foreground tracking-tight uppercase">
          Financial Goals
        </h2>
        <button
          onClick={() => setIsAdding(!isAdding)}
             className="flex items-center gap-2 px-3 py-1.5 bg-foreground/5 hover:bg-foreground/10 rounded-xl
transition-all group"
        >
                    <span  className="text-[10px]  font-black  uppercase  tracking-widest  text-foreground/60
group-hover:text-foreground">
            {isAdding ? "Cancel" : "+ Add goal"}
          </span>
        </button>
      </div>

      {isAdding && (
        <div className="bg-background border-2 border-foreground rounded-3xl p-6 shadow-2xl animate-in fade-in
slide-in-from-top-4 duration-300">
```

```
          <form onSubmit={handleSubmit} className="space-y-4">
            <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Goal Designation
                </label>
                <input
                  type="text"
                  required
                  placeholder="e.g. 6-Month Emergency Fund"
                  value={form.goal_name}
                  onChange={(e) =>
                    setForm({ ...form, goal_name: e.target.value })
                  }
                                  className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
                />
              </div>
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Target Capital (KSh)
                </label>
                <input
                  type="number"
                  required
                  placeholder="0.00"
                  value={form.target_amount}
                  onChange={(e) =>
                    setForm({ ...form, target_amount: e.target.value })
                  }
                                  className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
                />
              </div>
            </div>

            <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Strategy Type
                </label>
                <select
                  value={form.goal_type}
                  onChange={(e) =>
                    setForm({ ...form, goal_type: e.target.value })
                  }
                                  className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold appearance-none"
                >
                  {GOAL_TYPES.map((t) => (
                    <option key={t.value} value={t.value}>
                      {t.label}
                    </option>
                  ))}
                </select>
              </div>
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Priority
                </label>
```

```
                <select
                  value={form.priority}
                  onChange={(e) =>
                    setForm({ ...form, priority: e.target.value })
                  }
                                className="w-full border-2 border-foreground/10 bg-background rounded-xl p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold appearance-none"
                >
                  {GOAL_PRIORITIES.map((p) => (
                    <option key={p.value} value={p.value}>
                      {p.label}
                    </option>
                  ))}
                </select>
              </div>
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Current Inception
                </label>
                <input
                  type="number"
                  placeholder="0.00"
                  value={form.current_progress}
                  onChange={(e) =>
                    setForm({ ...form, current_progress: e.target.value })
                  }
                                className="w-full border-2 border-foreground/10 bg-background rounded-xl p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
                />
              </div>
            </div>

            {error && (
              <p className="text-red-500 text-[10px] font-black uppercase tracking-tight ml-1">
                {error}
              </p>
            )}

            <button
              type="submit"
              disabled={isLoading}
                        className="w-full bg-foreground text-background py-3 rounded-xl font-black uppercase
tracking-widest hover:bg-foreground/90 transition-colors disabled:opacity-50"
            >
              {isLoading ? "INITIALIZING..." : "INITIALIZE GOAL"}
            </button>
          </form>
        </div>
      )}

      <div className="grid grid-cols-1 gap-4">
        {goals.length === 0 ? (
              <div className="border-2 border-dashed border-foreground/10 rounded-3xl p-12 text-center group
hover:border-foreground/20 transition-colors">
            <p className="text-foreground/60 text-xs font-bold uppercase tracking-widest">
              No financial goals defined.
            </p>
            <p className="text-foreground/40 text-[10px] mt-2 uppercase tracking-tight opacity-60">
              Capital currently operating without specific achievement targets.
            </p>
          </div>
        ) : (
          goals.map((goal) => {
```

```
            const progress = calculateGoalProgressPercentage(goal);
            return (
              <div
                key={goal.id}
                            className="bg-foreground/5  border  border-foreground/10  rounded-2xl  p-5  group
hover:bg-foreground/10 transition-all relative overflow-hidden"
              >
                <div className="flex items-center justify-between relative z-10 mb-4">
                  <div className="space-y-1">
                    <div className="flex items-center gap-2">
                      <span
                             className={`text-[8px] font-black text-background px-1.5 py-0.5 rounded uppercase
tracking-tighter ${
                               goal.priority === "critical"
                                 ? "bg-orange-500"
                                 : goal.priority === "high"
                                   ? "bg-foreground/60"
                                   : "bg-foreground/30"
                             }`}
                      >
                        {goal.priority}
                      </span>
                      <h3 className="text-sm font-black text-foreground uppercase tracking-tight">
                        {goal.goal_name}
                      </h3>
                    </div>
                    <p className="text-[10px] font-bold text-foreground/60 uppercase tracking-widest">
                      {goal.goal_type.replace("_", " ").toUpperCase()} ?{" "}
                      {goal.status}
                    </p>
                  </div>
                  <div className="flex items-center gap-4">
                    <div className="text-right">
                      <p className="text-lg font-black tabular-nums text-foreground">
                        {Math.round(progress)}%
                      </p>
                             <p className="text-[8px] font-black text-foreground/60 uppercase tracking-widest
opacity-60">
                        Goal Fulfullment
                      </p>
                    </div>
                    <button
                      onClick={() => handleDelete(goal.id)}
                             className="p-2 text-foreground/40 hover:text-red-500 transition-colors opacity-0
group-hover:opacity-100"
                    >
                      <svg
                        width="14"
                        height="14"
                        viewBox="0 0 24 24"
                        fill="none"
                        stroke="currentColor"
                        strokeWidth="3"
                      >
                         <path d="M3 6h18M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0 1 2-2h4a2 2 0 0
1 2 2v2" />
                      </svg>
                    </button>
                  </div>
                </div>

                {/* Deterministic Progress Track */}
                <div className="h-1.5 w-full bg-foreground/5 rounded-full overflow-hidden">
                  <div
```

```
                  className={`h-full transition-all duration-1000 ease-out ${
                    progress >= 100 ? "bg-green-500" : "bg-foreground/40"
                  }`}
                  style={{ width: `${progress}%` }}
                />
              </div>
              <div className="flex justify-between mt-2">
                <p className="text-[9px] font-black text-foreground/40 uppercase tracking-widest">
                  {formatCurrency(goal.current_progress)}
                </p>
                <p className="text-[9px] font-black text-foreground/40 uppercase tracking-widest">
                  Target: {formatCurrency(goal.target_amount)}
                </p>
              </div>
            </div>
          );
        })
      )}
    </div>
  </div>
);
}
```

File: src/app/dashboard/HistoricalAudit.tsx

```
"use client";

import React, { useEffect, useState } from "react";
import { fetchHistoricalAuditAction } from "./actions";
import { AuditRecord } from "@/lib/db/audit";
import { formatCurrency } from "@/lib/utils/formatters";

export function HistoricalAudit() {
  const [auditRecords, setAuditRecords] = useState<AuditRecord[]>([]);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    async function loadAudit() {
      try {
        const records = await fetchHistoricalAuditAction();
        setAuditRecords(records);
      } catch (err) {
        console.error("Failed to fetch audit history:", err);
        setError("Forensic layer synchronization failure.");
      } finally {
        setIsLoading(false);
      }
    }
    loadAudit();
  }, []);

  if (isLoading) {
    return (
      <div className="p-6 text-center animate-pulse">
        <p className="text-[10px] font-black uppercase tracking-[0.3em] text-foreground/20">
          Synchronizing Forensic Layer...
        </p>
      </div>
    );
  }

  if (error) {
```

```
    return (
      <div className="p-6 text-center">
        <p className="text-[10px] font-black uppercase tracking-[0.3em] text-red-500/40">
          {error}
        </p>
      </div>
    );
  }

  if (auditRecords.length === 0) {
    return (
      <div className="p-8 text-center border-2 border-dashed border-foreground/5 rounded-3xl">
        <p className="text-[10px] font-black uppercase tracking-[0.3em] text-foreground/20">
          No audit history found
        </p>
      </div>
    );
  }

  return (
    <div className="space-y-6">
      <div className="flex items-center gap-3 opacity-20">
        <h3 className="text-[11px] font-black uppercase tracking-[0.2em] text-foreground">
          Historical Audit Trail
        </h3>
        <div className="h-0.5 flex-1 bg-foreground/10"></div>
      </div>

      <div className="grid grid-cols-1 gap-3">
        {auditRecords.map((record) => (
          <div
            key={`${record.type}-${record.id}`}
               className="group relative bg-foreground/[0.02] border border-foreground/5 rounded-2xl p-4 flex
items-center justify-between transition-colors hover:bg-foreground/[0.04]"
          >
            <div className="flex items-center gap-4">
               <div className="w-10 h-10 bg-foreground/5 rounded-xl flex items-center justify-center grayscale
opacity-40">
                <svg
                  width="18"
                  height="18"
                  viewBox="0 0 24 24"
                  fill="none"
                  stroke="currentColor"
                  strokeWidth="2.5"
                  className="text-foreground"
                >
                  {record.type === "deployment" && (
                    <path d="M12 2v20M17 5H9.5a3.5 3.5 0 0 0 0 7h5a3.5 3.5 0 0 1 0 7H6" />
                  )}
                  {record.type === "income" && (
                    <path d="M12 20V10M18 20V4M6 20v-4" />
                  )}
                  {record.type === "liability" && (
                    <path d="M12 22s8-4 8-10V5l-8-3-8 3v7c0 6 8 10 8 10z" />
                  )}
                </svg>
              </div>
              <div>
                <div className="flex items-center gap-2 mb-0.5">
                  <h4 className="font-bold text-sm text-foreground/40 line-through decoration-foreground/20">
                    {record.name}
                  </h4>
                     <span className="text-[8px] font-black px-2 py-0.5 bg-foreground/5 rounded-full uppercase
```

```
tracking-tighter text-foreground/30">
                    {record.type}
                  </span>
                </div>
                <div className="text-[9px] font-bold text-foreground/30 uppercase tracking-tighter">
                  Deleted:{" "}
                  {new Date(record.deleted_at).toLocaleDateString(undefined, {
                    month: "short",
                    day: "numeric",
                    year: "numeric",
                  })}
                </div>
              </div>
            </div>
            <div className="text-right">
                            <p  className="font-black  text-sm  tabular-nums  text-foreground/30  line-through
decoration-foreground/10">
                {formatCurrency(record.amount)}
                {record.cadence && (
                  <span className="text-[9px] ml-1 uppercase">
                    / {record.cadence}
                  </span>
                )}
              </p>
              <p className="text-[8px] font-black text-foreground/20 uppercase tracking-widest mt-0.5">
                Archived
              </p>
            </div>
          </div>
        ))}
      </div>

      <p className="text-center text-[9px] font-black text-foreground/10 uppercase tracking-[0.2em] mt-4">
        Forensic view only :: Data immutable
      </p>
    </div>
  );
}
```

File: src/app/dashboard/IncomeSection.tsx

```
"use client";

import { useState } from "react";
import {
  INCOME_TYPES,
  CADENCES,
  type IncomeStream,
  type IncomeType,
  type Cadence,
} from "@/lib/finance/income";
import { calculateMonthlyInflow } from "@/lib/finance/income";
import {
  createIncomeAction,
  deleteIncomeAction,
  type DashboardSnapshot,
  type CreateIncomeActionInput,
} from "./actions";
import { formatCurrency } from "@/lib/utils/formatters";
import { IncomeMap } from "@/lib/utils/taxonomy";

interface IncomeSectionProps {
  incomeStreams: IncomeStream[];
```

```
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

interface FormEntry {
  income_name: string;
  income_type: IncomeType;
  amount: string;
  cadence: Cadence;
  execution_day: number;
  is_recurring: boolean;
  source: string;
}

export function IncomeSection({
  incomeStreams,
  onSnapshot,
}: IncomeSectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [entries, setEntries] = useState<FormEntry[]>([
    {
      income_name: "",
      income_type: "salary",
      amount: "",
      cadence: "monthly",
      execution_day: 1,
      is_recurring: true,
      source: "",
    },
  ]);

  const addEntry = () => {
    setEntries([
      ...entries,
      {
        income_name: "",
        income_type: "salary",
        amount: "",
        cadence: "monthly",
        execution_day: 1,
        is_recurring: true,
        source: "",
      },
    ]);
  };

  const removeEntry = (index: number) => {
    if (entries.length === 1) return;
    setEntries(entries.filter((_, i) => i !== index));
  };

  const updateEntry = <K extends keyof FormEntry>(
    index: number,
    field: K,
    value: FormEntry[K],
  ) => {
    const newEntries = [...entries];
    newEntries[index][field] = value;
    setEntries(newEntries);
  };

  async function handleSubmit(e: React.FormEvent) {
```

```
  e.preventDefault();
  setIsLoading(true);
  setError(null);
  try {
    const snapshot = await createIncomeAction(
      entries.map((entry) => ({
        ...entry,
        amount: Number(entry.amount),
        execution_day:
          entry.cadence === "monthly" ||
          entry.cadence === "weekly" ||
          entry.cadence === "biweekly"
            ? Number(entry.execution_day)
            : null,
        source: entry.source || undefined,
      })),
    );
    setEntries([
      {
        income_name: "",
        income_type: "salary",
        amount: "",
        cadence: "monthly",
        execution_day: 1,
        is_recurring: true,
        source: "",
      },
    ]);
    setIsAdding(false);
    onSnapshot(snapshot);
  } catch (err: unknown) {
    setError(
      err instanceof Error ? err.message : "Failed to record income streams",
    );
  } finally {
    setIsLoading(false);
  }
}

async function handleDelete(id: string) {
  if (
    !confirm(
      "Archive this replenishment source? Historical inflow data will be preserved.",
    )
  )
    return;
  setIsLoading(true);
  try {
    const snapshot = await deleteIncomeAction(id);
    onSnapshot(snapshot);
  } catch {
    alert("Failed to archive income stream");
  } finally {
    setIsLoading(false);
  }
}

return (
  <div className="space-y-6">
    <div className="flex items-center justify-between px-1">
      <h2 className="text-xl font-black text-foreground tracking-tight uppercase">
        Inflows
      </h2>
      <button
```

```
          onClick={() => setIsAdding(!isAdding)}
             className="flex items-center gap-2 px-3 py-1.5 bg-foreground/5 hover:bg-foreground/10 rounded-xl
transition-all group"
        >
                      <span  className="text-[10px]  font-black  uppercase  tracking-widest  text-foreground/60
group-hover:text-foreground">
            {isAdding ? "Cancel" : "+ Add inflow source"}
          </span>
        </button>
      </div>

      {isAdding && (
        <div className="bg-background border-2 border-foreground rounded-3xl p-6 shadow-2xl animate-in fade-in
slide-in-from-top-4 duration-300">
          <form onSubmit={handleSubmit} className="space-y-8">
            {entries.map((entry, index) => (
              <div key={index} className="space-y-4 relative">
                {index > 0 && (
                  <div className="border-t-2 border-foreground/10 pt-8" />
                )}
                <div className="flex items-center justify-between mb-2">
                  <span className="text-[10px] font-black uppercase tracking-widest text-foreground/40">
                    Source #{index + 1}
                  </span>
                  {entries.length > 1 && (
                    <button
                      type="button"
                      onClick={() => removeEntry(index)}
                                  className="text-[10px] font-black uppercase tracking-widest text-red-500/60
hover:text-red-500 transition-colors"
                      >
                      Remove
                    </button>
                  )}
                </div>

                <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
                  <div>
                        <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest
mb-1.5 block ml-1">
                      Inflow Name
                    </label>
                    <input
                      type="text"
                      required
                      placeholder="e.g. Primary Salary"
                      value={entry.income_name}
                      onChange={(e) =>
                        updateEntry(index, "income_name", e.target.value)
                      }
                                 className="w-full border-2 border-foreground/10 bg-background rounded-xl p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
                    />
                  </div>
                  <div>
                        <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest
mb-1.5 block ml-1">
                      Amount (per cadence)
                    </label>
                    <input
                      type="number"
                      required
                      placeholder="0.00"
                      value={entry.amount}
```

```
                    onChange={(e) =>
                      updateEntry(index, "amount", e.target.value)
                    }
                              className="w-full border-2 border-foreground/10 bg-background rounded-xl p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
                    />
                  </div>
                </div>

                <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
                  <div>
                       <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest
mb-1.5 block ml-1">
                      Type
                    </label>
                    <select
                      value={entry.income_type}
                      onChange={(e) =>
                        updateEntry(
                          index,
                          "income_type",
                          e.target.value as IncomeType,
                        )
                      }
                              className="w-full border-2 border-foreground/10 bg-background rounded-xl p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold appearance-none"
                    >
                      {INCOME_TYPES.map((t) => (
                        <option key={t.value} value={t.value}>
                          {t.label}
                        </option>
                      ))}
                    </select>
                  </div>
                  <div>
                       <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest
mb-1.5 block ml-1">
                      Cadence
                    </label>
                    <div className="flex gap-2">
                      <select
                        value={entry.cadence}
                        onChange={(e) =>
                          updateEntry(
                            index,
                            "cadence",
                            e.target.value as Cadence,
                          )
                        }
                               className="flex-1 border-2 border-foreground/10 bg-background rounded-xl p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold appearance-none"
                      >
                        {CADENCES.map((c) => (
                          <option key={c.value} value={c.value}>
                            {c.label}
                          </option>
                        ))}
                      </select>

                      {(entry.cadence === "weekly" ||
                        entry.cadence === "monthly" ||
                        entry.cadence === "biweekly") && (
                        <select
                          value={entry.execution_day}
```

```
                        onChange={(e) =>
                          updateEntry(
                            index,
                            "execution_day",
                            Number(e.target.value),
                          )
                        }
                                  className="w-20 border-2 border-foreground/10 bg-background rounded-xl p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold appearance-none"
                      >
                        {entry.cadence === "weekly" ? (
                          <>
                            <option value={1}>Mon</option>
                            <option value={2}>Tue</option>
                            <option value={3}>Wed</option>
                            <option value={4}>Thu</option>
                            <option value={5}>Fri</option>
                            <option value={6}>Sat</option>
                            <option value={7}>Sun</option>
                          </>
                        ) : (
                          Array.from({ length: 31 }, (_, i) => (
                            <option key={i + 1} value={i + 1}>
                              {i + 1}
                            </option>
                          ))
                        )}
                      </select>
                    )}
                  </div>
                </div>
                <div classNamе="flex flex-col justify-center pl-2">
                  <label classNamе="flex items-center gap-2 cursor-pointer group">
                    <input
                      type="checkbox"
                      checked={entry.is_recurring}
                      onChange={(e) =>
                        updateEntry(index, "is_recurring", e.target.checked)
                      }
                      classNamе="w-4 h-4 rounded border-2 border-foreground accent-foreground"
                    />
                    <span classNamе="text-[10px] font-black text-foreground/60 uppercase tracking-widest">
                      Recurring Inflow
                    </span>
                  </label>
                </div>
              </div>

              <div>
                 <label classNamе="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Source (Optional)
                </label>
                <input
                  type="text"
                  placeholder="e.g. Acme Corp"
                  value={entry.source}
                  onChange={(e) =>
                    updateEntry(index, "source", e.target.value)
                  }
                              classNamе="w-full border-2 border-foreground/10 bg-background rounded-xl p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
                />
              </div>
```

```
              </div>
            ))}

            <div className="flex flex-col gap-4">
              <button
                type="button"
                onClick={addEntry}
                    className="w-full border-2 border-dashed border-foreground/20 py-3 rounded-xl text-[10px] font-black uppercase tracking-widest text-foreground/60 hover:border-foreground/40 hover:text-foreground transition-all"
              >
                + Add another income source
              </button>

              {error && (
                <p className="text-red-500 text-[10px] font-black uppercase tracking-tight ml-1">
                  {error}
                </p>
              )}

              <button
                type="submit"
                disabled={isLoading}
                        className="w-full bg-foreground text-background py-3 rounded-xl font-black uppercase tracking-widest hover:bg-foreground/90 transition-colors disabled:opacity-50"
              >
                {isLoading ? "ACKNOWLEDGING..." : "VERIFY ALL INFLOWS"}
              </button>
            </div>
          </form>
        </div>
      )}

      <div className="grid grid-cols-1 gap-4">
        {incomeStreams.length === 0 ? (
              <div className="border-2 border-dashed border-foreground/10 rounded-3xl p-12 text-center group hover:border-foreground/20 transition-colors">
            <p className="text-foreground/60 text-xs font-bold uppercase tracking-widest">
              No replenishment sources recorded.
            </p>
            <p className="text-foreground/40 text-[10px] mt-2 uppercase tracking-tight opacity-60">
              Add recurring inflows to improve runway accuracy.
            </p>
          </div>
        ) : (
          incomeStreams.map((stream) => (
            <div
              key={stream.id}
                            className="bg-foreground/5 border border-foreground/10 rounded-2xl p-5 group hover:bg-foreground/10 transition-all relative overflow-hidden"
            >
              <div className="flex items-center justify-between relative z-10">
                <div className="space-y-1">
                  <div className="flex items-center gap-2">
                        <span className="text-[9px] font-black text-background bg-foreground/30 px-1.5 py-0.5 rounded uppercase tracking-tighter">
                      {IncomeMap[stream.income_type] || stream.income_type}
                    </span>
                    <h3 className="text-sm font-black text-foreground uppercase tracking-tight">
                      {stream.income_name}
                    </h3>
                  </div>
                  <div className="flex gap-3">
                    <p className="text-[10px] font-bold text-foreground/60 uppercase tracking-widest">
```

```
                    {stream.cadence} ?{" "}
                    {stream.is_recurring ? "Recurring" : "One-off"}
                  </p>
                  {stream.source && (
                    <p className="text-[10px] font-black text-foreground/40 uppercase tracking-widest">
                      Via {stream.source}
                    </p>
                  )}
                </div>
              </div>
              <div className="flex items-center gap-4">
                <div className="text-right">
                  <p className="text-lg font-black tabular-nums text-foreground">
                    {formatCurrency(stream.amount)}
                    <span className="text-[10px] ml-1 text-foreground/60 uppercase">
                      /{" "}
                      {CADENCES.find((c) => c.value === stream.cadence)
                        ?.label || stream.cadence}
                    </span>
                  </p>
                  {stream.cadence !== "monthly" && (
                    <p className="text-[8px] font-black text-foreground/40 uppercase tracking-widest mt-0.5">
                      ? {formatCurrency(calculateMonthlyInflow(stream))}{" "}
                      Monthly
                    </p>
                  )}
                </div>
                <button
                  onClick={() => handleDelete(stream.id)}
                            className="p-2 text-foreground/40 hover:text-red-500 transition-colors opacity-0
group-hover:opacity-100"
                >
                  <svg
                    width="14"
                    height="14"
                    viewBox="0 0 24 24"
                    fill="none"
                    stroke="currentColor"
                    strokeWidth="3"
                  >
                     <path d="M3 6h18M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0 1 2-2h4a2 2 0 0 1
2 2v2" />
                  </svg>
                </button>
              </div>
            </div>
          </div>
        ))
      )}
    </div>
  </div>
  );
}
```

File: src/app/dashboard/LiabilitySection.tsx

```
"use client";

import { useState } from "react";
import { LIABILITY_TYPES, type Liability } from "@/lib/finance/liabilities";
import {
  createLiabilityAction,
  deleteLiabilityAction,
```

```
  type DashboardSnapshot,
} from "./actions";
import { formatCurrency } from "@/lib/utils/formatters";
import { LiabilityMap } from "@/lib/utils/taxonomy";

interface LiabilitySectionProps {
  liabilities: Liability[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function LiabilitySection({
  liabilities,
  onSnapshot,
}: LiabilitySectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [form, setForm] = useState({
    liability_name: "",
    liability_type: "credit_card",
    outstanding_balance: "",
    interest_rate: "",
    minimum_payment: "",
    institution: "",
  });

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setIsLoading(true);
    setError(null);
    try {
      const snapshot = await createLiabilityAction({
        liability_name: form.liability_name,
        liability_type: form.liability_type,
        outstanding_balance: Number(form.outstanding_balance),
        interest_rate: form.interest_rate ? Number(form.interest_rate) : 0,
        minimum_payment: form.minimum_payment
          ? Number(form.minimum_payment)
          : 0,
        institution: form.institution || undefined,
      });
      setForm({
        liability_name: "",
        liability_type: "credit_card",
        outstanding_balance: "",
        interest_rate: "",
        minimum_payment: "",
        institution: "",
      });
      setIsAdding(false);
      onSnapshot(snapshot);
    } catch (err: unknown) {
      setError(
        err instanceof Error ? err.message : "Failed to record obligation",
      );
    } finally {
      setIsLoading(false);
    }
  }

  async function handleDelete(id: string) {
    if (
      !confirm(
```

```
        "Archive this obligation? If it has been settled, this will preserve the resolution record.",
      )
    )
      return;
    setIsLoading(true);
    try {
      const snapshot = await deleteLiabilityAction(id);
      onSnapshot(snapshot);
    } catch {
      alert("Failed to archive obligation");
    } finally {
      setIsLoading(false);
    }
  }

  return (
    <div className="space-y-6">
      <div className="flex items-center justify-between px-1">
        <h2 className="text-xl font-black text-foreground tracking-tight uppercase">
          Liabilities
        </h2>
        <button
          onClick={() => setIsAdding(!isAdding)}
             className="flex items-center gap-2 px-3 py-1.5 bg-foreground/5 hover:bg-foreground/10 rounded-xl
transition-all group"
        >
                     <span  className="text-[10px]  font-black  uppercase  tracking-widest  text-foreground/60
group-hover:text-foreground">
            {isAdding ? "Cancel" : "+ Add liabilities"}
          </span>
        </button>
      </div>

      {isAdding && (
         <div className="bg-background border-2 border-foreground rounded-3xl p-6 shadow-2xl animate-in fade-in
slide-in-from-top-4 duration-300">
          <form onSubmit={handleSubmit} className="space-y-4">
            <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Commitment Name
                </label>
                <input
                  type="text"
                  required
                  placeholder="e.g. Equity Bank Loan"
                  value={form.liability_name}
                  onChange={(e) =>
                    setForm({ ...form, liability_name: e.target.value })
                  }
                               className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
                />
              </div>
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Outstanding Balance (KSh)
                </label>
                <input
                  type="number"
                  required
                  placeholder="0.00"
```

```
                value={form.outstanding_balance}
                onChange={(e) =>
                  setForm({ ...form, outstanding_balance: e.target.value })
                }
                            className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
              />
            </div>
          </div>

          <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
            <div>
                <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                Type
              </label>
              <select
                value={form.liability_type}
                onChange={(e) =>
                  setForm({ ...form, liability_type: e.target.value })
                }
                            className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold appearance-none"
              >
                {LIABILITY_TYPES.map((t) => (
                  <option key={t.value} value={t.value}>
                    {t.label}
                  </option>
                ))}
              </select>
            </div>
            <div>
                <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                Min. Payment
              </label>
              <input
                type="number"
                placeholder="0.00"
                value={form.minimum_payment}
                onChange={(e) =>
                  setForm({ ...form, minimum_payment: e.target.value })
                }
                            className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
              />
            </div>
            <div>
                <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                Interest (%)
              </label>
              <input
                type="number"
                step="0.01"
                placeholder="0.00"
                value={form.interest_rate}
                onChange={(e) =>
                  setForm({ ...form, interest_rate: e.target.value })
                }
                            className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
              />
            </div>
```

```
            </div>

            <div>
                 <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                Institution (Optional)
              </label>
              <input
                type="text"
                placeholder="e.g. NCBA Bank"
                value={form.institution}
                onChange={(e) =>
                  setForm({ ...form, institution: e.target.value })
                }
                className="w-full border-2 border-foreground/10 bg-background rounded-xl p-3 focus:outline-none
focus:border-foreground transition-colors text-foreground text-sm font-bold"
              />
            </div>

            {error && (
              <p className="text-red-500 text-[10px] font-black uppercase tracking-tight ml-1">
                {error}
              </p>
            )}

            <button
              type="submit"
              disabled={isLoading}
                       className="w-full bg-foreground text-background py-3 rounded-xl font-black uppercase
tracking-widest hover:bg-foreground/90 transition-colors disabled:opacity-50"
            >
              {isLoading ? "ACKNOWLEDGING..." : "VERIFY COMMITMENT"}
            </button>
          </form>
        </div>
      )}

      <div className="grid grid-cols-1 gap-4">
        {liabilities.length === 0 ? (
              <div className="border-2 border-dashed border-foreground/10 rounded-3xl p-12 text-center group
hover:border-foreground/20 transition-colors">
            <p className="text-foreground/60 text-xs font-bold uppercase tracking-widest">
              &amp;quot;No commitments recorded.&amp;quot;
            </p>
            <p className="text-foreground/40 text-[10px] mt-2 uppercase tracking-tight opacity-60">
              Operational capital is currently un-obligated.
            </p>
          </div>
        ) : (
          liabilities.map((liability) => (
            <div
              key={liability.id}
                            className="bg-foreground/5 border border-foreground/10 rounded-2xl p-5 group
hover:bg-foreground/10 transition-all relative overflow-hidden"
            >
              <div className="flex items-center justify-between relative z-10">
                <div className="space-y-1">
                  <div className="flex items-center gap-2">
                        <span className="text-[9px] font-black text-background bg-foreground/30 px-1.5 py-0.5
rounded uppercase tracking-tighter">
                      {LiabilityMap[liability.liability_type] ||
                        liability.liability_type}
                    </span>
                    <h3 className="text-sm font-black text-foreground uppercase tracking-tight">
```

```
                    {liability.liability_name}
                  </h3>
                </div>
                <div className="flex gap-3">
                  {liability.institution && (
                    <p className="text-[10px] font-bold text-foreground/60 uppercase tracking-widest">
                      {liability.institution}
                    </p>
                  )}
                  {liability.interest_rate > 0 && (
                    <p className="text-[10px] font-black text-red-500/60 uppercase tracking-widest">
                      {liability.interest_rate}% APR
                    </p>
                  )}
                </div>
              </div>
              <div className="flex items-center gap-4">
                <div className="text-right">
                  <p className="text-lg font-black tabular-nums text-foreground">
                    {formatCurrency(liability.outstanding_balance)}
                  </p>
                          <p  className="text-[8px]  font-black  text-foreground/60  uppercase  tracking-widest
opacity-60">
                    Left to pay
                  </p>
                </div>
                <button
                  onClick={() => handleDelete(liability.id)}
                           className="p-2  text-foreground/40  hover:text-red-500  transition-colors  opacity-0
group-hover:opacity-100"
                >
                  <svg
                    width="14"
                    height="14"
                    viewBox="0 0 24 24"
                    fill="none"
                    stroke="currentColor"
                    strokeWidth="3"
                  >
                     <path d="M3 6h18M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0 1 2-2h4a2 2 0 0 1
2 2v2" />
                  </svg>
                </button>
              </div>
            </div>
          </div>
        ))
      )}
    </div>
  </div>
  );
}
```

File: src/app/dashboard/PendingInflows.tsx

```
"use client";

import { useState } from "react";
import type { IncomeStream, Account } from "@/lib/analytics/types";
import { resolvePendingInflowAction, type DashboardSnapshot } from "./actions";
import { formatCurrency } from "@/lib/utils/formatters";
import { IncomeMap } from "@/lib/utils/taxonomy";
```

```
interface PendingInflowsProps {
  incomeStreams: IncomeStream[];
  accounts: Account[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function PendingInflows({
  incomeStreams,
  accounts,
  onSnapshot,
}: PendingInflowsProps) {
  const [loadingId, setLoadingId] = useState<string | null>(null);
  const [selectedAccounts, setSelectedAccounts] = useState<Record<string, string>>({});

  const today = new Date();
  const currentDayOfWeek = today.getDay(); // 0 (Sun) to 6 (Sat)
  const currentDayOfMonth = today.getDate();

  // Map JS getDay (0-6) to our 1-7 (Mon-Sun) if necessary
  // Standard getDay: 0=Sun, 1=Mon, 2=Tue, 3=Wed, 4=Thu, 5=Fri, 6=Sat
  // Our Weekly selection used 1=Mon, ..., 7=Sun
  const mappedDayOfWeek = currentDayOfWeek === 0 ? 7 : currentDayOfWeek;

  const pendingInflows = incomeStreams.filter((stream) => {
    if (!stream.is_recurring || !stream.execution_day) return false;

    // Check if last_executed_at is today
    if (stream.last_executed_at) {
      const lastExecuted = new Date(stream.last_executed_at);
      if (
        lastExecuted.getDate() === today.getDate() &&
        lastExecuted.getMonth() === today.getMonth() &&
        lastExecuted.getFullYear() === today.getFullYear()
      ) {
        return false;
      }
    }

    if (stream.cadence === "weekly") {
      return mappedDayOfWeek === stream.execution_day;
    }
    if (stream.cadence === "monthly" || stream.cadence === "biweekly") {
      return currentDayOfMonth === stream.execution_day;
    }

    return false;
  });

  if (pendingInflows.length === 0) return null;

  async function handleVerify(stream: IncomeStream) {
    const accountId = selectedAccounts[stream.id];
    if (!accountId) {
      alert("Please select a destination account.");
      return;
    }

    setLoadingId(stream.id);
    try {
      const snapshot = await resolvePendingInflowAction({
        incomeId: stream.id,
        accountId,
        amount: stream.amount,
      });
```

```
      onSnapshot(snapshot);
    } catch (err: unknown) {
      alert(err instanceof Error ? err.message : "Verification failed");
    } finally {
      setLoadingId(null);
    }
  }

  return (
    <div className="space-y-4 animate-in fade-in slide-in-from-top-4 duration-500">
      <div className="flex items-center gap-3">
        <div className="h-0.5 w-8 bg-orange-500/20"></div>
        <h2 className="text-[10px] font-black text-orange-500/60 uppercase tracking-[0.3em]">
          Verification Required
        </h2>
      </div>

      {pendingInflows.map((stream) => (
        <div
          key={stream.id}
           className="bg-background border-2 border-foreground rounded-2xl p-4 md:p-6 shadow-xl flex flex-col
md:flex-row md:items-center justify-between gap-6"
        >
          <div className="flex items-center gap-4">
            <div className="w-10 h-10 bg-foreground text-background rounded-xl flex items-center justify-center
shrink-0">
              <svg
                width="20"
                height="20"
                viewBox="0 0 24 24"
                fill="none"
                stroke="currentColor"
                strokeWidth="3"
              >
                <path d="M12 2v20M17 5H9.5a3.5 3.5 0 0 0 0 7h5a3.5 3.5 0 0 1 0 7H6" />
              </svg>
            </div>
            <div>
              <p className="text-[10px] font-black text-foreground/40 uppercase tracking-widest mb-0.5">
                [PENDING INFLOW]
              </p>
              <h3 className="text-sm font-black text-foreground uppercase tracking-tight">
                    {formatCurrency(stream.amount)} / {IncomeMap[stream.income_type] || stream.income_type} /
{stream.income_name}
              </h3>
            </div>
          </div>

          <div className="flex flex-col sm:flex-row items-stretch sm:items-center gap-3">
            <select
              disabled={loadingId === stream.id}
              value={selectedAccounts[stream.id] || ""}
              onChange={(e) =>
                setSelectedAccounts({
                  ...selectedAccounts,
                  [stream.id]: e.target.value,
                })
              }
                    className="border-2 border-foreground/10 bg-background rounded-xl px-4 py-2 text-[10px]
font-black    uppercase    tracking-widest    focus:outline-none    focus:border-foreground    transition-colors
appearance-none min-w-[180px]"
            >
              <option value="">Select Account</option>
              {accounts.map((acc) => (
```

```
                <option key={acc.id} value={acc.id}>
                  {acc.account_name}
                </option>
              ))}
            </select>

            <button
              disabled={loadingId === stream.id}
              onClick={() => handleVerify(stream)}
                className="bg-foreground text-background px-6 py-2 rounded-xl text-[10px] font-black uppercase
tracking-widest hover:bg-foreground/90 transition-all disabled:opacity-50 shrink-0"
            >
              {loadingId === stream.id ? "VERIFYING..." : "Verify & Clear"}
            </button>
          </div>
        </div>
      ))}
    </div>
  );
}
```

File: src/app/dashboard/RunwayCard.tsx

```
"use client";

import React from "react";

interface RunwayCardProps {
  runwayDays: number | null;
  netWorth?: number;
}

export function RunwayCard({ runwayDays, netWorth = 0 }: RunwayCardProps) {
  // 1. Logic for determining visual state
  const isInfiniteRunway = runwayDays === null;
  const isCritical = !isInfiniteRunway && runwayDays! < 30;
  const isInsolvent = netWorth < 0;

  // 2. Formatting
  const displayValue = isInfiniteRunway
    ? isInsolvent
      ? "INSOLVENT STABLE"
      : "SYSTEM STABLE"
    : `${(runwayDays! / 30).toFixed(1)} MONTHS`;

  // 3. Conditional Styles
   const containerClasses = `bg-foreground/5 border rounded-2xl p-6 md:p-8 flex flex-col justify-between
transition-all duration-500 ${
    isInfiniteRunway
      ? isInsolvent
        ? "border-rose-500/20"
        : "border-emerald-500/10"
      : isCritical
        ? "border-rose-500/30 bg-rose-500/5 shadow-inner"
        : "border-foreground/10"
  }`;

  const labelClasses = `text-[11px] font-black uppercase tracking-[0.2em] mb-4 ${
    isInfiniteRunway
      ? isInsolvent
        ? "text-rose-500/50"
        : "text-emerald-500/50"
      : isCritical
```

```
        ? "text-rose-500/60"
        : "text-foreground/40"
  }`;

  const valueClasses = `text-4xl font-black tabular-nums transition-colors duration-500 ${
    isInfiniteRunway
      ? isInsolvent
        ? "text-rose-500/80 font-mono tracking-tighter"
        : "text-emerald-500/80 font-mono tracking-tighter"
      : isCritical
        ? "text-rose-600"
        : "text-foreground"
  }`;

  const description = isInfiniteRunway
    ? isInsolvent
      ? "Burn is absorbed by inflows, but total commitments exceed assets."
      : "Net inbound capital completely absorbs current structural burn."
    : isCritical
      ? "A recalibration of The Foundation is recommended."
      : "Duration of sustained operation based on current structural burn and liquidity.";

  return (
    <div className={containerClasses}>
      <div className="flex items-center justify-between mb-4">
        <span className={labelClasses}>
          {isInfiniteRunway
            ? isInsolvent
              ? "Insolvent Stability"
              : "Infinite Stability"
            : "Operating Horizon"}
        </span>
        {isInfiniteRunway && !isInsolvent && (
          <div className="flex items-center gap-1.5">
            <span className="w-1.5 h-1.5 bg-emerald-500/40 rounded-full animate-pulse"></span>
            <span className="text-[8px] font-black text-emerald-500/40 uppercase tracking-widest">
              Active Protection
            </span>
          </div>
        )}
        {isInfiniteRunway && isInsolvent && (
          <div className="flex items-center gap-1.5">
            <span className="w-1.5 h-1.5 bg-rose-500/40 rounded-full animate-pulse"></span>
            <span className="text-[8px] font-black text-rose-500/40 uppercase tracking-widest">
              Solvency Risk
            </span>
          </div>
        )}
      </div>

      <div className="flex flex-col">
        <span className={valueClasses}>{displayValue}</span>
        <p
          className={`text-[10px] font-bold uppercase tracking-widest mt-4 ${
            isInfiniteRunway ? "text-emerald-600/40" : "text-foreground/40"
          }`}
        >
          {description}
        </p>
      </div>
    </div>
  );
}
```

File: src/app/dashboard/StrategicObjectiveSection.tsx

```
"use client";

import { useState } from "react";
import {
  OBJECTIVE_TYPES,
  OBJECTIVE_PRIORITIES,
  OBJECTIVE_STATUSES,
  calculateObjectiveProgressPercentage,
  calculateObjectiveFundingRatio,
  type StrategicObjective,
} from "@/lib/finance/objectives";
import {
  createStrategicObjectiveAction,
  updateStrategicObjectiveAction,
  deleteStrategicObjectiveAction,
  type DashboardSnapshot,
} from "./actions";
import { formatCurrency, formatDate } from "@/lib/utils/formatters";

interface StrategicObjectiveSectionProps {
  objectives: StrategicObjective[];
  onSnapshot: (snapshot: DashboardSnapshot) => void;
}

export function StrategicObjectiveSection({
  objectives,
  onSnapshot,
}: StrategicObjectiveSectionProps) {
  const [isAdding, setIsAdding] = useState(false);
  const [editingId, setEditingId] = useState<string | null>(null);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [form, setForm] = useState({
    objective_name: "",
    objective_type: "emergency_reserve",
    target_amount: "",
    current_amount: "",
    priority_level: "moderate",
    status: "active",
    target_date: "",
    notes: "",
  });

  function startEdit(obj: StrategicObjective) {
    setEditingId(obj.id);
    setForm({
      objective_name: obj.objective_name,
      objective_type: obj.objective_type,
      target_amount: obj.target_amount.toString(),
      current_amount: obj.current_amount.toString(),
      priority_level: obj.priority_level,
      status: obj.status,
      target_date: obj.target_date || "",
      notes: obj.notes || "",
    });
    setIsAdding(true);
  }

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
```

```
    setIsLoading(true);
    setError(null);
    try {
      let snapshot;
      if (editingId) {
        snapshot = await updateStrategicObjectiveAction(editingId, {
          objective_name: form.objective_name,
          objective_type: form.objective_type,
          target_amount: Number(form.target_amount),
          current_amount: form.current_amount ? Number(form.current_amount) : 0,
          priority_level: form.priority_level,
          status: form.status,
          target_date: form.target_date || null,
          notes: form.notes || undefined,
        });
      } else {
        snapshot = await createStrategicObjectiveAction({
          objective_name: form.objective_name,
          objective_type: form.objective_type,
          target_amount: Number(form.target_amount),
          current_amount: form.current_amount ? Number(form.current_amount) : 0,
          priority_level: form.priority_level,
          status: form.status,
          target_date: form.target_date || null,
          notes: form.notes || undefined,
        });
      }
      setForm({
        objective_name: "",
        objective_type: "emergency_reserve",
        target_amount: "",
        current_amount: "",
        priority_level: "moderate",
        status: "active",
        target_date: "",
        notes: "",
      });
      setIsAdding(false);
      setEditingId(null);
      onSnapshot(snapshot);
    } catch (err: unknown) {
      setError(
        err instanceof Error
          ? err.message
          : "Failed to establish strategic intent",
      );
    } finally {
      setIsLoading(false);
    }
  }

  async function handleDelete(id: string) {
    if (
      !confirm(
         "Archive this strategic objective? It will be removed from the active mission board but retained for
alignment history.",
      )
    )
      return;
    setIsLoading(true);
    try {
      const snapshot = await deleteStrategicObjectiveAction(id);
      onSnapshot(snapshot);
    } catch {
```

```
      alert("Failed to archive objective");
    } finally {
      setIsLoading(false);
    }
  }

  return (
    <div className="space-y-8">
      <div className="flex items-center justify-between px-1">
        <div className="space-y-1">
          <h2 className="text-xl font-black text-foreground tracking-tight uppercase">
            Strategic Objectives
          </h2>
          <p className="text-[10px] font-bold text-foreground/40 uppercase tracking-widest">
            Directional Financial Intentions
          </p>
        </div>
        <button
          onClick={() => {
            if (isAdding) {
              setEditingId(null);
              setForm({
                objective_name: "",
                objective_type: "emergency_reserve",
                target_amount: "",
                current_amount: "",
                priority_level: "moderate",
                status: "active",
                target_date: "",
                notes: "",
              });
            }
            setIsAdding(!isAdding);
          }}
             className="flex items-center gap-2 px-3 py-1.5 bg-foreground/5 hover:bg-foreground/10 rounded-xl
transition-all group"
        >
                      <span  className="text-[10px]  font-black  uppercase  tracking-widest  text-foreground/60
group-hover:text-foreground">
            {isAdding ? "Cancel" : "+ Set Objective"}
          </span>
        </button>
      </div>

      {isAdding && (
         <div className="bg-background border-2 border-foreground rounded-3xl p-6 shadow-2xl animate-in fade-in
slide-in-from-top-4 duration-300">
          <form onSubmit={handleSubmit} className="space-y-6">
            <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Objective Name
                </label>
                <input
                  type="text"
                  required
                  placeholder="e.g. Multi-Asset Liquidity Base"
                  value={form.objective_name}
                  onChange={(e) =>
                    setForm({ ...form, objective_name: e.target.value })
                  }
                                 className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
```

```
                />
              </div>
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Target Date (Optional)
                </label>
                <input
                  type="date"
                  value={form.target_date}
                  onChange={(e) =>
                    setForm({ ...form, target_date: e.target.value })
                  }
                                  className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
                />
              </div>
            </div>

            <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Target Amount (KSh)
                </label>
                <input
                  type="number"
                  required
                  placeholder="0.00"
                  value={form.target_amount}
                  onChange={(e) =>
                    setForm({ ...form, target_amount: e.target.value })
                  }
                                  className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
                />
              </div>
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Current Position (KSh)
                </label>
                <input
                  type="number"
                  placeholder="0.00"
                  value={form.current_amount}
                  onChange={(e) =>
                    setForm({ ...form, current_amount: e.target.value })
                  }
                                  className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold"
                />
              </div>
            </div>

            <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
              <div>
                  <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                  Objective Type
                </label>
                <select
                  value={form.objective_type}
                  onChange={(e) =>
```

```
                  setForm({ ...form, objective_type: e.target.value })
                }
                            className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold appearance-none"
              >
                {OBJECTIVE_TYPES.map(
                  (t: { value: string; label: string }) => (
                    <option key={t.value} value={t.value}>
                      {t.label}
                    </option>
                  ),
                )}
              </select>
            </div>
            <div>
                <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                Priority Level
              </label>
              <select
                value={form.priority_level}
                onChange={(e) =>
                  setForm({ ...form, priority_level: e.target.value })
                }
                            className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold appearance-none"
              >
                {OBJECTIVE_PRIORITIES.map(
                  (p: { value: string; label: string }) => (
                    <option key={p.value} value={p.value}>
                      {p.label}
                    </option>
                  ),
                )}
              </select>
            </div>
            <div>
                <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                Operational Status
              </label>
              <select
                value={form.status}
                onChange={(e) => setForm({ ...form, status: e.target.value })}
                            className="w-full  border-2  border-foreground/10  bg-background  rounded-xl  p-3
focus:outline-none focus:border-foreground transition-colors text-foreground text-sm font-bold appearance-none"
              >
                {OBJECTIVE_STATUSES.map(
                  (s: { value: string; label: string }) => (
                    <option key={s.value} value={s.value}>
                      {s.label}
                    </option>
                  ),
                )}
              </select>
            </div>
          </div>

          <div>
               <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest mb-1.5
block ml-1">
                Strategic Notes (Optional)
              </label>
              <textarea
```

```
                placeholder="Details of this strategic intention..."
                value={form.notes}
                onChange={(e) => setForm({ ...form, notes: e.target.value })}
                className="w-full border-2 border-foreground/10 bg-background rounded-xl p-3 focus:outline-none
focus:border-foreground transition-colors text-foreground text-sm font-bold min-h-[80px]"
              />
            </div>

            {error && (
              <p className="text-red-500 text-[10px] font-black uppercase tracking-tight ml-1">
                {error}
              </p>
            )}

            <button
              type="submit"
              disabled={isLoading}
                       className="w-full bg-foreground text-background py-4 rounded-xl font-black uppercase
tracking-widest hover:bg-foreground/90 transition-colors disabled:opacity-50 shadow-xl"
            >
              {isLoading
                ? "ESTABLISHING..."
                : editingId
                  ? "Update Strategic Intent"
                  : "Establish Strategic Intent"}
            </button>
          </form>
        </div>
      )}

      <div className="grid grid-cols-1 gap-6">
        {objectives.length === 0 ? (
              <div className="border-2 border-dashed border-foreground/10 rounded-3xl p-16 text-center group
hover:border-foreground/20 transition-colors">
            <p className="text-foreground/60 text-xs font-bold uppercase tracking-widest">
              No strategic objectives defined.
            </p>
                    <p  className="text-foreground/40  text-[10px]  mt-3  uppercase  tracking-tight  opacity-60
leading-relaxed">
              Axiom currently observes financial behavior <br /> without
              directional context.
            </p>
          </div>
        ) : (
          objectives.map((obj) => {
            const ratio = calculateObjectiveFundingRatio(obj);
            return (
              <div
                key={obj.id}
                            className="bg-foreground/5  border  border-foreground/10  rounded-2xl  p-6  group
hover:bg-foreground/10 transition-all relative overflow-hidden"
              >
                <div className="flex items-center justify-between relative z-10 mb-6">
                  <div className="space-y-1.5">
                    <div className="flex items-center gap-3">
                      <span
                               className={`text-[9px] font-black text-background px-2 py-0.5 rounded uppercase
tracking-[0.1em] ${
                          obj.priority_level === "critical"
                            ? "bg-orange-500"
                            : obj.priority_level === "high"
                              ? "bg-foreground/70"
                              : "bg-foreground/40"
                        }`}
```

```
                    >
                      {obj.priority_level}
                    </span>
                    <h3 className="text-base font-black text-foreground uppercase tracking-tight">
                      {obj.objective_name}
                    </h3>
                  </div>
                  <p className="text-[10px] font-bold text-foreground/40 uppercase tracking-[0.2em]">
                    {obj.objective_type.replace("_", " ").toUpperCase()} ::{" "}
                    {obj.status}
                    {obj.target_date &&
                      ` ? Target: ${formatDate(obj.target_date)}`}
                  </p>
                </div>
                <div className="flex items-center gap-6">
                  <div className="text-right">
                    <p className="text-2xl font-black tabular-nums text-foreground">
                      {Math.round(ratio)}%
                    </p>
                    <p className="text-[9px] font-black text-foreground/40 uppercase tracking-widest">
                      Funding Ratio
                    </p>
                  </div>
                  <div className="flex items-center gap-2">
                    <button
                      onClick={() => startEdit(obj)}
                         className="p-2 text-foreground/20 hover:text-foreground transition-colors opacity-0
group-hover:opacity-100"
                    >
                      <svg
                        width="16"
                        height="16"
                        viewBox="0 0 24 24"
                        fill="none"
                        stroke="currentColor"
                        strokeWidth="3"
                      >
                        <path d="M12 20h9M16.5 3.5a2.121 2.121 0 0 1 3 3L7 19l-4 1 1-4L16.5 3.5z" />
                      </svg>
                    </button>
                    <button
                      onClick={() => handleDelete(obj.id)}
                            className="p-2 text-foreground/20 hover:text-red-500 transition-colors opacity-0
group-hover:opacity-100"
                    >
                      <svg
                        width="16"
                        height="16"
                        viewBox="0 0 24 24"
                        fill="none"
                        stroke="currentColor"
                        strokeWidth="3"
                      >
                         <path d="M3 6h18M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0 1 2-2h4a2 2 0
0 1 2 2v2" />
                      </svg>
                    </button>
                  </div>
                </div>
              </div>

              <div className="space-y-3">
                <div className="h-2 w-full bg-foreground/5 rounded-full overflow-hidden">
                  <div
```

```
                    className={`h-full transition-all duration-1000 ease-out ${
                      ratio >= 100 ? "bg-green-500" : "bg-foreground/60"
                    }`}
                    style={{ width: `${ratio}%` }}
                  />
                </div>
                <div className="flex justify-between items-end">
                  <div>
                          <p className="text-[10px] font-black text-foreground/30 uppercase tracking-widest
mb-0.5">
                      Current Position
                    </p>
                    <p className="text-xs font-black text-foreground tabular-nums">
                      {formatCurrency(obj.current_amount)}
                    </p>
                  </div>
                  <div className="text-right">
                          <p className="text-[10px] font-black text-foreground/30 uppercase tracking-widest
mb-0.5">
                      Strategic Target
                    </p>
                    <p className="text-xs font-black text-foreground/60 tabular-nums">
                      {formatCurrency(obj.target_amount)}
                    </p>
                  </div>
                </div>
              </div>

              {obj.notes && (
                <div className="mt-5 pt-4 border-t border-foreground/5">
                  <p className="text-[10px] font-bold text-foreground/40 italic leading-relaxed">
                    &amp;quot;{obj.notes}&amp;quot;
                  </p>
                </div>
              )}
            </div>
          );
        })
      )}
    </div>
  </div>
);
}
```

File: src/app/dashboard/actions.ts

```
"use server";

import { revalidatePath } from "next/cache";
import { createClient } from "@/lib/supabase/server";
import {
  getDeployments,
  createDeployment,
  updateDeployment,
  deleteDeployment,
} from "@/lib/db/deployments";
import {
  getAccounts,
  createAccount,
  updateAccount,
  deleteAccount,
} from "@/lib/db/accounts";
import {
```

```
  getLiabilities,
  createLiability,
  updateLiability,
  deleteLiability,
} from "@/lib/db/liabilities";
import {
  getIncomeStreams,
  createIncomeStream,
  createIncomeStreams,
  updateIncomeStream,
  deleteIncomeStream,
} from "@/lib/db/income";
import { getGoals, createGoal, updateGoal, deleteGoal } from "@/lib/db/goals";
import {
  getStrategicObjectives,
  createStrategicObjective,
  updateStrategicObjective,
  deleteStrategicObjective,
} from "@/lib/db/objectives";
import {
  getOperationalBaseline,
  createOperationalBaseline,
  updateOperationalBaseline,
  deleteOperationalBaseline,
} from "@/lib/db/baseline";
import { getUserSettings, updateLiquidity } from "@/lib/db/settings";
import { getInsights, saveInsight } from "@/lib/db/insights";
import { getDeletedLedgerRecords } from "@/lib/db/audit";
import { generateKairosAIInsight, type KairosInsight } from "@/lib/ai/kairos";
import { generateSummary, type AnalyticsSummary } from "@/lib/analytics";
import type {
  Deployment,
  Account,
  Liability,
  IncomeStream,
  FinancialGoal,
  StrategicObjective,
  OperationalBaseline,
  BaselineCadence,
} from "@/lib/analytics/types";
import type { DeploymentAdvancedContextInput } from "@/lib/finance/deploymentContext";

export interface DashboardSnapshot {
  authenticated: boolean;
  deployments: Deployment[];
  accounts: Account[];
  liabilities: Liability[];
  incomeStreams: IncomeStream[];
  goals: FinancialGoal[];
  objectives: StrategicObjective[];
  baseline: OperationalBaseline[];
  analytics: AnalyticsSummary | null;
  liquidity: number;
  kairosInsight: KairosInsight | null;
}

export interface CreateDeploymentActionInput {
  title: string;
  amount: number;
  category: string;
  advancedContext?: DeploymentAdvancedContextInput;
  accountId?: string;
}
```

```
export interface UpdateDeploymentActionInput {
  title: string;
  amount: number;
  category: string;
  accountId?: string;
}

export interface CreateAccountActionInput {
  account_name: string;
  account_type: string;
  current_balance: number;
  institution?: string;
}

export interface CreateLiabilityActionInput {
  liability_name: string;
  liability_type: string;
  outstanding_balance: number;
  interest_rate?: number;
  minimum_payment?: number;
  due_date?: string | null;
  institution?: string;
}

export interface CreateIncomeActionInput {
  income_name: string;
  income_type: string;
  amount: number;
  cadence: string;
  execution_day?: number | null;
  is_recurring?: boolean;
  source?: string;
  start_date?: string;
  end_date?: string | null;
}

export interface CreateGoalActionInput {
  goal_name: string;
  goal_type: string;
  target_amount: number;
  current_progress?: number;
  target_date?: string | null;
  priority: string;
  status: string;
  notes?: string;
}

export interface CreateObjectiveActionInput {
  objective_name: string;
  objective_type: string;
  target_amount: number;
  current_amount?: number;
  target_date?: string | null;
  priority_level: string;
  status: string;
  notes?: string;
}

function unauthenticatedSnapshot(): DashboardSnapshot {
  return {
    authenticated: false,
    deployments: [],
    accounts: [],
    liabilities: [],
```

```
    incomeStreams: [],
    goals: [],
    objectives: [],
    baseline: [],
    analytics: null,
    liquidity: 0,
    kairosInsight: null,
  };
}

const requireAuth = async () => {
  const supabase = await createClient();
  const {
    data: { user },
    error,
  } = await supabase.auth.getUser();
  if (error || !user) throw new Error("Unauthorized mutation attempt.");
  return { userId: user.id, supabase };
};

function normalizeSavedInsight(row: Record<string, unknown>): KairosInsight {
  const metadata =
    row.metadata && typeof row.metadata === "object"
      ? (row.metadata as Record<string, unknown>)
      : {};

  return {
    type: row.type as KairosInsight["type"],
    severity: (metadata.severity as KairosInsight["severity"]) || "observation",
    category: row.category as KairosInsight["category"],
    message: String(row.message || ""),
    supportingSignals: Array.isArray(metadata.supporting_signals)
      ? (metadata.supporting_signals as string[])
      : metadata.supporting_signal
        ? [String(metadata.supporting_signal)]
        : [],
    timestamp: String(
      metadata.timestamp || row.created_at || new Date().toISOString(),
    ),
    runway: metadata.runway !== undefined ? Number(metadata.runway) : null,
    capitalEfficiency:
      metadata.capital_efficiency !== undefined
        ? Number(metadata.capital_efficiency)
        : 100,
    isSilent: Boolean(metadata.is_silent),
    metadataQuality:
      metadata.metadata_quality && typeof metadata.metadata_quality === "object"
        ? (metadata.metadata_quality as KairosInsight["metadataQuality"])
        : undefined,
    is_new_signal: false,
  };
}

async function buildDashboardSnapshot(
  options: { forceInsightEvaluation?: boolean } = {},
): Promise<DashboardSnapshot> {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();

  if (!user) return unauthenticatedSnapshot();

  console.log("ACTIVE SESSION USER ID:", user.id);
```

```
const [
  deploymentsData,
  accountsData,
  liabilitiesData,
  incomeStreamsData,
  goalsData,
  objectivesData,
  baselineData,
  settings,
] = await Promise.all([
  getDeployments(supabase),
  getAccounts(supabase),
  getLiabilities(supabase),
  getIncomeStreams(supabase),
  getGoals(supabase),
  getStrategicObjectives(supabase),
  getOperationalBaseline(supabase),
  getUserSettings(supabase, user.id),
]);

const deployments = (deploymentsData || []) as Deployment[];
const accounts = (accountsData || []) as Account[];
const liabilities = (liabilitiesData || []) as Liability[];
const incomeStreams = (incomeStreamsData || []) as IncomeStream[];
const goals = (goalsData || []) as FinancialGoal[];
const objectives = (objectivesData || []) as StrategicObjective[];
const baseline = (baselineData || []) as OperationalBaseline[];
const liquidity = Number(settings.total_liquidity);
const analytics = generateSummary(
  deployments,
  liquidity,
  accounts,
  liabilities,
  incomeStreams,
  goals,
  objectives,
  baseline,
);
const savedInsights = await getInsights(supabase, 1);
const previousInsight =
  savedInsights && savedInsights.length > 0
    ? normalizeSavedInsight(savedInsights[0] as Record<string, unknown>)
    : null;

let kairosInsight: KairosInsight | null = null;

if (!options.forceInsightEvaluation && previousInsight) {
  // OPTIMIZATION SHORTCUT: Reuse cached strategic signal
  kairosInsight = previousInsight;
} else {
  // FRESH EVALUATION: Process authoritative telemetry
  kairosInsight = await generateKairosAIInsight({
    deployments,
    liquidity,
    previousInsight,
    objectives,
    accounts,
    liabilities,
    incomeStreams,
    goals,
    baseline,
  });
```

```
    // Only save if it's actually a material change
    if (kairosInsight.is_new_signal) {
      await saveInsight(supabase, kairosInsight, user.id);
    }
  }

  return {
    authenticated: true,
    deployments,
    accounts,
    liabilities,
    incomeStreams,
    goals,
    objectives,
    baseline,
    analytics,
    liquidity,
    kairosInsight,
  };
}

export async function getDashboardSnapshotAction() {
  return buildDashboardSnapshot();
}

export async function createDeploymentAction(
  input: CreateDeploymentActionInput,
) {
  const { userId, supabase } = await requireAuth();

  try {
    // 1. Create the deployment record (The database trigger handles account balance deduction)
    await createDeployment(
      supabase,
      input.title,
      input.amount,
      userId,
      input.category,
      0,
      input.advancedContext,
      input.accountId,
    );

    // 2. Update global liquidity setting if this was a liquidity deployment
    if (input.accountId) {
      const settings = await getUserSettings(supabase, userId);
      const newLiquidity = Number(settings.total_liquidity) - input.amount;
      await updateLiquidity(supabase, userId, newLiquidity);
    }

    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to create deployment:", error);
    throw error;
  }
}

export async function updateDeploymentAction(
  id: string,
  input: UpdateDeploymentActionInput,
) {
  const { userId, supabase } = await requireAuth();
```

```
  try {
    await updateDeployment(supabase, id, userId, {
      title: input.title,
      amount: input.amount,
      category: input.category,
    });
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to update deployment:", error);
    throw error;
  }
}

export async function deleteDeploymentAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteDeployment(supabase, id, userId);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to delete deployment:", error);
    throw error;
  }
}

export async function updateLiquidityAction(amount: number) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateLiquidity(supabase, userId, amount);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to update liquidity:", error);
    throw error;
  }
}

export async function createAccountAction(input: CreateAccountActionInput) {
  const { userId, supabase } = await requireAuth();

  try {
    await createAccount(supabase, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to create account:", error);
    throw error;
  }
}

export async function updateAccountAction(
  id: string,
  input: Partial<CreateAccountActionInput>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateAccount(supabase, id, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
```

```
    console.error("Failed to update account:", error);
    throw error;
  }
}

export async function deleteAccountAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteAccount(supabase, id, userId);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to delete account:", error);
    throw error;
  }
}

export async function createLiabilityAction(input: CreateLiabilityActionInput) {
  const { userId, supabase } = await requireAuth();

  try {
    await createLiability(supabase, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to create liability:", error);
    throw error;
  }
}

export async function updateLiabilityAction(
  id: string,
  input: Partial<CreateLiabilityActionInput>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateLiability(supabase, id, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to update liability:", error);
    throw error;
  }
}

export async function deleteLiabilityAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteLiability(supabase, id, userId);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to delete liability:", error);
    throw error;
  }
}

export async function createIncomeAction(inputs: CreateIncomeActionInput[]) {
  const { userId, supabase } = await requireAuth();

  try {
```

```
    await createIncomeStreams(supabase, userId, inputs);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to create income streams:", error);
    throw error;
  }
}

export async function updateIncomeAction(
  id: string,
  input: Partial<CreateIncomeActionInput>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateIncomeStream(supabase, id, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to update income stream:", error);
    throw error;
  }
}

export async function deleteIncomeAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteIncomeStream(supabase, id, userId);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to delete income stream:", error);
    throw error;
  }
}

export async function resolvePendingInflowAction(payload: {
  incomeId: string;
  accountId: string;
  amount: number;
}) {
  const { userId, supabase } = await requireAuth();

  try {
    // 1. Get current account balance
    const { data: account, error: accError } = await supabase
      .from("accounts")
      .select("current_balance")
      .eq("id", payload.accountId)
      .eq("user_id", userId)
      .single();

    if (accError || !account) throw new Error("Target account not found.");

    // 2. Update the target account balance
    const newBalance = Number(account.current_balance) + payload.amount;
    await updateAccount(supabase, payload.accountId, userId, {
      current_balance: newBalance,
    });

    // 3. Update the income stream last_executed_at
    await updateIncomeStream(supabase, payload.incomeId, userId, {
```

```
      last_executed_at: new Date().toISOString(),
    });

    // 4. Update global liquidity if this account is part of it
    const settings = await getUserSettings(supabase, userId);
    const newLiquidity = Number(settings.total_liquidity) + payload.amount;
    await updateLiquidity(supabase, userId, newLiquidity);

    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to resolve pending inflow:", error);
    throw error;
  }
}

export async function createGoalAction(input: CreateGoalActionInput) {
  const { userId, supabase } = await requireAuth();

  try {
    await createGoal(supabase, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to create goal:", error);
    throw error;
  }
}

export async function updateGoalAction(
  id: string,
  input: Partial<CreateGoalActionInput>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateGoal(supabase, id, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to update goal:", error);
    throw error;
  }
}

export async function deleteGoalAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteGoal(supabase, id, userId);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to delete goal:", error);
    throw error;
  }
}

export async function createStrategicObjectiveAction(
  input: CreateObjectiveActionInput,
) {
  const { userId, supabase } = await requireAuth();

  try {
```

```
    await createStrategicObjective(supabase, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to create strategic objective:", error);
    throw error;
  }
}

export const createObjectiveAction = createStrategicObjectiveAction;

export async function updateStrategicObjectiveAction(
  id: string,
  input: Partial<CreateObjectiveActionInput>,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateStrategicObjective(supabase, id, userId, input);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to update strategic objective:", error);
    throw error;
  }
}

export const updateObjectiveAction = updateStrategicObjectiveAction;

export async function deleteStrategicObjectiveAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteStrategicObjective(supabase, id, userId);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to delete strategic objective:", error);
    throw error;
  }
}

export const deleteObjectiveAction = deleteStrategicObjectiveAction;

export async function createOperationalBaselineAction(
  data: Omit<
    OperationalBaseline,
    "id" | "user_id" | "created_at" | "updated_at"
  >,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await createOperationalBaseline(supabase, userId, data);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to create operational baseline:", error);
    throw error;
  }
}

export async function updateOperationalBaselineAction(
  id: string,
```

```
  updates: Partial<
    Omit<OperationalBaseline, "id" | "user_id" | "created_at" | "updated_at">
  >,
) {
  const { userId, supabase } = await requireAuth();

  try {
    await updateOperationalBaseline(supabase, id, userId, updates);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to update operational baseline:", error);
    throw error;
  }
}

export async function deleteOperationalBaselineAction(id: string) {
  const { userId, supabase } = await requireAuth();

  try {
    await deleteOperationalBaseline(supabase, id, userId);
    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Failed to delete operational baseline:", error);
    throw error;
  }
}

export async function submitDayZeroBaselineAction(payload: {
  accounts: {
    account_name: string;
    account_type: string;
    current_balance: number;
  }[];
  incomes: {
    income_name: string;
    income_type: string;
    amount: number;
    cadence: string;
  }[];
  liabilities: {
    liability_name: string;
    liability_type: string;
    outstanding_balance: number;
  }[];
  baseline: { amount: number; cadence: string };
}) {
  const { userId, supabase } = await requireAuth();

  try {
    // 1. Insert Accounts
    for (const acc of payload.accounts) {
      await createAccount(supabase, userId, acc);
    }

    // 2. Insert Incomes (if any)
    if (payload.incomes.length > 0) {
      await createIncomeStreams(
        supabase,
        userId,
        payload.incomes.map((inc) => ({
          ...inc,
          is_recurring: true,
```

```
        })),
      );
    }

    // 3. Insert Liabilities (if any)
    for (const liab of payload.liabilities) {
      if (liab.outstanding_balance > 0) {
        await createLiability(supabase, userId, liab);
      }
    }

    // 4. Insert Operational Baseline
    await createOperationalBaseline(supabase, userId, {
      title: "Core Survival Baseline",
      amount: payload.baseline.amount,
      category: "Maintenance",
      cadence: payload.baseline.cadence as BaselineCadence,
      baseline_type: "expense",
      is_active: true,
    });

    // 5. Update initial liquidity pool to match sum of all accounts
    const totalLiquidity = payload.accounts.reduce(
      (sum, acc) => sum + acc.current_balance,
      0,
    );
    await updateLiquidity(supabase, userId, totalLiquidity);

    revalidatePath("/dashboard");
    return buildDashboardSnapshot({ forceInsightEvaluation: true });
  } catch (error) {
    console.error("Day Zero Onboarding failed:", error);
    throw error;
  }
}

export async function fetchHistoricalAuditAction() {
  const { userId, supabase } = await requireAuth();
  return getDeletedLedgerRecords(supabase, userId);
}

export interface AuditLog {
  id: string;
  rule_id: string;
  severity: string;
  evaluation_time_ms: number;
  created_at: string;
}

export interface TelemetrySummary {
  logs: AuditLog[];
  averageLatency: number;
  matchRate: number;
  totalCycles: number;
  ruleHits: Record<string, number>;
}

export async function fetchTelemetryLogsAction(): Promise<TelemetrySummary> {
  const { userId, supabase } = await requireAuth();

  const { data, error } = await supabase
    .from("kairos_insights")
    .select("*")
    .eq("user_id", userId)
```

```
    .order("created_at", { ascending: false })
    .limit(50);

  if (error) {
    console.error("[Telemetry] Fetch failed:", error.message);
    return {
      logs: [],
      averageLatency: 0,
      matchRate: 0,
      totalCycles: 0,
      ruleHits: {},
    };
  }

  const logs = (data || []) as AuditLog[];
  const totalLogs = logs.length;

  if (totalLogs === 0) {
    return {
      logs: [],
      averageLatency: 0,
      matchRate: 0,
      totalCycles: 0,
      ruleHits: {},
    };
  }

  const totalLatency = logs.reduce(
    (sum, log) => sum + (log.evaluation_time_ms || 0),
    0,
  );
  const averageLatency = totalLatency / totalLogs;

  const matches = logs.filter((log) => log.severity !== "none").length;
  const matchRate = (matches / totalLogs) * 100;

  const ruleHits: Record<string, number> = {};
  logs.forEach((log) => {
    if (log.severity && log.severity !== "none") {
      ruleHits[log.rule_id] = (ruleHits[log.rule_id] || 0) + 1;
    }
  });

  return {
    logs,
    averageLatency: Math.round(averageLatency * 100) / 100,
    matchRate: Math.round(matchRate * 10) / 10,
    totalCycles: totalLogs,
    ruleHits,
  };
}
```

File: src/app/dashboard/error.tsx

```
"use client";

import { useEffect } from "react";

export default function DashboardError({
  error,
  reset,
}: {
  error: Error & { digest?: string };
```

```
  reset: () => void;
}) {
  useEffect(() => {
    // Log the error to an error reporting service
    console.error("Dashboard Route Error:", error);
  }, [error]);

  return (
    <main className="min-h-[80vh] flex items-center justify-center p-6">
      <div className="w-full max-w-md bg-background border-2 border-red-500/20 rounded-[2.5rem] p-10 shadow-2xl
text-center">
        <div className="w-20 h-20 bg-red-500/10 rounded-3xl flex items-center justify-center mx-auto mb-6">
          <svg
            width="40"
            height="40"
            viewBox="0 0 24 24"
            fill="none"
            stroke="currentColor"
            strokeWidth="3"
            className="text-red-500"
          >
             <path d="M10.29 3.86L1.82 18a2 2 0 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0 0 0-3.42 0z"
/>
            <line x1="12" y1="9" x2="12" y2="13" />
            <line x1="12" y1="17" x2="12.01" y2="17" />
          </svg>
        </div>

        <h1 className="text-3xl font-black tracking-tighter uppercase mb-4 text-foreground">
          System Interrupted
        </h1>

        <p className="text-foreground/60 text-sm font-bold uppercase tracking-widest mb-8 leading-relaxed">
          The intelligence engine encountered an unhandled exception.
        </p>

        <div className="bg-foreground/5 rounded-2xl p-4 mb-8 text-left">
          <p className="text-[10px] font-black text-foreground/40 uppercase tracking-widest mb-2">
            Error Diagnostic
          </p>
          <p className="text-xs font-mono text-red-400 break-all leading-tight">
            {error.message || "Unknown cryptographic failure"}
          </p>
        </div>

        <button
          onClick={() => reset()}
                 className="w-full  bg-foreground  text-background  px-4  py-5  rounded-2xl  font-black  text-lg
hover:scale-[1.02] active:scale-[0.98] transition-all shadow-xl shadow-foreground/10 uppercase"
        >
          Attempt Recovery
        </button>
      </div>
    </main>
  );
}
```

File: src/app/dashboard/layout.tsx

```
"use client";

import { supabase } from "@/lib/supabase";
import { useEffect, useState } from "react";
```

```
import { User } from "@supabase/supabase-js";

export const dynamic = "force-dynamic";

export default function DashboardLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  const [user, setUser] = useState<User | null>(null);
  const [isProfileOpen, setIsProfileOpen] = useState(false);
  const [activeSection, setActiveSection] = useState("overview");
  const [isSidebarCollapsed, setIsSidebarCollapsed] = useState(false);
  const [isMobileMenuOpen, setIsMobileMenuOpen] = useState(false);

  useEffect(() => {
    async function getUser() {
      const {
        data: { user },
      } = await supabase.auth.getUser();
      setUser(user);
    }
    getUser();

    const handleScroll = () => {
      const sections = ["overview", "deploy", "intelligence", "ledger"];
      const scrollPosition = window.scrollY + 100;

      for (const section of sections) {
        const element = document.getElementById(section);
        if (element) {
          const { offsetTop, offsetHeight } = element;
          if (
            scrollPosition >= offsetTop &&
            scrollPosition < offsetTop + offsetHeight
          ) {
            setActiveSection(section);
            break;
          }
        }
      }
    };

    window.addEventListener("scroll", handleScroll);
    return () => window.removeEventListener("scroll", handleScroll);
  }, []);

  async function handleLogout() {
    await supabase.auth.signOut();
    window.location.href = "/";
  }

  const userInitial = user?.email?.[0].toUpperCase() || "U";

  const navItems = [
    {
      id: "overview",
      label: "Overview",
      icon: (
        <svg
          width="18"
          height="18"
          viewBox="0 0 24 24"
          fill="none"
```

```
          stroke="currentColor"
          strokeWidth="2.5"
        >
          <rect x="3" y="3" width="7" height="7" />
          <rect x="14" y="3" width="7" height="7" />
          <rect x="14" y="14" width="7" height="7" />
          <rect x="3" y="14" width="7" height="7" />
        </svg>
      ),
    },
    {
      id: "deploy",
      label: "Deploy",
      icon: (
        <svg
          width="18"
          height="18"
          viewBox="0 0 24 24"
          fill="none"
          stroke="currentColor"
          strokeWidth="2.5"
        >
          <path d="M12 2v20M17 5H9.5a3.5 3.5 0 0 0 0 7h5a3.5 3.5 0 0 1 0 7H6" />
        </svg>
      ),
    },
    {
      id: "intelligence",
      label: "Intelligence",
      icon: (
        <svg
          width="18"
          height="18"
          viewBox="0 0 24 24"
          fill="none"
          stroke="currentColor"
          strokeWidth="2.5"
        >
          <path d="M21 16V8a2 2 0 0 0-1-1.73l-7-4a2 2 0 0 0-2 0l-7 4A2 2 0 0 0 3 8v8a2 2 0 0 0 1 1.73l7 4a2 2 0
0 0 2 0l7-4A2 2 0 0 0 21 16z" />
          <polyline points="3.27 6.96 12 12.01 20.73 6.96" />
          <line x1="12" y1="22.08" x2="12" y2="12" />
        </svg>
      ),
    },
    {
      id: "ledger",
      label: "Expense Log",
      icon: (
        <svg
          width="18"
          height="18"
          viewBox="0 0 24 24"
          fill="none"
          stroke="currentColor"
          strokeWidth="2.5"
        >
          <path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z" />
          <polyline points="14 2 14 8 20 8" />
          <line x1="16" y1="13" x2="8" y2="13" />
          <line x1="16" y1="17" x2="8" y2="17" />
          <polyline points="10 9 9 9 8 9" />
        </svg>
      ),
```

```
    },
  ];

  const scrollToSection = (id: string) => {
    const element = document.getElementById(id);
    if (element) {
      const offset = 80;
      const bodyRect = document.body.getBoundingClientRect().top;
      const elementRect = element.getBoundingClientRect().top;
      const elementPosition = elementRect - bodyRect;
      const offsetPosition = elementPosition - offset;

      window.scrollTo({
        top: offsetPosition,
        behavior: "smooth",
      });
      setIsMobileMenuOpen(false);
    }
  };

  const activeLabel = navItems.find((item) => item.id === activeSection)?.label;

  return (
    <div className="min-h-screen bg-background text-foreground flex flex-col md:flex-row">
      {/* MOBILE HEADER */}
        <header className="md:hidden h-16 border-b border-foreground/5 bg-background/80 backdrop-blur-xl flex
items-center justify-between px-6 sticky top-0 z-[60]">
        <div className="flex items-center gap-2">
          <div className="w-6 h-6 bg-foreground rounded-lg flex items-center justify-center">
            <span className="font-black text-background text-[8px]">A</span>
          </div>
          <span className="text-xs font-black uppercase tracking-widest">
            {activeLabel}
          </span>
        </div>

        {/* Profile trigger moved to top header for mobile */}
        <button
          onClick={() => setIsProfileOpen(!isProfileOpen)}
                className="w-8 h-8 rounded-lg bg-foreground/5 border border-foreground/10 flex items-center
justify-center overflow-hidden shrink-0"
        >
          {user?.user_metadata?.avatar_url ? (
            <img
              src={user.user_metadata.avatar_url}
              alt="Avatar"
              className="w-full h-full object-cover"
            />
          ) : (
            <span className="font-black text-foreground/40 text-[10px]">
              {userInitial}
            </span>
          )}
        </button>

        {isProfileOpen && (
          <>
            <div
              className="fixed inset-0 z-10"
              onClick={() => setIsProfileOpen(false)}
            ></div>
                <div className="absolute top-full right-6 mt-2 w-48 bg-background border border-foreground/10
rounded-2xl shadow-2xl z-20 overflow-hidden animate-in fade-in slide-in-from-top-2 duration-200">
              <div className="p-4 border-b border-foreground/5">
```

```
              <p className="text-[9px] font-black text-foreground/40 uppercase tracking-widest mb-1">
                Session Details
              </p>
              <p className="text-[10px] font-bold text-foreground truncate">
                {user?.email}
              </p>
            </div>
            <div className="p-2">
              <button
                disabled
                        className="w-full text-left px-3 py-2 rounded-lg text-[10px] font-black uppercase
tracking-widest text-foreground/40 cursor-not-allowed flex items-center justify-between"
              >
                Settings
                <span className="text-[7px] bg-foreground/5 px-1.5 py-0.5 rounded text-foreground/30">
                  Soon
                </span>
              </button>
              <button
                onClick={handleLogout}
                        className="w-full text-left px-3 py-2 rounded-lg text-[10px] font-black uppercase
tracking-widest text-red-500 hover:bg-red-500/5 transition-colors"
              >
                Log Out
              </button>
            </div>
          </div>
        </>
      )}
    </header>

    {/* MOBILE BOTTOM NAV */}
     <nav className="md:hidden fixed bottom-0 left-0 right-0 h-16 bg-background/80 backdrop-blur-xl border-t
border-foreground/5 flex items-center justify-around px-4 z-[60] pb-safe">
      {navItems.map((item) => (
        <button
          key={item.id}
          onClick={() => scrollToSection(item.id)}
          className={`flex flex-col items-center gap-1 transition-all duration-300 ${
            activeSection === item.id
              ? "text-foreground"
              : "text-foreground/30"
          }`}
        >
          <span
            className={`transition-transform duration-300 ${activeSection === item.id ? "scale-110" : ""}`}
          >
            {item.icon}
          </span>
          <span className="text-[8px] font-black uppercase tracking-[0.2em]">
            {item.label}
          </span>
          {activeSection === item.id && (
            <div className="w-1 h-1 bg-foreground rounded-full animate-pulse mt-0.5"></div>
          )}
        </button>
      ))}
    </nav>

    {/* SIDEBAR (DESKTOP ONLY) */}
    <aside
      className={`${
        isSidebarCollapsed ? "md:w-20" : "md:w-64"
       } hidden md:flex border-r border-foreground/5 bg-background flex-col fixed inset-y-0 left-0 z-50
```

```
transition-all duration-300 ease-in-out`}
      >
        {/* BRANDING */}
        <div className={`p-8 ${isSidebarCollapsed ? "px-4" : "pb-12"}`}>
          <div className="flex items-center gap-3 group cursor-pointer">
            <div className="relative shrink-0">
                          <div  className="w-8  h-8  bg-foreground  rounded-xl  rotate-3  transition-transform
group-hover:rotate-12 group-hover:scale-110"></div>
                    <div  className="absolute  inset-0  w-8  h-8  bg-foreground/20  rounded-xl  -rotate-6  scale-95
transition-transform group-hover:-rotate-12"></div>
                 <div className="absolute inset-0 flex items-center justify-center font-black text-background
text-[10px]">
                  A
                </div>
              </div>
              {!isSidebarCollapsed && (
                <div className="flex flex-col animate-in fade-in slide-in-from-left-2 duration-300">
                  <span className="text-lg font-black tracking-tighter uppercase leading-none">
                    AXIOM
                  </span>
                  <span className="text-[7px] font-black text-gray-500 uppercase tracking-widest mt-1">
                    Intelligence Layer
                  </span>
                </div>
              )}
            </div>
          </div>

          {/* NAVIGATION */}
          <nav className="flex-1 px-4 space-y-1">
            {!isSidebarCollapsed && (
                   <p  className="px-4  text-[9px]  font-black  text-foreground/30  uppercase  tracking-[0.3em]  mb-4
animate-in fade-in duration-300">
                Financial Terminal
              </p>
            )}
            {navItems.map((item) => (
              <button
                key={item.id}
                onClick={() => scrollToSection(item.id)}
                className={`w-full flex items-center ${isSidebarCollapsed ? "justify-center" : "gap-4 px-4"} py-3
rounded-2xl transition-all duration-300 group ${
                  activeSection === item.id
                    ? "bg-foreground text-background shadow-lg shadow-foreground/10"
                    : "text-foreground/50 hover:bg-foreground/5 hover:text-foreground"
                }`}
              >
                <span
                      className={`transition-transform duration-300 ${activeSection === item.id ? "scale-110" :
"group-hover:scale-110"}`}
                >
                  {item.icon}
                </span>
                {!isSidebarCollapsed && (
                        <span  className="text-[10px]  font-black  uppercase  tracking-widest  animate-in  fade-in
slide-in-from-left-2 duration-300">
                    {item.label}
                  </span>
                )}
                {activeSection === item.id && !isSidebarCollapsed && (
                  <div className="ml-auto w-1.5 h-1.5 bg-background rounded-full animate-pulse"></div>
                )}
              </button>
            ))}
```

```
        </nav>

        {/* COLLAPSE TOGGLE (DESKTOP) */}
        <div className="hidden md:block px-4 mb-4">
          <button
            onClick={() => setIsSidebarCollapsed(!isSidebarCollapsed)}
                     className="w-full  flex  items-center  justify-center  p-3  rounded-2xl  text-foreground/30
hover:text-foreground hover:bg-foreground/5 transition-all"
          >
            <svg
              width="16"
              height="16"
              viewBox="0 0 24 24"
              fill="none"
              stroke="currentColor"
              strokeWidth="3"
              className={`transition-transform duration-300 ${isSidebarCollapsed ? "rotate-180" : ""}`}
            >
              <path d="M15 18l-6-6 6-6" />
            </svg>
          </button>
        </div>

        {/* USER PROFILE (BOTTOM) */}
        <div className="p-4 mt-auto border-t border-foreground/5">
          <div className="relative">
            <button
              onClick={() => setIsProfileOpen(!isProfileOpen)}
                  className={`w-full flex items-center ${isSidebarCollapsed ? "justify-center" : "gap-3 p-3"}
rounded-2xl hover:bg-foreground/5 transition-all group text-left`}
            >
               <div className="w-9 h-9 rounded-xl bg-foreground/5 border border-foreground/10 flex items-center
justify-center group-hover:bg-foreground/10 transition-all overflow-hidden shrink-0">
                {user?.user_metadata?.avatar_url ? (
                  <img
                    src={user.user_metadata.avatar_url}
                    alt="Avatar"
                    className="w-full h-full object-cover"
                  />
                ) : (
                   <span className="font-black text-foreground/40 group-hover:text-foreground transition-colors
text-xs">
                    {userInitial}
                  </span>
                )}
              </div>
              {!isSidebarCollapsed && (
                <>
                  <div className="flex flex-col min-w-0 animate-in fade-in slide-in-from-left-2 duration-300">
                    <p className="text-[10px] font-black text-foreground truncate">
                      {user?.email?.split("@")[0]}
                    </p>
                    <p className="text-[8px] font-bold text-foreground/40 uppercase tracking-widest">
                      Verified
                    </p>
                  </div>
                  <svg
                    width="12"
                    height="12"
                    viewBox="0 0 24 24"
                    fill="none"
                    stroke="currentColor"
                    strokeWidth="3"
                        className={`ml-auto transition-transform duration-300 ${isProfileOpen ? "rotate-180" :
```

```
""}`}
                  >
                    <path d="m6 9 6 6 6-6" />
                  </svg>
                </>
              )}
            </button>

            {isProfileOpen && (
              <>
                <div
                  className="fixed inset-0 z-10"
                  onClick={() => setIsProfileOpen(false)}
                ></div>
                <div
                   className={`absolute ${isSidebarCollapsed ? "left-full bottom-0 ml-2" : "bottom-full left-0
mb-2"} w-48 md:w-full bg-background border border-foreground/10 rounded-2xl shadow-2xl z-20 overflow-hidden
animate-in fade-in slide-in-from-bottom-2 duration-200`}
                >
                  <div className="p-4 border-b border-foreground/5">
                    <p className="text-[9px] font-black text-foreground/40 uppercase tracking-widest mb-1">
                      Session Details
                    </p>
                    <p className="text-[10px] font-bold text-foreground truncate">
                      {user?.email}
                    </p>
                  </div>
                  <div className="p-2">
                    <button
                      disabled
                            className="w-full text-left px-3 py-2 rounded-lg text-[10px] font-black uppercase
tracking-widest text-foreground/40 cursor-not-allowed flex items-center justify-between"
                    >
                      Settings
                      <span className="text-[7px] bg-foreground/5 px-1.5 py-0.5 rounded text-foreground/30">
                        Soon
                      </span>
                    </button>
                    <button
                      onClick={handleLogout}
                            className="w-full text-left px-3 py-2 rounded-lg text-[10px] font-black uppercase
tracking-widest text-red-500 hover:bg-red-500/5 transition-colors"
                    >
                      Log Out
                    </button>
                  </div>
                </div>
              </>
            )}
          </div>
        </div>
      </aside>

      {/* MAIN CONTENT AREA */}
      <main
        className={`flex-1 min-h-screen transition-all duration-300 ease-in-out ${
          isSidebarCollapsed ? "md:ml-20" : "md:ml-64"
        }`}
      >
        {/* DESKTOP STICKY HEADER FOR SECTION TITLE */}
        <header className="hidden md:flex h-20 sticky top-0 bg-background/50 backdrop-blur-xl z-30 items-center
px-12 border-b border-foreground/5">
          <div className="flex items-center gap-4">
            <h1 className="text-xl font-black uppercase tracking-tighter">
```

```
              {activeLabel}
            </h1>
            <div className="h-4 w-[1px] bg-foreground/10"></div>
            <div className="flex items-center gap-2">
              <div className="w-1.5 h-1.5 bg-green-500 rounded-full animate-pulse"></div>
              <span className="text-[9px] font-black text-foreground/40 uppercase tracking-widest">
                Real-time Intelligence Active
              </span>
            </div>
          </div>
        </header>

        <div className="max-w-6xl mx-auto px-6 md:px-12 pt-8 md:py-16 pb-32 md:pb-16">
          {children}
        </div>
      </main>
    </div>
  );
}
```

File: src/app/dashboard/loading.tsx

```
export default function DashboardLoading() {
  return (
    <div className="max-w-6xl mx-auto p-6 pb-20 animate-pulse">
      <div className="flex flex-col md:flex-row md:items-end justify-between gap-6 mb-12">
        <div className="space-y-3">
          <div className="h-12 w-64 bg-foreground/10 rounded-2xl"></div>
          <div className="h-4 w-48 bg-foreground/10 rounded-full"></div>
        </div>
        <div className="flex gap-4">
          <div className="h-20 w-40 bg-foreground/10 rounded-2xl"></div>
          <div className="h-20 w-40 bg-foreground/10 rounded-2xl"></div>
        </div>
      </div>

      <div className="grid grid-cols-1 lg:grid-cols-12 gap-8">
        <div className="lg:col-span-4 space-y-6">
          <div className="h-96 bg-foreground/10 rounded-[2.5rem]"></div>
          <div className="h-48 bg-foreground/10 rounded-[2.5rem]"></div>
        </div>
        <div className="lg:col-span-8 space-y-10">
          <div className="space-y-4">
            <div className="h-8 w-48 bg-foreground/10 rounded-full"></div>
            <div className="grid grid-cols-2 gap-4">
              <div className="h-32 bg-foreground/10 rounded-3xl"></div>
              <div className="h-32 bg-foreground/10 rounded-3xl"></div>
            </div>
          </div>
          <div className="space-y-4">
            <div className="h-8 w-48 bg-foreground/10 rounded-full"></div>
            <div className="space-y-4">
              <div className="h-24 bg-foreground/10 rounded-3xl"></div>
              <div className="h-24 bg-foreground/10 rounded-3xl"></div>
              <div className="h-24 bg-foreground/10 rounded-3xl"></div>
            </div>
          </div>
        </div>
      </div>
    </div>
  );
}
```

File: src/app/dashboard/page.tsx

```
"use client";

import {
  useEffect,
  useState,
  useCallback,
  useRef,
  useSyncExternalStore,
} from "react";
import {
  createDeploymentAction,
  deleteDeploymentAction,
  getDashboardSnapshotAction,
  updateDeploymentAction,
  updateLiquidityAction,
  type DashboardSnapshot,
} from "./actions";
import type { AnalyticsSummary } from "@/lib/analytics";
import {
  getTaxonomyInterpretation,
  TAXONOMY_CATEGORIES,
} from "@/lib/finance/taxonomy";
import { evaluateMetadataQuality } from "@/lib/finance/metadataQuality";
import {
  EXPECTED_RETURN_HORIZONS,
  type DeploymentAdvancedContextInput,
} from "@/lib/finance/deploymentContext";
import { AccountSection } from "./AccountSection";
import { LiabilitySection } from "./LiabilitySection";
import { IncomeSection } from "./IncomeSection";
import { GoalSection } from "./GoalSection";
import { StrategicObjectiveSection } from "./StrategicObjectiveSection";
import { BaselineSection } from "./BaselineSection";
import { RunwayCard } from "./RunwayCard";
import { PendingInflows } from "./PendingInflows";
import { HistoricalAudit } from "./HistoricalAudit";
import { TelemetryDashboard } from "@/components/dashboard/TelemetryDashboard";
import DayZeroOnboarding from "@/components/dashboard/DayZeroOnboarding";
import type {
  Account,
  Liability,
  IncomeStream,
  FinancialGoal,
  StrategicObjective,
  OperationalBaseline,
  Deployment,
} from "@/lib/analytics/types";
import { formatCurrency } from "@/lib/utils/formatters";
import { DeploymentMap } from "@/lib/utils/taxonomy";

interface LedgerState {
  deployments: Deployment[];
  accounts: Account[];
  liabilities: Liability[];
  incomeStreams: IncomeStream[];
  goals: FinancialGoal[];
  objectives: StrategicObjective[];
  baseline: OperationalBaseline[];
  analytics: AnalyticsSummary | null;
}
```

Repository Audit

```
type KairosInsight = DashboardSnapshot["kairosInsight"];

const EMPTY_ADVANCED_CONTEXT: DeploymentAdvancedContextInput = {
  associatedAccount: "",
  expectedReturnHorizon: "",
  tags: "",
};

/**
 * COMPONENT: Taxonomy Category Selector
 * Implementation of the "Taxonomy Clarity Layer".
 */
function CategorySelector({
  value,
  onChange,
  disabled,
  compact = false,
}: {
  value: string;
  onChange: (category: string) => void;
  disabled?: boolean;
  compact?: boolean;
}) {
  const interpretation = getTaxonomyInterpretation(value);

  return (
    <div className={compact ? "space-y-2" : "space-y-4"}>
      <div
        role="radiogroup"
        aria-label="Capital category"
        className={
          compact
            ? "grid grid-cols-1 gap-1.5"
            : "grid grid-cols-1 sm:grid-cols-2 gap-3"
        }
      >
        {TAXONOMY_CATEGORIES.map((category) => {
          const isSelected = value === category.value;

          return (
            <button
              key={category.value}
              type="button"
              role="radio"
              aria-checked={isSelected}
              disabled={disabled}
              onClick={() => onChange(category.value)}
                                  className={`w-full  rounded-2xl  border-2  text-left  transition-all  group
disabled:cursor-not-allowed disabled:opacity-50 ${
                compact ? "p-3" : "p-4"
              } ${
                isSelected
                  ? "border-foreground bg-foreground text-background shadow-xl scale-[1.02]"
                            :  "border-foreground/10  bg-background  text-foreground  hover:border-foreground/20
hover:bg-foreground/5"
              }`}
            >
              <div className="flex items-center justify-between">
                <span
                           className={`block  font-black  uppercase  tracking-widest  ${compact  ?  "text-[9px]"  :
"text-[10px]"}`}
                >
                  {DeploymentMap[
                    category.value as keyof typeof DeploymentMap
```

```
                ] || category.label}
              </span>
              {isSelected && (
                <span className="w-1.5 h-1.5 bg-background rounded-full animate-pulse"></span>
              )}
            </div>
            <span
              className={`mt-1 block font-bold leading-snug ${
                compact ? "text-[8px]" : "text-[9px]"
              } ${isSelected ? "text-background/70" : "text-foreground/60"}`}
            >
              {category.definition}
            </span>
          </button>
        );
      })}
    </div>

    {value !== "Unclassified" && interpretation && (
      <div className="p-4 bg-foreground/5 border-l-2 border-foreground rounded-r-2xl animate-in fade-in
slide-in-from-left-2 duration-500">
        <p className="text-[9px] font-black text-foreground/60 uppercase tracking-widest mb-1 opacity-60">
          System Interpretation
        </p>
        <p
          className={`font-bold leading-snug text-foreground ${compact ? "text-[9px]" : "text-[11px]"}`}
        >
          {interpretation}
        </p>
      </div>
    )}
  </div>
);
}

const emptySubscribe = () => () => {};

export default function Dashboard() {
  const isClient = useSyncExternalStore(
    emptySubscribe,
    () => true,
    () => false,
  );

  const [ledger, setLedger] = useState<LedgerState>({
    deployments: [],
    accounts: [],
    liabilities: [],
    incomeStreams: [],
    goals: [],
    objectives: [],
    baseline: [],
    analytics: null,
  });
  const [liquidity, setLiquidity] = useState<number>(0);
  const [category, setCategory] = useState("Unclassified");
  const [kairosInsight, setKairosInsight] = useState<KairosInsight | null>(
    null,
  );
  const [isKairosAcknowledged, setIsKairosAcknowledged] = useState(false);
  const [lastAcknowledgedAt, setLastAcknowledgedAt] = useState<string | null>(
    null,
  );
```

```
const [title, setTitle] = useState("");
const [amount, setAmount] = useState("");
const [deploymentAccountId, setDeploymentAccountId] = useState("");
const [isAdvancedContextOpen, setIsAdvancedContextOpen] = useState(false);
const [advancedContext, setAdvancedContext] =
  useState<DeploymentAdvancedContextInput>({ ...EMPTY_ADVANCED_CONTEXT });

const [isInitialLoading, setIsInitialLoading] = useState(true);
const [isActionLoading, setIsActionLoading] = useState(false);
const [isLiquidityLoading, setIsLiquidityLoading] = useState(false);
const [deletingId, setDeletingId] = useState<string | null>(null);
const [updatingId, setUpdatingId] = useState<string | null>(null);
const [isIntelligenceSyncing, setIsIntelligenceSyncing] = useState(false);

const [activeFlashInsight, setActiveFlashInsight] = useState<{
  message: string;
  action: string;
} | null>(null);

const titleMetadataQuality = evaluateMetadataQuality(title);
const showTitleQualityHint =
  title.trim().length > 0 && titleMetadataQuality.isLowQuality;

const isExecuting = useRef(false);
const fetchCount = useRef(0);
const flashTimeoutRef = useRef<NodeJS.Timeout | null>(null);

useEffect(() => {
  // Persistence Refinement: Flash insights now remain active until user dismissal
  // to ensure guidance is fully absorbed.
  return () => {
    if (flashTimeoutRef.current) clearTimeout(flashTimeoutRef.current);
  };
}, [activeFlashInsight]);

const [globalError, setGlobalError] = useState<string | null>(null);
const [formError, setFormError] = useState<string | null>(null);
const [liquidityError, setLiquidityError] = useState<string | null>(null);

const [editingId, setEditingId] = useState<string | null>(null);
const [ledgerFilter, setLedgerFilter] = useState("all");
const [isFilterOpen, setIsFilterOpen] = useState(false);
const [editForm, setEditForm] = useState({
  title: "",
  amount: "",
  category: "Unclassified",
});
const [isEditingLiquidity, setIsEditingLiquidity] = useState(false);
const [liquidityInput, setLiquidityInput] = useState("");

const applyDashboardSnapshot = useCallback((snapshot: DashboardSnapshot) => {
  if (!snapshot.authenticated) return;
  setLiquidity(snapshot.liquidity);
  setLiquidityInput(snapshot.liquidity.toString());
  setLedger({
    deployments: snapshot.deployments as Deployment[],
    accounts: snapshot.accounts,
    liabilities: snapshot.liabilities,
    incomeStreams: snapshot.incomeStreams,
    goals: snapshot.goals,
    objectives: snapshot.objectives,
    baseline: snapshot.baseline,
    analytics: snapshot.analytics,
  });
```

```
  setKairosInsight(snapshot.kairosInsight);
  if (snapshot.kairosInsight?.is_new_signal) {
    setIsKairosAcknowledged(false);
  }
}, []);

const fetchDashboardData = useCallback(async () => {
  const requestId = ++fetchCount.current;
  setGlobalError(null);
  try {
    const snapshot = await getDashboardSnapshotAction();
    if (requestId !== fetchCount.current) return;
    applyDashboardSnapshot(snapshot);
  } catch {
    setGlobalError("Connectivity failure. Intelligence layer offline.");
  } finally {
    if (requestId === fetchCount.current) setIsInitialLoading(false);
  }
}, [applyDashboardSnapshot]);

useEffect(() => {
  // eslint-disable-next-line react-hooks/set-state-in-effect
  fetchDashboardData();
}, [fetchDashboardData]);

async function handleAddDeployment(e: React.FormEvent) {
  e.preventDefault();
  if (isExecuting.current) return;
  setIsActionLoading(true);
  setFormError(null);
  isExecuting.current = true;
  setIsIntelligenceSyncing(true);

  try {
    if (category === "Unclassified")
      throw new Error("Strategic classification required before execution");

    if (Number(amount) > liquidity) {
      throw new Error("Deployment exceeds available liquidity.");
    }

    const snapshot = await createDeploymentAction({
      title,
      amount: Number(amount),
      category,
      advancedContext,
      accountId: deploymentAccountId || undefined,
    });
    setTitle("");
    setAmount("");
    setDeploymentAccountId("");
    setCategory("Unclassified");
    setAdvancedContext({ ...EMPTY_ADVANCED_CONTEXT });
    setIsAdvancedContextOpen(false);
    applyDashboardSnapshot(snapshot);
  } catch (err: unknown) {
    setFormError(err instanceof Error ? err.message : "Deployment failed");
  } finally {
    setIsActionLoading(false);
    isExecuting.current = false;
    setIsIntelligenceSyncing(false);
  }
}
```

```
  async function handleUpdateLiquidity() {
    if (isExecuting.current) return;
    setIsLiquidityLoading(true);
    setLiquidityError(null);
    isExecuting.current = true;
    setIsIntelligenceSyncing(true);
    try {
      const newAmount = Number(liquidityInput);
      if (isNaN(newAmount)) throw new Error("Invalid value");
      const snapshot = await updateLiquidityAction(newAmount);
      setIsEditingLiquidity(false);
      applyDashboardSnapshot(snapshot);
    } catch {
      setLiquidityError("Update failed");
    } finally {
      setIsLiquidityLoading(false);
      isExecuting.current = false;
      setIsIntelligenceSyncing(false);
    }
  }

  async function handleDelete(id: string) {
    if (isExecuting.current) return;
    if (
      !confirm(
          "Archive this deployment? It will be removed from your active ledger but retained for historical
analytics.",
      )
    )
      return;
    setDeletingId(id);
    isExecuting.current = true;
    setIsIntelligenceSyncing(true);
    try {
      const snapshot = await deleteDeploymentAction(id);
      applyDashboardSnapshot(snapshot);
    } catch {
      setGlobalError("Deletion interrupted.");
    } finally {
      setDeletingId(null);
      isExecuting.current = false;
      setIsIntelligenceSyncing(false);
    }
  }

  function startEdit(deployment: Deployment) {
    setEditingId(deployment.id);
    setEditForm({
      title: deployment.title,
      amount: deployment.amount.toString(),
      category: deployment.category || "Unclassified",
    });
  }

  async function handleUpdate(id: string) {
    if (isExecuting.current) return;
    setUpdatingId(id);
    isExecuting.current = true;
    setIsIntelligenceSyncing(true);
    try {
      if (!editForm.title.trim()) throw new Error("Title required");
      if (Number(editForm.amount) <= 0) throw new Error("Amount invalid");
      const snapshot = await updateDeploymentAction(id, {
        title: editForm.title,
```

```
        amount: Number(editForm.amount),
        category: editForm.category,
      });
      setEditingId(null);
      applyDashboardSnapshot(snapshot);
    } catch (err: unknown) {
      alert(err instanceof Error ? err.message : "Update failed");
    } finally {
      setUpdatingId(null);
      isExecuting.current = false;
      setIsIntelligenceSyncing(false);
    }
  }

  if (isInitialLoading) {
    return (
      <div className="max-w-6xl mx-auto p-6 pb-20 animate-pulse space-y-12">
        <div className="flex flex-col md:flex-row md:items-end justify-between gap-6 mb-12">
          <div className="space-y-3">
            <div className="h-12 w-64 bg-foreground/5 rounded-2xl"></div>
            <div className="h-4 w-48 bg-foreground/5 rounded-full"></div>
          </div>
        </div>
        <div className="h-96 bg-foreground/5 rounded-4xl"></div>
        <div className="h-96 bg-foreground/5 rounded-4xl"></div>
      </div>
    );
  }

  // Day Zero Onboarding Gate: If no financial truth exists, force established baseline.
  if (ledger.accounts.length === 0 && ledger.baseline.length === 0) {
    return <DayZeroOnboarding onComplete={applyDashboardSnapshot} />;
  }

  return (
    <div className="space-y-24">
      {/* Zone 1 ? FINANCIAL TRUTH */}
      <section id="overview" className="space-y-12">
        <div className="flex items-center gap-4 mb-8">
          <div className="h-0.5 w-12 bg-foreground/10"></div>
          <h2 className="text-2xl font-black text-foreground tracking-tighter uppercase opacity-30">
            Current Position
          </h2>
        </div>

        {/* PENDING ACTIONS LAYER */}
        <PendingInflows
          incomeStreams={ledger.incomeStreams}
          accounts={ledger.accounts}
          onSnapshot={applyDashboardSnapshot}
        />

        {/* ROW 1: PRIMARY POSITION */}
        <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
             <div className="bg-foreground/5 border border-foreground/10 rounded-2xl p-6 md:p-8 flex flex-col
justify-between">
            <span className="text-[11px] font-black text-foreground/40 uppercase tracking-[0.2em] mb-4">
              Net Worth
            </span>
            <span
               className={`text-4xl font-black tabular-nums ${ledger.analytics && ledger.analytics.netWorth < 0
? "text-red-500" : "text-foreground"}`}
            >
              {formatCurrency(ledger.analytics?.netWorth || 0)}
```

```
            </span>
            <p className="text-[10px] font-bold text-foreground/40 uppercase tracking-widest mt-4">
              Assets minus liabilities
            </p>
          </div>

           <div className="bg-foreground/5 border border-foreground/10 rounded-2xl p-6 md:p-8 flex flex-col
justify-between">
            <span className="text-[11px] font-black text-foreground/40 uppercase tracking-[0.2em] mb-4">
              Total Assets
            </span>
            <span className="text-4xl font-black tabular-nums text-foreground">
              {formatCurrency(ledger.analytics?.totalAssets || 0)}
            </span>
            <p className="text-[10px] font-bold text-foreground/40 uppercase tracking-widest mt-4">
              Monetary value possessions
            </p>
          </div>

           <div className="bg-foreground/5 border border-foreground/10 rounded-2xl p-6 md:p-8 flex flex-col
justify-between">
            <span className="text-[11px] font-black text-foreground/40 uppercase tracking-[0.2em] mb-4">
              Liabilities
            </span>
            <span className="text-4xl font-black tabular-nums text-foreground">
              {formatCurrency(ledger.analytics?.totalLiabilities || 0)}
            </span>
            <p className="text-[10px] font-bold text-foreground/40 uppercase tracking-widest mt-4">
              Debts you owe
            </p>
          </div>
        </div>

        {/* ROW 2: OPERATIONAL SURVIVAL */}
        <div className="space-y-8">
          <div className="flex items-center gap-4">
            <div className="h-0.5 w-8 bg-foreground/10"></div>
            <h2 className="text-xl font-black text-foreground tracking-tighter uppercase opacity-30">
              The Horizon
            </h2>
          </div>
          <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
            <div
                  className={`bg-foreground/5 border rounded-2xl p-6 md:p-8 relative group transition-colors
${liquidityError ? "border-red-500/50 bg-red-500/5" : "border-foreground/10"}`}
            >
              <div className="flex items-center justify-between mb-4">
                <span className="text-[11px] font-black text-foreground/40 uppercase tracking-[0.2em]">
                  {liquidityError ? "Sync Error" : "Liquidity Pool"}
                </span>
                <button
                  onClick={() =>
                    !isLiquidityLoading && setIsEditingLiquidity(true)
                  }
                       className="p-1.5 rounded-lg hover:bg-foreground/10 transition-colors text-foreground/60
hover:text-foreground"
                  title="Set starting liquid capital"
                >
                  <svg
                    width="12"
                    height="12"
                    viewBox="0 0 24 24"
                    fill="none"
                    stroke="currentColor"
```

```
                  strokeWidth="3"
                >
                  <path d="M12 20h9M16.5 3.5a2.121 2.121 0 0 1 3 3L7 19l-4 1 1-4L16.5 3.5z" />
                </svg>
              </button>
            </div>
            {isEditingLiquidity ? (
              <div className="flex flex-col gap-2">
                <input
                  autoFocus
                  type="number"
                  disabled={isLiquidityLoading}
                  value={liquidityInput}
                  onChange={(e) => setLiquidityInput(e.target.value)}
                     className="bg-background border-none rounded p-1 text-center font-black text-xl w-full
focus:outline-none disabled:opacity-50"
                />
                <button
                  disabled={isLiquidityLoading}
                  onClick={() => handleUpdateLiquidity()}
                            className="bg-foreground text-background text-[10px] font-black py-2 rounded-lg
disabled:opacity-50 uppercase tracking-widest"
                >
                  {isLiquidityLoading ? "SYNCING..." : "Confirm Liquidity"}
                </button>
              </div>
            ) : (
              <div className="flex flex-col">
                <span className="text-4xl font-black tabular-nums text-foreground">
                  {formatCurrency(liquidity)}
                </span>
                <p className="text-[10px] font-bold text-foreground/40 uppercase tracking-widest mt-3">
                  Ready to invest money
                </p>{" "}
              </div>
            )}
          </div>

            <div className="bg-foreground/5 border border-foreground/10 rounded-2xl p-6 md:p-8 flex flex-col
justify-between">
              <div className="flex items-center justify-between mb-4">
                <span className="text-[11px] font-black text-foreground/40 uppercase tracking-[0.2em]">
                  Monthly Inflow
                </span>
              </div>
              <div className="flex flex-col">
                <span className="text-4xl font-black tabular-nums text-foreground">
                  {formatCurrency(ledger.analytics?.totalMonthlyIncome || 0)}
                </span>
                <p className="text-[10px] font-bold text-foreground/40 uppercase tracking-widest mt-3">
                  Aggregate Yield
                </p>{" "}
              </div>
            </div>

            <RunwayCard
              runwayDays={ledger.analytics?.runwayDays ?? null}
              netWorth={ledger.analytics?.netWorth ?? 0}
            />
          </div>
        </div>

        {globalError && (
              <div className="bg-red-500/10 border-2 border-red-500/20 p-6 md:p-8 rounded-4xl flex flex-col
```

```
md:flex-row items-center justify-between gap-6 animate-in fade-in slide-in-from-top-4 duration-500">
            <div className="flex items-center gap-5 text-center md:text-left">
              <div className="w-16 h-16 bg-red-500/10 rounded-2xl flex items-center justify-center shrink-0">
                <svg
                  width="32"
                  height="32"
                  viewBox="0 0 24 24"
                  fill="none"
                  stroke="currentColor"
                  strokeWidth="3"
                  className="text-red-500"
                >
                  <circle cx="12" cy="12" r="10" />
                  <line x1="12" y1="8" x2="12" y2="12" />
                  <line x1="12" y1="16" x2="12.01" y2="16" />
                </svg>
              </div>
              <div>
                <h3 className="text-xl font-black text-foreground uppercase tracking-tight mb-1">
                  System Anomaly Detected
                </h3>
                      <p className="text-red-600/70 text-xs font-bold uppercase tracking-widest leading-tight
max-w-md">
                  {globalError}
                </p>
              </div>
            </div>
            <button
              disabled={isInitialLoading}
              onClick={fetchDashboardData}
                        className="bg-foreground text-background px-8 py-4 rounded-2xl font-black uppercase
tracking-widest    hover:scale-105    active:scale-95    transition-all    shadow-xl    shadow-foreground/10
disabled:opacity-50"
            >
              Attempt Re-Sync
            </button>
          </div>
        )}

        {/* SUBSECTION C ? FINANCIAL CONTAINERS, OBLIGATIONS & STRATEGY */}
        <div className="space-y-8">
          <div
            id="capital-velocity"
            className="flex items-center gap-4 scroll-mt-20"
          >
            <div className="h-0.5 w-8 bg-foreground/10"></div>
            <h2 className="text-xl font-black text-foreground tracking-tighter uppercase opacity-30">
              Capital Velocity
            </h2>
          </div>
          <div className="grid grid-cols-1 md:grid-cols-2 gap-10">
            <div
              id="accounts"
                      className="bg-background border border-foreground/10 rounded-3xl p-6 md:p-8 shadow-2xl
scroll-mt-10"
            >
              <AccountSection
                accounts={ledger.accounts}
                deployments={ledger.deployments}
                onSnapshot={applyDashboardSnapshot}
              />
            </div>
            <div
              id="liabilities"
```

```
                    className="bg-background border border-foreground/10 rounded-3xl p-6 md:p-8 shadow-2xl
scroll-mt-10"
            >
              <LiabilitySection
                liabilities={ledger.liabilities}
                onSnapshot={applyDashboardSnapshot}
              />
            </div>
            <div
              id="income"
                    className="bg-background border border-foreground/10 rounded-3xl p-6 md:p-8 shadow-2xl
scroll-mt-10"
            >
              <IncomeSection
                incomeStreams={ledger.incomeStreams}
                onSnapshot={applyDashboardSnapshot}
              />
            </div>
            <div
              id="baseline"
                    className="bg-background border border-foreground/10 rounded-3xl p-6 md:p-8 shadow-2xl
scroll-mt-10"
            >
              <BaselineSection
                baseline={ledger.baseline}
                onSnapshot={applyDashboardSnapshot}
              />
            </div>
            <div
              id="objectives"
                    className="bg-background border border-foreground/10 rounded-3xl p-6 md:p-8 shadow-2xl
scroll-mt-10"
            >
              <StrategicObjectiveSection
                objectives={ledger.objectives}
                onSnapshot={applyDashboardSnapshot}
              />
            </div>
            <div
              id="goals"
                    className="bg-background border border-foreground/10 rounded-3xl p-6 md:p-8 shadow-2xl
scroll-mt-10"
            >
              <GoalSection
                goals={ledger.goals}
                onSnapshot={applyDashboardSnapshot}
              />
            </div>
          </div>
        </div>

        {/* Strategic Fulfillment Context (Analytical Summary) */}
         <div className="bg-foreground border border-foreground/10 rounded-3xl md:rounded-[2.5rem] p-6 md:p-10
text-background shadow-2xl flex flex-col md:flex-row items-center justify-between gap-8 md:gap-10
overflow-hidden relative">
            <div className="absolute top-0 right-0 w-64 h-64 bg-background/5 blur-[80px] rounded-full -mr-32
-mt-32"></div>

          <div className="space-y-2 relative z-10 text-center md:text-left">
            <h2 className="text-xl md:text-2xl font-black uppercase tracking-[0.1em]">
              Strategic Fulfillment
            </h2>
                <p className="text-background/60 text-[11px] md:text-sm font-bold uppercase tracking-widest
max-w-lg leading-relaxed">
```

```
              Alignment of authoritative capital with defined long-term
              intentions. Calculated mean across all active objectives.
            </p>
          </div>

            <div className="flex items-center justify-between md:justify-end w-full md:w-auto gap-4 md:gap-12
relative z-10">
            <div className="text-center md:text-right">
              <p className="text-4xl md:text-5xl font-black tabular-nums">
                {Math.round(ledger.analytics?.averageGoalProgress || 0)}%
              </p>
               <p className="text-[8px] md:text-[10px] font-black text-background/40 uppercase tracking-[0.2em]
mt-1 md:mt-2">
                Fulfillment Mean
              </p>
            </div>

            {ledger.analytics && ledger.analytics.criticalGoalCount > 0 && (
              <div className="bg-orange-500 text-background px-4 md:px-6 py-3 md:py-4 rounded-xl md:rounded-2xl
shadow-xl shadow-orange-500/20">
                <p className="text-xs md:text-sm font-black uppercase tracking-widest leading-none">
                  {ledger.analytics.criticalGoalCount}
                </p>
                <p className="text-[7px] md:text-[9px] font-black uppercase tracking-tighter mt-1 opacity-80">
                  Critical
                </p>
              </div>
            )}

            <div className="bg-background/10 h-12 md:h-16 w-[1px] hidden sm:block"></div>

            <div className="text-center md:text-right">
              <p className="text-4xl md:text-5xl font-black tabular-nums">
                {ledger.goals.length + ledger.objectives.length}
              </p>
               <p className="text-[8px] md:text-[10px] font-black text-background/40 uppercase tracking-[0.2em]
mt-1 md:mt-2">
                Active Intentions
              </p>
            </div>
          </div>
        </div>
      </section>

      {/* GUIDING STRATEGIC INSIGHT (Persistent bottom-right) */}
      {activeFlashInsight && (
           <div className="fixed bottom-8 right-8 z-50 animate-in fade-in slide-in-from-right-8 duration-700
max-w-sm w-full">
                          <div  className="bg-background  border-2  border-foreground  rounded-[2rem]  p-6
shadow-[0_32px_64px_-12px_rgba(0,0,0,0.2)] text-foreground relative overflow-hidden group">
                <div className="absolute top-0 right-0 w-24 h-24 bg-foreground/5 blur-2xl rounded-full -mr-12
-mt-12 group-hover:bg-foreground/10 transition-colors"></div>

            <div className="flex items-start gap-5 relative z-10">
                      <div  className="w-12  h-12  bg-foreground  text-background  rounded-2xl  flex  items-center
justify-center shrink-0 shadow-lg">
                <svg
                  width="22"
                  height="22"
                  viewBox="0 0 24 24"
                  fill="none"
                  stroke="currentColor"
                  strokeWidth="3"
                >
```

```
                <circle cx="12" cy="12" r="10" />
                <path d="M12 16v-4M12 8h.01" />
              </svg>
            </div>

            <div className="space-y-1 pr-4">
              <span className="text-[10px] font-black uppercase tracking-[0.2em] text-foreground/40 block">
                Strategic Guidance
              </span>
              <p className="text-sm font-bold leading-relaxed text-foreground">
                {activeFlashInsight.message}
              </p>
            </div>
          </div>

           <div className="mt-6 flex items-center justify-between pt-4 border-t border-foreground/5 relative
z-10">
            <div className="flex items-center gap-2">
              <span className="w-1.5 h-1.5 bg-foreground/20 rounded-full"></span>
              <span className="text-[9px] font-black uppercase tracking-widest text-foreground/40">
                {activeFlashInsight.action}
              </span>
            </div>
            <button
              onClick={() => setActiveFlashInsight(null)}
               className="px-4 py-1.5 bg-foreground/5 hover:bg-foreground/10 rounded-lg text-[9px] font-black
uppercase tracking-widest transition-all hover:scale-105 active:scale-95"
            >
              Dismiss
            </button>
          </div>
        </div>
      </div>
    )}

    {/* Zone 2 ? DEPLOY CAPITAL (REFINED CONSOLE) */}
    <section
      id="deploy"
                  className={`transition-opacity duration-500 ${globalError && !isInitialLoading &&
ledger.deployments.length === 0 ? "opacity-40 pointer-events-none grayscale" : "opacity-100"}`}
    >
          <div className="bg-background border rounded-3xl p-6 md:p-10 shadow-2xl relative overflow-hidden
group">
               <div className="absolute -top-24 -left-24 w-48 h-48 bg-foreground/5 blur-3xl rounded-full
transition-all group-hover:bg-foreground/10"></div>
        <div className="relative z-10 max-w-3xl mx-auto">
          <div className="flex items-center gap-4 mb-10">
            <div className="w-12 h-12 bg-foreground rounded-2xl flex items-center justify-center shadow-lg">
              <svg
                width="24"
                height="24"
                viewBox="0 0 24 24"
                fill="none"
                stroke="currentColor"
                strokeWidth="3"
                className="text-background"
              >
                <path d="M12 5v14M5 12h14" />
              </svg>
            </div>
            <div className="space-y-1">
              <h2 className="text-3xl font-black text-foreground tracking-tighter uppercase">
                Deploy Capital
              </h2>
```

```
                <p className="text-foreground/40 text-[10px] font-black uppercase tracking-widest">
                  Strategic Allocation Console :: v1.0
                </p>
              </div>
            </div>

            <form onSubmit={handleAddDeployment} className="space-y-10">
              {/* Primary Intent Layer */}
              <div className="space-y-6">
                <div>
                   <label className="text-[11px] font-black text-foreground/60 uppercase tracking-[0.2em] mb-3
block ml-1">
                    Strategic Designation
                  </label>
                  <input
                    type="text"
                    disabled={isActionLoading}
                    placeholder="What is this capital achieving?"
                    value={title}
                    onChange={(e) => {
                      setTitle(e.target.value);
                      if (formError) setFormError(null);
                    }}
                              className="w-full border-2 border-foreground/10 bg-background rounded-2xl p-5
focus:outline-none focus:border-foreground transition-all text-foreground text-xl
placeholder:text-foreground/20 font-bold shadow-sm disabled:opacity-50"
                    required
                  />
                  {showTitleQualityHint && (
                       <p className="mt-3 ml-2 text-[11px] font-bold leading-snug text-orange-500/80 uppercase
tracking-tight">
                      Notice: Strategic clarity improves intelligence signal
                      quality.
                    </p>
                  )}
                </div>

                <div className="grid grid-cols-1 md:grid-cols-12 gap-10 items-start">
                  <div className="md:col-span-5 space-y-6">
                    <div>
                        <label className="text-[11px] font-black text-foreground/60 uppercase tracking-[0.2em]
mb-3 block ml-1">
                        Deployment Amount (KSh)
                      </label>
                      <input
                        type="number"
                        disabled={isActionLoading}
                        placeholder="0.00"
                        value={amount}
                        onChange={(e) => {
                          setAmount(e.target.value);
                          if (formError) setFormError(null);
                        }}
                                 className="w-full border-2 border-foreground/10 bg-background rounded-2xl p-5
focus:outline-none focus:border-foreground transition-all text-foreground text-2xl
placeholder:text-foreground/20 font-black tabular-nums shadow-sm disabled:opacity-50"
                        required
                      />
                    </div>
                  </div>

                  <div className="md:col-span-7">
                    <label className="text-[11px] font-black text-foreground/60 uppercase tracking-[0.2em] mb-3
block ml-1">
```

```
                    Strategic Classification
                  </label>
                  <CategorySelector
                    disabled={isActionLoading}
                    value={category}
                    onChange={(nextCategory) => {
                      setCategory(nextCategory);
                      if (formError) setFormError(null);
                    }}
                  />
                </div>
              </div>
            </div>

            {/* Infrastructural Layer */}
            <div className="border-t border-foreground/10 pt-6">
              <button
                type="button"
                disabled={isActionLoading}
                aria-expanded={isAdvancedContextOpen}
                aria-controls="advanced-context-drawer"
                onClick={() => setIsAdvancedContextOpen((isOpen) => !isOpen)}
                className="flex w-full items-center justify-between py-2 text-left group disabled:opacity-50"
              >
                    <span className="text-[10px] font-black uppercase tracking-[0.3em] text-foreground/40
group-hover:text-foreground/60 transition-colors">
                  Advanced Context
                </span>
                   <span className="flex items-center gap-3 text-[9px] font-black uppercase tracking-widest
text-foreground/40">
                  {isAdvancedContextOpen
                    ? "Hide Options"
                    : "Reveal Infrastructural Parameters"}
                  <span
                        className={`inline-block transition-transform duration-500 ${isAdvancedContextOpen ?
"rotate-180" : ""}`}
                    aria-hidden="true"
                  >
                    v
                  </span>
                </span>
              </button>
              <div
                id="advanced-context-drawer"
                aria-hidden={!isAdvancedContextOpen}
                          className={`grid transition-[grid-template-rows,opacity] duration-500 ease-in-out
${isAdvancedContextOpen ? "grid-rows-[1fr] opacity-100 mt-6" : "grid-rows-[0fr] opacity-0"}`}
              >
                <div className="overflow-hidden">
                  <div className="space-y-6 pb-4">
                    <div>
                       <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest
mb-2 block ml-1">
                        Funding Source
                      </label>
                      <select
                        disabled={isActionLoading || !isAdvancedContextOpen}
                        value={deploymentAccountId}
                        onChange={(e) => {
                          setDeploymentAccountId(e.target.value);
                          if (formError) setFormError(null);
                        }}
                           className="w-full border-2 border-foreground/10 bg-background rounded-xl px-5 py-4
focus:outline-none    focus:border-foreground/40    transition-colors    text-sm    text-foreground    font-bold
```

```
appearance-none disabled:opacity-50"
                        >
                          <option value="">No Deduction (Manual)</option>
                          {ledger.accounts.map((acc) => (
                            <option key={acc.id} value={acc.id}>
                              {acc.account_name} (
                              {formatCurrency(acc.current_balance)})
                            </option>
                          ))}
                        </select>
                      </div>
                      <div className="grid grid-cols-1 sm:grid-cols-2 gap-6">
                        <div>
                          <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest
mb-2 block ml-1">
                            Expected Return Horizon
                          </label>
                          <select
                            disabled={isActionLoading || !isAdvancedContextOpen}
                            value={advancedContext.expectedReturnHorizon || ""}
                            onChange={(e) => {
                              setAdvancedContext((current) => ({
                                ...current,
                                expectedReturnHorizon: e.target.value,
                              }));
                              if (formError) setFormError(null);
                            }}
                             className="w-full border-2 border-foreground/10 bg-background rounded-xl px-5 py-4
focus:outline-none focus:border-foreground/40 transition-colors text-sm text-foreground font-black uppercase
tracking-tighter appearance-none disabled:opacity-50"
                          >
                            <option value="">Unspecified</option>
                            {EXPECTED_RETURN_HORIZONS.map((horizon) => (
                              <option key={horizon.value} value={horizon.value}>
                                {horizon.label}
                              </option>
                            ))}
                          </select>
                        </div>
                        <div>
                          <label className="text-[10px] font-black text-foreground/60 uppercase tracking-widest
mb-2 block ml-1">
                            Strategic Tags
                          </label>
                          <input
                            type="text"
                            disabled={isActionLoading || !isAdvancedContextOpen}
                            placeholder="ops, recurring, essential"
                            value={
                              typeof advancedContext.tags === "string"
                                ? advancedContext.tags
                                : advancedContext.tags?.join(", ") || ""
                            }
                            onChange={(e) => {
                              setAdvancedContext((current) => ({
                                ...current,
                                tags: e.target.value,
                              }));
                              if (formError) setFormError(null);
                            }}
                             className="w-full border-2 border-foreground/10 bg-background rounded-xl px-5 py-4
focus:outline-none       focus:border-foreground/40       transition-colors       text-sm       text-foreground
placeholder:text-foreground/20 font-bold disabled:opacity-50"
                          />
```

```
                      </div>
                    </div>
                  </div>
                </div>
              </div>

              {/* Execution Action */}
              <div className="space-y-4">
                {formError && (
                    <div className="bg-red-500/10 border-2 border-red-500/20 p-5 rounded-2xl flex items-center
gap-4 animate-in fade-in zoom-in-95 duration-300">
                  <svg
                    width="20"
                    height="20"
                    viewBox="0 0 24 24"
                    fill="none"
                    stroke="currentColor"
                    strokeWidth="3"
                    className="text-red-500 shrink-0"
                  >
                    <circle cx="12" cy="12" r="10" />
                    <line x1="12" y1="8" x2="12" y2="12" />
                    <line x1="12" y1="16" x2="12.01" y2="16" />
                  </svg>
                              <p  className="text-[11px]  font-black  text-red-600  uppercase  tracking-[0.1em]
leading-tight">
                    {formError}
                  </p>
                </div>
              )}

              <div className="space-y-4">
                <button
                  type="submit"
                  disabled={isActionLoading}
                            className={`w-full  px-6  py-6  rounded-2xl  font-black  text-xl  hover:scale-[1.01]
active:scale-[0.99]  transition-all  disabled:opacity-50  shadow-2xl  uppercase  tracking-[0.2em]  ${formError  ?
"bg-red-600 text-white shadow-red-500/20" : "bg-foreground text-background shadow-foreground/20"}`}
                >
                  {isActionLoading
                    ? "INITIALIZING DEPLOYMENT..."
                    : "Execute deployment"}
                </button>
                            <p  className="text-center  text-[10px]  font-black  text-foreground/30  uppercase
tracking-[0.1em]">
                  Deployment becomes part of immutable financial truth once
                  verified.
                </p>
              </div>
            </div>
          </form>
        </div>
      </div>
    </section>

    {/* Zone 3 ? KAIROS INTELLIGENCE */}
    <section
      id="intelligence"
      className="grid grid-cols-1 lg:grid-cols-12 gap-10"
    >
      <div
           className={`lg:col-span-8 bg-foreground border rounded-3xl p-6 md:p-8 text-background shadow-2xl
min-h-64 flex flex-col justify-between transition-all duration-500 ${kairosInsight?.severity === "critical" ?
```

```
"border-orange-500/50 ring-4 ring-orange-500/10" : "border-background/10"} ${isIntelligenceSyncing ?
"opacity-70 grayscale scale-[0.98]" : "opacity-100 scale-100"}`}
        >
          <div>
            {/* LAYER A ? STRATEGIC STATUS BAR (Refined v5E) */}
               <div className="flex flex-wrap items-center justify-between gap-4 border-b border-background/10
pb-4 mb-6">
              <div className="flex items-center gap-6">
                <div className="flex items-center gap-2">
                  <div
                     className={`w-2 h-2 rounded-full ${kairosInsight?.severity === "critical" ? "bg-orange-500
animate-pulse" : "bg-background/20"}`}
                  ></div>
                  <span className="text-[10px] font-black uppercase tracking-[0.2em] text-background">
                    {kairosInsight?.category?.replace("_", " ") ||
                      "STRATEGIC EVALUATION"}
                  </span>
                </div>
                <div className="hidden sm:flex items-center gap-2">
                  <span className="text-[10px] font-black uppercase tracking-widest text-background/60">
                    Efficiency: {kairosInsight?.capitalEfficiency ?? 100}%
                  </span>
                </div>
                <div className="hidden sm:flex items-center gap-2">
                  <span className="text-[10px] font-black uppercase tracking-widest text-background/60">
                    Runway:{" "}
                    {kairosInsight?.runway !== undefined &&
                    kairosInsight.runway !== null
                      ? `${Math.round(kairosInsight.runway)} Days`
                      : "Infinite"}
                  </span>
                </div>
              </div>
              <div className="flex items-center gap-6">
                {kairosInsight && (
                  <span
                     className={`text-[10px] font-black uppercase tracking-widest ${kairosInsight.severity ===
"critical" ? "text-orange-500" : kairosInsight.severity === "warning" ? "text-yellow-500/80" :
"text-background/60"}`}
                  >
                    {kairosInsight.severity.toUpperCase()}
                  </span>
                )}
                <span className="text-[10px] font-black uppercase tracking-widest text-background/40">
                  Last strategic evaluation:{" "}
                  {isClient && kairosInsight
                    ? new Date(kairosInsight.timestamp).toLocaleTimeString([], {
                        hour: "2-digit",
                        minute: "2-digit",
                        second: "2-digit",
                      })
                    : "--:--:--"}
                </span>
              </div>
            </div>

            {/* LAYER B ? PRIMARY ASSESSMENT & SILENCE STATE */}
            <div className="space-y-6">
              {kairosInsight ? (
                kairosInsight.isSilent ? (
                  <div className="py-4">
                    <p className="text-base font-bold leading-tight text-background/60 italic">
                      No material structural deterioration detected since
                      previous evaluation.
```

```
                  </p>
                </div>
              ) : (
                <div
                  className={`transition-all duration-700 ${kairosInsight.is_new_signal ? "animate-in fade-in
slide-in-from-bottom-2" : ""}`}
                >
                  <p className="text-base md:text-lg font-bold leading-tight text-background">
                       <span className="text-background/40 font-black uppercase text-[10px] tracking-tighter
not-italic block mb-2">
                         Strategic Assessment:
                       </span>
                       {kairosInsight.message}
                  </p>

                  {/* LAYER C ? SUPPORTING SIGNALS (Deterministic Telemetry) */}
                  {kairosInsight.supportingSignals &&
                    kairosInsight.supportingSignals.length > 0 && (
                      <div className="mt-6 space-y-3">
                        {kairosInsight.supportingSignals.map(
                          (signal, idx) => (
                            <p
                              key={idx}
                                  className="text-[11px] font-bold leading-snug text-background/60 border-l-2
border-background/20 pl-3"
                            >
                              {signal}
                            </p>
                          ),
                        )}
                      </div>
                    )}
                </div>
              )
            ) : (
              <div className="space-y-3 opacity-20">
                <div className="h-4 bg-background/20 rounded-full w-full"></div>
                <div className="h-4 bg-background/20 rounded-full w-3/4"></div>
                <p className="text-[10px] font-black uppercase mt-6 tracking-widest text-background">
                  Orchestrating strategic evaluation...
                </p>
              </div>
            )}
          </div>

          {/* ACTIVE SIGNAL MEMORY */}
          {kairosInsight &&
            (kairosInsight.severity === "critical" ||
              kairosInsight.severity === "warning") &&
            !kairosInsight.isSilent && (
              <div
                        className={`mt-8 p-5 border rounded-2xl flex flex-col md:flex-row md:items-center
justify-between gap-4 animate-in fade-in duration-700 ${kairosInsight.severity === "critical" ?
"border-orange-500 bg-orange-500/10 shadow-[0_0_20px_rgba(249,115,22,0.1)]" : "border-orange-500/30
bg-orange-500/5"}`}
                >
                  <div>
                    <p
                      className={`text-[9px] font-black uppercase tracking-widest mb-1 ${kairosInsight.severity
=== "critical" ? "text-orange-500" : "text-orange-400"}`}
                    >
                      {kairosInsight.severity === "critical"
                        ? "Critical Operational Breach"
                        : "Active Operational Signal"}
```

```
                  </p>
                  <p className="text-xs font-bold text-background/80">
                    {kairosInsight.category === "solvency_pressure"
                      ? "Liquidity shortfall detected. Immediate structural adjustment required."
                      : kairosInsight.category === "objective_starvation"
                        ? "Strategic objective stagnation detected. Review funding velocity."
                        : kairosInsight.category === "capital_efficiency"
                          ? "Capital allocation conflict detected. Review deployment behavior."
                          : "Material strategic shift detected. Review required."}
                  </p>
                </div>
                <button
                  onClick={() => {
                    const targetMap: Record<string, string> = {
                      solvency_pressure: "capital-velocity",
                      objective_starvation: "capital-velocity",
                      capital_efficiency: "ledger",
                      strategic_alignment: "overview",
                    };
                    const targetId =
                      targetMap[kairosInsight.category] || "overview";

                    // Set Flash Context
                    setActiveFlashInsight({
                      message: kairosInsight.message,
                      action: `Reviewing ${targetId.replace("_", " ")}`,
                    });

                    // Scroll to target
                    const element = document.getElementById(targetId);
                    if (element) {
                      element.scrollIntoView({
                        behavior: "smooth",
                        block: "start",
                      });
                    }
                  }}
                        className={`px-5 py-2.5 rounded-xl text-[9px] font-black uppercase tracking-widest transition-colors shrink-0 flex items-center gap-2 ${kairosInsight.severity === "critical" ? "bg-orange-500 text-background hover:bg-orange-600 shadow-lg shadow-orange-500/20" : "bg-background/10 hover:bg-background/20 text-background"}`}
                >
                  <span>
                    {kairosInsight.category === "solvency_pressure"
                      ? "Adjust Objectives"
                      : kairosInsight.category === "objective_starvation"
                        ? "View Objectives"
                        : kairosInsight.category === "capital_efficiency"
                          ? "Adjust Allocation"
                          : "Investigate Signal"}
                  </span>
                  <svg
                    width="10"
                    height="10"
                    viewBox="0 0 24 24"
                    fill="none"
                    stroke="currentColor"
                    strokeWidth="4"
                  >
                    <path d="M5 12h14M12 5l7 7-7 7" />
                  </svg>
                </button>
              </div>
            )}
```

```
          </div>

          {/* LAYER D ? SYSTEM TELEMETRY */}
          {ledger.analytics && (
            <div className="mt-12 grid grid-cols-2 md:grid-cols-4 gap-4 border-t border-background/10 pt-6">
              <div>
                <p className="text-[8px] font-black uppercase tracking-widest text-background/40 mb-1">
                  Liquidity baseline
                </p>
                <p className="text-[11px] font-black tabular-nums text-background/80">
                  {formatCurrency(liquidity)}
                </p>
              </div>
              <div>
                <p className="text-[8px] font-black uppercase tracking-widest text-background/40 mb-1">
                  Commitment pressure
                </p>
                <p className="text-[11px] font-black tabular-nums text-background/80">
                  {formatCurrency(ledger.analytics?.totalLiabilities || 0)}
                </p>
              </div>
              <div>
                <p className="text-[8px] font-black uppercase tracking-widest text-background/40 mb-1">
                  Monthly Inflow
                </p>
                <p className="text-[11px] font-black tabular-nums text-background/80">
                  {formatCurrency(ledger.analytics?.totalMonthlyIncome || 0)}
                </p>
              </div>
              <div>
                <p className="text-[8px] font-black uppercase tracking-widest text-background/40 mb-1">
                  Horizon stability
                </p>
                <p className="text-[11px] font-black tabular-nums text-background/80">
                  {kairosInsight?.runway !== null ? "Evaluative" : "Structural"}
                </p>
              </div>
            </div>
          )}
        </div>

        {/* Capital Allocation Section (Contextual Interpretation) */}
        <div id="capital-purpose" className="lg:col-span-4 space-y-6">
          {ledger.analytics && ledger.analytics.totalDeployed > 0 && (
            <div className="space-y-6 animate-in fade-in duration-500">
              <div className="flex items-center gap-3">
                <h2 className="text-xl font-black text-foreground tracking-tight uppercase">
                  Capital Purpose
                </h2>
                <div className="h-0.5 w-8 bg-foreground/10"></div>
              </div>
              <div className="grid grid-cols-1 gap-4">
                {Object.entries(ledger.analytics.categoryBreakdown)
                  .sort(([, a], [, b]) => b - a)
                  .map(([cat, amt]) => {
                    const percentage =
                      (amt / ledger.analytics!.totalDeployed) * 100;
                    return (
                      <div
                        key={cat}
                           className="bg-background border rounded-3xl p-5 shadow-sm hover:border-foreground/20
transition-all group"
                      >
                        <div className="flex justify-between items-end mb-3">
```

```
                        <div>
                          <span className="text-[9px] font-black text-foreground/60 uppercase tracking-widest
block mb-1">
                            {DeploymentMap[
                              cat as keyof typeof DeploymentMap
                            ] || cat}
                          </span>
                          <span className="text-base font-black text-foreground tabular-nums">
                            {formatCurrency(amt)}
                          </span>
                        </div>
                        <span className="text-xs font-black text-foreground/40">
                          {Math.round(percentage)}%
                        </span>
                      </div>
                      <div className="w-full h-1 bg-foreground/5 rounded-full overflow-hidden">
                        <div
                          className="h-full bg-foreground transition-all duration-1000 ease-out"
                          style={{ width: `${percentage}%` }}
                        ></div>
                      </div>
                    </div>
                  );
                })}
              </div>
            </div>
          )}
        </div>
      </section>

      {/* Zone 4 ? CHRONOLOGICAL LEDGER STREAM */}
      <section id="ledger" className="space-y-8">
        <div className="flex items-center justify-between mb-4">
          <div className="flex items-center gap-3">
            <h2 className="text-3xl font-black text-foreground tracking-tight uppercase">
              Capital purpose
            </h2>
            <div className="h-0.5 w-16 bg-foreground/10"></div>
          </div>
          <div className="flex items-center gap-4 relative">
            <button
              onClick={() => setIsFilterOpen(!isFilterOpen)}
                  className={`flex items-center gap-2 px-4 py-2 rounded-xl border transition-all text-[10px]
font-black uppercase tracking-widest ${
                ledgerFilter !== "all"
                  ? "bg-foreground text-background border-foreground"
                  : "bg-background text-foreground/60 border-foreground/10 hover:border-foreground/20"
              }`}
            >
              <svg
                width="12"
                height="12"
                viewBox="0 0 24 24"
                fill="none"
                stroke="currentColor"
                strokeWidth="3"
              >
                <path d="M22 3H2l8 9.46V19l4 2v-8.54L22 3z" />
              </svg>
              {ledgerFilter === "all" ? "Filter" : ledgerFilter}
            </button>

            {isFilterOpen && (
              <>
```

```
                <div
                  className="fixed inset-0 z-10"
                  onClick={() => setIsFilterOpen(false)}
                ></div>
                 <div className="absolute right-0 top-full mt-2 w-48 bg-background border border-foreground/10
rounded-2xl shadow-2xl z-20 overflow-hidden animate-in fade-in zoom-in-95 duration-200">
                  <div className="p-2 space-y-1">
                    <button
                      onClick={() => {
                        setLedgerFilter("all");
                        setIsFilterOpen(false);
                      }}
                           className={`w-full text-left px-3 py-2 rounded-lg text-[10px] font-black uppercase
tracking-widest transition-colors ${
                        ledgerFilter === "all"
                          ? "bg-foreground text-background"
                          : "text-foreground/60 hover:bg-foreground/5"
                      }`}
                    >
                      All Signals
                    </button>
                    {TAXONOMY_CATEGORIES.map((cat) => (
                      <button
                        key={cat.value}
                        onClick={() => {
                          setLedgerFilter(cat.value);
                          setIsFilterOpen(false);
                        }}
                            className={`w-full text-left px-3 py-2 rounded-lg text-[10px] font-black uppercase
tracking-widest transition-colors ${
                          ledgerFilter === cat.value
                            ? "bg-foreground text-background"
                            : "text-foreground/60 hover:bg-foreground/5"
                        }`}
                      >
                        {DeploymentMap[
                          cat.value as keyof typeof DeploymentMap
                        ] || cat.label}
                      </button>
                    ))}
                    <button
                      onClick={() => {
                        setLedgerFilter("Unclassified");
                        setIsFilterOpen(false);
                      }}
                           className={`w-full text-left px-3 py-2 rounded-lg text-[10px] font-black uppercase
tracking-widest transition-colors ${
                        ledgerFilter === "Unclassified"
                          ? "bg-foreground text-background"
                          : "text-foreground/60 hover:bg-foreground/5"
                      }`}
                    >
                      Unclassified
                    </button>
                  </div>
                </div>
              </>
            )}

                     <span  className="hidden  md:inline  text-[10px]  font-black  text-foreground/40  uppercase
tracking-widest ml-2">
                Audit Trail :: Verified
              </span>
            </div>
```

```
        </div>

        {ledger.deployments.length === 0 && !globalError ? (
                <div  className="bg-foreground/5  border-2  border-dashed  border-foreground/10  rounded-4xl  p-16
text-center animate-in fade-in slide-in-from-bottom-4 duration-1000">
            <div className="max-w-md mx-auto">
                <div className="w-20 h-20 bg-foreground/5 rounded-3xl flex items-center justify-center mx-auto
mb-8 text-foreground">
                <svg
                  width="40"
                  height="40"
                  viewBox="0 0 24 24"
                  fill="none"
                  stroke="currentColor"
                  strokeWidth="2"
                >
                  <path d="M12 2v20M17 5H9.5a3.5 3.5 0 0 0 0 7h5a3.5 3.5 0 0 1 0 7H6" />
                </svg>
              </div>
              <h2 className="text-3xl font-black tracking-tighter text-foreground uppercase mb-4">
                Establish Financial Truth
              </h2>
              <p className="text-foreground/60 text-sm font-bold uppercase tracking-widest leading-relaxed">
                Axiom is observing your capital behavior. Deployments will
                materialize here as an immutable audit trail.
              </p>
            </div>
          </div>
        ) : (
          <div className="grid grid-cols-1 gap-4">
            {ledger.deployments
              .filter(
                (d) => ledgerFilter === "all" || d.category === ledgerFilter,
              )
              .map((deployment) => (
                <div
                  key={deployment.id}
                                  className="bg-background  border  rounded-4xl  p-6  shadow-sm  hover:shadow-2xl
hover:border-foreground/20 transition-all flex flex-col gap-4 group"
                >
                  {editingId === deployment.id ? (
                    <div className="space-y-4">
                      <div className="grid grid-cols-2 gap-4">
                        <input
                          type="text"
                          disabled={updatingId === deployment.id}
                          value={editForm.title}
                          onChange={(e) =>
                            setEditForm({
                              ...editForm,
                              title: e.target.value,
                            })
                          }
                               className="bg-foreground/5 border-none rounded-xl p-2 text-foreground font-bold
disabled:opacity-50"
                        />
                        <input
                          type="number"
                          disabled={updatingId === deployment.id}
                          value={editForm.amount}
                          onChange={(e) =>
                            setEditForm({
                              ...editForm,
                              amount: e.target.value,
```

```
                          })
                        }
                            className="bg-foreground/5 border-none rounded-xl p-2 text-foreground font-black
disabled:opacity-50"
                      />
                    </div>
                    <div className="flex flex-col gap-3 sm:flex-row sm:items-start sm:justify-between">
                      <div className="min-w-0 flex-1">
                        <CategorySelector
                          disabled={updatingId === deployment.id}
                          value={editForm.category}
                          onChange={(nextCategory) =>
                            setEditForm({
                              ...editForm,
                              category: nextCategory,
                            })
                          }
                          compact
                        />
                      </div>
                      <div className="flex shrink-0 gap-2 pt-1">
                        <button
                          disabled={updatingId === deployment.id}
                          onClick={() => setEditingId(null)}
                                      className="text-xs font-black text-foreground/60 uppercase px-3 py-1
disabled:opacity-50"
                        >
                          Cancel
                        </button>
                        <button
                          disabled={updatingId === deployment.id}
                          onClick={() => handleUpdate(deployment.id)}
                            className="text-xs font-black bg-foreground text-background rounded-lg px-4 py-1
uppercase disabled:opacity-50"
                        >
                          {updatingId === deployment.id
                            ? "SAVING..."
                            : "Save"}
                        </button>
                      </div>
                    </div>
                  </div>
                ) : (
                  <div className="flex flex-col md:flex-row md:items-center justify-between gap-4">
                    <div className="flex items-center gap-5">
                      <div
                           className={`w-14 h-14 bg-foreground/5 rounded-2xl flex items-center justify-center
transition-colors group-hover:bg-foreground group-hover:text-background ${deletingId === deployment.id ?
"animate-pulse" : ""}`}
                      >
                        <svg
                          width="24"
                          height="24"
                          viewBox="0 0 24 24"
                          fill="none"
                          stroke="currentColor"
                          strokeWidth="2.5"
                        >
                          <path d="M12 2v20M17 5H9.5a3.5 3.5 0 0 0 0 7h5a3.5 3.5 0 0 1 0 7H6" />
                        </svg>
                      </div>
                      <div>
                        <div className="flex items-center gap-2 mb-1">
                          <h3 className="font-black text-xl text-foreground transition-colors leading-none">
```

```
                            {deployment.title}
                          </h3>
                             <span className="text-[8px] font-black px-2 py-0.5 bg-foreground/5 rounded-full
uppercase tracking-tighter text-foreground/40">
                            {DeploymentMap[
                              deployment.category as keyof typeof DeploymentMap
                            ] ||
                              deployment.category ||
                              "Unclassified"}
                          </span>
                          {deployment.account_id && (
                             <span className="text-[8px] font-black px-2 py-0.5 bg-foreground/10 rounded-full
uppercase tracking-tighter text-foreground/60">
                              via{" "}
                              {ledger.accounts.find(
                                (a) => a.id === deployment.account_id,
                              )?.account_name || "Source"}
                            </span>
                          )}
                        </div>
                               <div className="flex items-center gap-3 text-xs text-foreground/60 font-bold
uppercase tracking-tighter">
                          <span>
                            {isClient
                              ? new Date(
                                  deployment.created_at,
                                ).toLocaleDateString(undefined, {
                                  month: "short",
                                  day: "numeric",
                                  year: "numeric",
                                })
                              : "--- --, ----"}
                          </span>
                          <span className="w-1 h-1 bg-foreground/20 rounded-full"></span>
                          <span>
                            {isClient
                              ? new Date(
                                  deployment.created_at,
                                ).toLocaleTimeString(undefined, {
                                  hour: "2-digit",
                                  minute: "2-digit",
                                })
                              : "--:--"}
                          </span>
                        </div>
                      </div>
                    </div>
                    <div className="text-right flex flex-col items-end">
                      <p className="font-black text-2xl tabular-nums text-foreground tracking-tighter">
                        {formatCurrency(deployment.amount)}
                      </p>
                      <div className="mt-1 flex items-center gap-3">
                        <button
                          disabled={deletingId !== null}
                          onClick={() => startEdit(deployment)}
                              className="text-[10px] font-black text-foreground/40 uppercase tracking-widest
hover:text-foreground disabled:opacity-30"
                        >
                          Edit
                        </button>
                        <button
                          disabled={deletingId !== null}
                          onClick={() => handleDelete(deployment.id)}
                              className="text-[10px] font-black text-foreground/40 uppercase tracking-widest
```

```
hover:text-red-500 disabled:opacity-30"
                          >
                            {deletingId === deployment.id
                              ? "ARCHIVING..."
                              : "Archive"}
                          </button>
                          <div className="px-3 py-1 bg-green-500/10 rounded-full ml-2">
                            <span className="text-[10px] font-black text-green-600 uppercase tracking-widest">
                              Verified
                            </span>
                          </div>
                        </div>
                      </div>
                    </div>
                  )}
                </div>
              ))}
          </div>
        )}
      </section>

      {/* Zone 5 ? FORENSIC AUDIT */}
      <section id="audit" className="pb-12">
        <div className="bg-background border border-foreground/5 rounded-[3rem] p-8 md:p-12 shadow-sm">
          <HistoricalAudit />
        </div>
      </section>

      {/* Zone 6 ? SYSTEM OBSERVABILITY */}
      <section id="observability" className="pb-24 space-y-12">
        <div className="flex items-center gap-4 mb-8">
          <div className="h-0.5 w-12 bg-foreground/10"></div>
          <h2 className="text-2xl font-black text-foreground tracking-tighter uppercase opacity-30">
            System Observability
          </h2>
        </div>
        <div className="bg-background border border-foreground/5 rounded-[3rem] p-8 md:p-12 shadow-sm">
          <TelemetryDashboard />
        </div>
      </section>
    </div>
  );
}
```

File: src/app/globals.css

```
@import "tailwindcss";

:root {
    --background: #ffffff;
    --foreground: #171717;
    --input-border: #e5e7eb;
    --input-placeholder: #9ca3af;
    --input-label: #6b7280;
}

/* stylelint-disable at-rule-no-unknown */
@theme {
    --color-background: var(--background);
    --color-foreground: var(--foreground);
    --font-sans: var(--font-geist-sans);
    --font-mono: var(--font-geist-mono);
}
```

```
@media (prefers-color-scheme: dark) {
    :root {
        --background: #0a0a0a;
        --foreground: #ededed;
        --input-border: #333333;
        --input-placeholder: #666666;
        --input-label: #999999;
    }
}

body {
    background: var(--background);
    color: var(--foreground);
    font-family: var(--font-sans), ui-sans-serif, system-ui, sans-serif;
}
```

File: src/app/layout.tsx

```
import type { Metadata } from "next";
import { Geist, Geist_Mono } from "next/font/google";
import "./globals.css";

const geistSans = Geist({
  variable: "--font-geist-sans",
  subsets: ["latin"],
});

const geistMono = Geist_Mono({
  variable: "--font-geist-mono",
  subsets: ["latin"],
});

export const metadata: Metadata = {
  title: "AXIOM // Intelligence Layer",
  description: "Strategic Financial Truth System",
};

export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html
      lang="en"
      className={`${geistSans.variable} ${geistMono.variable} h-full antialiased`}
    >
          <body  className="min-h-full  flex  flex-col  bg-background  text-foreground  selection:bg-foreground
selection:text-background">
        {children}
      </body>
    </html>
  );
}
```

File: src/app/page.tsx

```
"use client";

import { useEffect } from "react";
import { useRouter } from "next/navigation";
```

```
import Link from "next/link";
import { supabase } from "@/lib/supabase";
import { LandingTerminal } from "@/components/landing/LandingTerminal";
import { RunwaySimulator } from "@/components/landing/RunwaySimulator";

export default function LandingPage() {
  const router = useRouter();

  useEffect(() => {
    const {
      data: { subscription },
    } = supabase.auth.onAuthStateChange((event, session) => {
      if (session) {
        router.push("/dashboard");
      }
    });

    return () => {
      subscription.unsubscribe();
    };
  }, [router]);

  return (
       <main  className="bg-black  text-white  min-h-screen  font-sans  selection:bg-white  selection:text-black
overflow-x-hidden">
      {/* SECTION 1: THE HERO (The Hook) */}
       <section className="min-h-screen flex flex-col justify-center px-6 md:px-24 max-w-7xl mx-auto relative
overflow-hidden">
        {/* Subtle Radial Gradient Background */}
            <div  className="absolute  top-1/2  left-1/2  -translate-x-1/2  -translate-y-1/2  w-[800px]  h-[800px]
bg-white/[0.03] rounded-full blur-[120px] pointer-events-none" />
           <div className="absolute top-0 right-0 w-1/2 h-screen bg-gradient-to-l from-white/5 to-transparent
pointer-events-none" />

        <div className="space-y-12 relative z-10 py-32">
              <div  className="inline-block  border  border-white/20  px-3  py-1  text-[10px]  font-mono  uppercase
tracking-[0.3em] text-white/60 mb-4 animate-in fade-in duration-700">
            Axiom Intelligence Layer // v1.0
          </div>
                  <h1  className="text-6xl  md:text-8xl  font-black  tracking-tighter  uppercase  leading-[0.9]
lg:text-[10rem] animate-in fade-in slide-in-from-bottom-8 duration-700 delay-100 fill-mode-both">
            Stop tracking <br />
            your expenses. <br />
            It's keeping <br />
            you broke.
          </h1>
             <p  className="max-w-2xl  text-xl  md:text-2xl  text-white/60  leading-relaxed  font-light  animate-in
fade-in slide-in-from-bottom-8 duration-700 delay-200 fill-mode-both">
            Traditional finance relies on passive tracking. Axiom is a
            deterministic engine that forces every shilling into a strict
            return-based taxonomy. Take control of your liquidity.
          </p>
                    <div  className="pt-8  animate-in  fade-in  slide-in-from-bottom-8  duration-700  delay-300
fill-mode-both">
            <Link
              href="/signup"
                        className="inline-block  bg-white  text-black  px-12  py-6  text-sm  font-black  uppercase
tracking-widest hover:bg-white/90 transition-all hover:scale-[1.02] active:scale-[0.98]"
            >
              Initialize Day Zero
            </Link>
          </div>
        </div>
      </section>
```

```
      {/* SECTION 2: THE PROBLEM (The Pain) */}
      <section className="py-64 px-6 md:px-24 max-w-7xl mx-auto border-t border-white/10">
        <div className="grid md:grid-cols-2 gap-24 items-start">
          <div className="text-[10px] font-mono uppercase tracking-[0.3em] text-white/40 sticky top-32">
            01 // The Systemic Failure
          </div>
          <div className="space-y-12">
            <h2 className="text-4xl md:text-6xl font-bold tracking-tighter uppercase leading-tight">
              Inflation is silent theft. <br />
              Cash is a depreciating liability.
            </h2>
            <p className="text-2xl text-white/60 leading-relaxed font-light">
              If you are leaving your liquidity in an M-Pesa account or a
              zero-interest checking account, you are losing purchasing power
              daily. The Kenyan ecosystem demands strategic deployment, not
              passive saving.
            </p>
          </div>
        </div>
      </section>

      {/* SECTION 3: THE PHILOSOPHY (The Pivot) */}
      <section className="py-64 px-6 md:px-24 max-w-7xl mx-auto border-t border-white/10 bg-white/5">
        <div className="grid md:grid-cols-2 gap-24 items-start">
          <div className="text-[10px] font-mono uppercase tracking-[0.3em] text-white/40 sticky top-32">
            02 // The Five Taxonomy Gates
          </div>
          <div className="space-y-12">
            <h2 className="text-4xl md:text-6xl font-bold tracking-tighter uppercase leading-tight">
              Axiom rejects <br />
              passive spending.
            </h2>
            <p className="text-2xl text-white/60 leading-relaxed font-light">
              Every outbound transaction must be classified: Is it an Asset, a
              Skill, Leverage, Experience, or Leakage?
            </p>
            <div className="grid grid-cols-1 gap-4 font-mono text-sm">
              {[
                { label: "ASSET", desc: "Capital that generates yield." },
                {
                  label: "SKILL",
                  desc: "Investment in human capital / earning power.",
                },
                {
                  label: "LEVERAGE",
                  desc: "Tools that multiply output per hour.",
                },
                { label: "EXPERIENCE", desc: "High-value memory equity." },
                { label: "LEAKAGE", desc: "Unstructured wealth destruction." },
              ].map((gate) => (
                <div
                  key={gate.label}
                   className="border border-white/10 p-8 flex justify-between items-center group hover:bg-white
hover:text-black transition-all cursor-default"
                >
                  <span className="font-black tracking-widest text-lg">
                    {gate.label}
                  </span>
                   <span className="text-[10px] opacity-40 group-hover:opacity-100 transition-opacity uppercase
text-right ml-4">
                    {gate.desc}
                  </span>
                </div>
```

```
            ))}
          </div>
        </div>
      </div>
    </section>

    {/* SECTION 4: THE SIMULATOR (Interactive Hook) */}
          <section  className="py-64  px-6  md:px-24  max-w-7xl  mx-auto  border-t  border-white/10  relative
overflow-hidden">
           <div  className="absolute  top-1/2  left-1/2  -translate-x-1/2  -translate-y-1/2  w-[600px]  h-[600px]
bg-white/[0.02] rounded-full blur-[100px] pointer-events-none" />
      <div className="grid md:grid-cols-2 gap-24 items-center relative z-10">
        <div className="space-y-12">
          <div className="text-[10px] font-mono uppercase tracking-[0.3em] text-white/40">
            03 // Operational Runway
          </div>
          <h2 className="text-4xl md:text-6xl font-bold tracking-tighter uppercase leading-tight">
            Calculate your <br />
            structural <br />
            solvency.
          </h2>
          <p className="text-2xl text-white/60 leading-relaxed font-light">
            How long would you survive if your primary income stream collapsed
            today? Axiom's deterministic engine measures your runway with
            clinical precision.
          </p>
        </div>
        <div>
          <RunwaySimulator />
        </div>
      </div>
    </section>

    {/* SECTION 5: THE ENGINE (The Proof / Terminal) */}
    <section className="py-64 px-6 md:px-24 max-w-7xl mx-auto border-t border-white/10 relative">
      <div className="absolute inset-0 bg-white/[0.01] pointer-events-none" />
      <div className="grid md:grid-cols-2 gap-24 items-center relative z-10">
        <div className="space-y-12">
          <div className="text-[10px] font-mono uppercase tracking-[0.3em] text-white/40">
            04 // Kairos Intelligence
          </div>
          <h2 className="text-4xl md:text-6xl font-bold tracking-tighter uppercase leading-tight">
            Meet Kairos. <br />
            Your clinical <br />
            financial analyst.
          </h2>
          <p className="text-2xl text-white/60 leading-relaxed font-light">
            No hallucinations. Just clinical, mathematical truth about your
            structural solvency and income concentration. Watch Kairos audit a
            sample deployment in real-time.
          </p>
        </div>
        <div>
          <LandingTerminal />
        </div>
      </div>
    </section>

    {/* SECTION 6: FINAL CTA */}
     <section className="py-64 px-6 md:px-24 text-center border-t border-white/10 bg-gradient-to-b from-black
to-[#0a0a0a]">
      <div className="max-w-4xl mx-auto space-y-16">
        <h2 className="text-6xl md:text-9xl font-black tracking-tighter uppercase leading-[0.9]">
          Ready to deploy <br />
```

```
            your capital?
          </h2>
          <div className="flex flex-col items-center gap-12">
            <Link
              href="/signup"
                       className="inline-block bg-white text-black px-16 py-8 text-sm font-black uppercase
tracking-widest       hover:bg-white/90       transition-all       hover:scale-[1.05]       active:scale-[0.95]
shadow-[0_0_50px_rgba(255,255,255,0.1)]"
            >
              Establish Your Baseline
            </Link>
            <div className="h-32 w-[1px] bg-gradient-to-b from-white/20 to-transparent" />
            <div className="text-[10px] font-mono uppercase tracking-[0.5em] text-white/20">
              Axiom // Strategic Solvency Protocol
            </div>
          </div>
        </div>
      </section>

      {/* FOOTER */}
         <footer  className="py-12  px-6  md:px-24  border-t  border-white/5  text-[10px]  font-mono  text-white/20
uppercase tracking-widest flex flex-col md:flex-row justify-between items-center gap-8">
        <div>© 2026 Axiom Labs // All Rights Reserved</div>
        <div className="flex gap-8">
          <span>Encrypted Session</span>
          <span>Kenya Ecosystem Optimized</span>
        </div>
      </footer>
    </main>
  );
}
```

File: src/app/signup/page.tsx

```
"use client";

import { useEffect, useState } from "react";
import { useRouter } from "next/navigation";
import { Auth } from "@supabase/auth-ui-react";
import { ThemeSupa } from "@supabase/auth-ui-shared";
import { supabase } from "@/lib/supabase";
import Link from "next/link";

export default function SignUp() {
  const router = useRouter();
  const [authError, setAuthError] = useState<string | null>(null);

  useEffect(() => {
    const {
      data: { subscription },
    } = supabase.auth.onAuthStateChange((event, session) => {
      if (session) {
        router.push("/dashboard");
      }
    });

    return () => {
      subscription.unsubscribe();
    };
  }, [router]);

  const handlePasskeyAuth = async () => {
    setAuthError(null);
```

```
    try {
      const { error } = await supabase.auth.signInWithPasskey();
      if (error) {
        if (
          error.message.includes("not found") ||
          error.message.includes("failed")
        ) {
          setAuthError(
            "BIOMETRIC HANDSHAKE FAILED. FALLBACK TO MANUAL DECRYPTION KEY REQUIRED.",
          );
        } else {
          setAuthError(`SYSTEM ERROR: ${error.message.toUpperCase()}`);
        }
        return;
      }
      router.push("/dashboard");
    } catch (err) {
      setAuthError("CRITICAL BIOMETRIC FAILURE. INITIALIZE MANUAL OVERRIDE.");
    }
  };

  return (
        <main  className="min-h-screen  flex  items-center  justify-center  bg-black  text-white  p-6  relative
overflow-hidden">
      <div className="w-full max-w-md relative z-10">
        <div className="flex flex-col items-center mb-12 group">
          <div className="relative mb-6">
             <div className="w-16 h-16 bg-white rounded-none rotate-3 transition-transform group-hover:rotate-0
group-hover:scale-110 shadow-[0_0_30px_rgba(255,255,255,0.1)]"></div>
            <div className="absolute inset-0 flex items-center justify-center font-black text-black text-2xl">
              A
            </div>
          </div>
          <h1 className="text-4xl md:text-5xl font-black tracking-tighter uppercase text-center">
            INITIALIZE <span className="text-white/40">ACCESS</span>
          </h1>
          <p className="text-white/40 text-[10px] font-mono uppercase tracking-[0.4em] mt-3 text-center">
            Deploy your personal intelligence layer
          </p>
        </div>

        <div className="bg-black border border-white/10 p-10 rounded-none shadow-2xl relative">
          <div className="absolute -top-3 left-10 px-4 py-1 bg-white text-black text-[8px] font-mono font-black
uppercase tracking-widest">
            New Protocol Initialization
          </div>

          <div className="mb-8">
            <button
              onClick={handlePasskeyAuth}
              className="w-full py-4 bg-white text-black font-mono font-black uppercase tracking-widest text-xs
hover:bg-white/90 transition-all border border-white active:scale-[0.98] cursor-pointer"
            >
              AUTHENTICATE VIA BIOMETRICS (PASSKEY)
            </button>
            {authError && (
                  <div className="mt-4 font-mono text-[9px] text-white/60 uppercase tracking-tighter border-l
border-white/20 pl-3 py-1">
                <span className="text-white/40 mr-2">{">"}</span> {authError}
              </div>
            )}
          </div>

          <div className="relative mb-8 flex items-center">
```

```
            <div className="flex-grow h-[1px] bg-white/10"></div>
                      <span  className="px-4  font-mono  text-[8px]  text-white/20  uppercase  tracking-widest
whitespace-nowrap">
              OR MANUAL DECRYPTION
            </span>
            <div className="flex-grow h-[1px] bg-white/10"></div>
          </div>

          <Auth
            supabaseClient={supabase}
            view="sign_up"
            appearance={{
              theme: ThemeSupa,
              variables: {
                default: {
                  colors: {
                    brand: "white",
                    brandAccent: "white",
                    brandButtonText: "black",
                    defaultButtonBackground: "transparent",
                    defaultButtonBackgroundHover: "#111111",
                    defaultButtonText: "white",
                    defaultButtonBorder: "#333333",
                    inputText: "white",
                    inputBackground: "transparent",
                    inputBorder: "#333333",
                    inputPlaceholder: "#555555",
                    inputLabelText: "#888888",
                  },
                  radii: {
                    buttonBorderRadius: "0px",
                    inputBorderRadius: "0px",
                  },
                  fonts: {
                    bodyFontFamily: `var(--font-geist-sans), ui-sans-serif, system-ui, sans-serif`,
                         buttonFontFamily: `var(--font-geist-mono), ui-monospace, SFMono-Regular, Roboto Mono,
Menlo, Monaco, Consolas, Liberation Mono, Courier New, monospace`,
                    inputFontFamily: `var(--font-geist-sans), ui-sans-serif, system-ui, sans-serif`,
                    labelFontFamily: `var(--font-geist-mono), ui-monospace, SFMono-Regular, Roboto Mono, Menlo,
Monaco, Consolas, Liberation Mono, Courier New, monospace`,
                  },
                },
              },
              className: {
                button: "font-mono uppercase tracking-widest text-xs",
                label: "font-mono uppercase tracking-tighter text-[10px]",
                input: "border-white/20 focus:border-white transition-colors",
              },
            }}
            localization={{
              variables: {
                sign_up: {
                  email_label: "Professional Identifier",
                  password_label: "Encryption Key",
                  button_label: "Initialize Ledger Access",
                  link_text: "Authorized? Access Terminal",
                },
                sign_in: {
                  email_label: "Professional Identifier",
                  password_label: "Encryption Key",
                  button_label: "Authorize Session",
                  link_text: "No Access? Initialize Ledger",
                },
                forgotten_password: {
```

```
                  link_text: "Recover Encryption Key?",
                },
              },
            }}
            providers={["google"]}
            redirectTo={`${typeof window !== "undefined" ? window.location.origin : ""}/auth/callback`}
          />

          <div className="mt-8 text-center">
            <Link
              href="/"
                    className="text-[10px] font-mono text-white/40 uppercase tracking-widest hover:text-white
transition-colors"
            >
              Return to Authorization
            </Link>
          </div>
        </div>

        <div className="mt-12 flex flex-col items-center gap-4 text-center">
          <div className="h-[1px] w-12 bg-white/10"></div>
          <p className="text-white/20 text-[9px] font-mono uppercase tracking-widest leading-relaxed max-w-xs">
            Initialization grants read-write access to the Axiom deterministic
            ledger. Secure your access key immediately after authorization.
          </p>
        </div>
      </div>
    </main>
  );
}
```

File: src/components/dashboard/DayZeroOnboarding.tsx

```
"use client";

import { useState } from "react";
import {
  submitDayZeroBaselineAction,
  type DashboardSnapshot,
} from "@/app/dashboard/actions";
import { formatCurrency } from "@/lib/utils/formatters";

interface DayZeroOnboardingProps {
  onComplete: (snapshot: DashboardSnapshot) => void;
}

const ACCOUNT_TYPES = [
  { value: "mobile_money", label: "Mobile Money (M-Pesa)" },
  { value: "checking", label: "Checking / Current Account" },
  { value: "savings", label: "Savings Account" },
  { value: "cash", label: "Cash on Hand" },
  { value: "crypto", label: "Digital Assets / Crypto" },
];

const INCOME_TYPES = [
  { value: "salary", label: "Salary" },
  { value: "freelance", label: "Freelance / Gig" },
  { value: "business", label: "Business Profits" },
  { value: "other", label: "Other Inflow" },
];

const CADENCES = [
  { value: "weekly", label: "Weekly" },
```

```
  { value: "biweekly", label: "Bi-weekly" },
  { value: "monthly", label: "Monthly" },
  { value: "annually", label: "Annually" },
];

const BASELINE_CADENCES = [
  { value: "daily", label: "Daily" },
  { value: "weekly", label: "Weekly" },
  { value: "monthly", label: "Monthly" },
  { value: "yearly", label: "Yearly" },
];

export default function DayZeroOnboarding({
  onComplete,
}: DayZeroOnboardingProps) {
  const [step, setStep] = useState(1);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const [accounts, setAccounts] = useState([
    {
      account_name: "",
      account_type: "mobile_money",
      current_balance: "",
    },
  ]);
  const [incomes, setIncomes] = useState<
    {
      income_name: string;
      income_type: string;
      amount: string;
      cadence: string;
    }[]
  >([]);
  const [liabilities, setLiabilities] = useState<
    {
      liability_name: string;
      liability_type: string;
      outstanding_balance: string;
    }[]
  >([]);
  const [baseline, setBaseline] = useState({ amount: "", cadence: "monthly" });

  const addAccount = () =>
    setAccounts([
      ...accounts,
      {
        account_name: "New Account",
        account_type: "checking",
        current_balance: "",
      },
    ]);
  const removeAccount = (index: number) =>
    setAccounts(accounts.filter((_, i) => i !== index));

  const addIncome = () =>
    setIncomes([
      ...incomes,
      {
        income_name: "",
        income_type: "salary",
        amount: "",
        cadence: "monthly",
      },
```

```
  ]);
const removeIncome = (index: number) =>
  setIncomes(incomes.filter((_, i) => i !== index));

const addLiability = () =>
  setLiabilities([
    ...liabilities,
    {
      liability_name: "",
      liability_type: "personal_loan",
      outstanding_balance: "",
    },
  ]);
const removeLiability = (index: number) =>
  setLiabilities(liabilities.filter((_, i) => i !== index));

const isStepValid = () => {
  if (step === 1)
    return (
      accounts.length > 0 &&
      accounts.every((a) => a.account_name && a.current_balance !== "")
    );
  if (step === 2)
    return (
      incomes.length === 0 ||
      incomes.every((i) => i.income_name && i.amount !== "")
    );
  if (step === 3) return baseline.amount !== "";
  if (step === 4)
    return (
      liabilities.length === 0 ||
      liabilities.every(
        (l) => l.liability_name && l.outstanding_balance !== "",
      )
    );
  return false;
};

async function handleSubmit(e: React.FormEvent) {
  e.preventDefault();
  if (step < 4) {
    setStep(step + 1);
    return;
  }

  setIsLoading(true);
  setError(null);
  try {
    await submitDayZeroBaselineAction({
      accounts: accounts.map((a) => ({
        ...a,
        current_balance: Number(a.current_balance),
      })),
      incomes: incomes.map((i) => ({ ...i, amount: Number(i.amount) })),
      liabilities: liabilities
        .filter((l) => Number(l.outstanding_balance) > 0)
        .map((l) => ({
          ...l,
          outstanding_balance: Number(l.outstanding_balance),
        })),
      baseline: {
        amount: Number(baseline.amount),
        cadence: baseline.cadence,
      },
```

```
      });
       // We don't need to call onComplete here because the page will re-render or we can handle it via the
action's return
      // But for consistency with the previous impl:
      window.location.reload();
    } catch (err: unknown) {
      setError(
        err instanceof Error
          ? err.message
          : "Configuration failed. System offline.",
      );
    } finally {
      setIsLoading(false);
    }
  }

  return (
       <div  className="fixed  inset-0  bg-background  z-[100]  flex  items-center  justify-center  p-6  md:p-12
overflow-y-auto">
      <div className="max-w-2xl w-full space-y-12 py-12">
        {/* Branding & Header */}
        <div className="space-y-4">
          <div className="flex items-center gap-3">
            <div className="w-8 h-8 bg-foreground rounded-xl flex items-center justify-center">
              <span className="font-black text-background text-[10px]">A</span>
            </div>
            <span className="text-xs font-black uppercase tracking-[0.3em] opacity-40">
              Axiom Terminal :: Day Zero v2
            </span>
          </div>
          <h1 className="text-4xl md:text-5xl font-black tracking-tighter uppercase leading-none">
            Financial <br /> Initialization
          </h1>
                    <p  className="text-foreground/40  text-xs  font-bold  uppercase  tracking-widest  max-w-md
leading-relaxed">
            The Axiom engine is scaling your telemetry. Provide a comprehensive
            view of your capital structure.
          </p>
        </div>

        {/* Progress Indicator */}
        <div className="flex gap-2 h-1">
          {[1, 2, 3, 4].map((s) => (
            <div
              key={s}
              className={`flex-1 transition-all duration-500 rounded-full ${
                s <= step ? "bg-foreground" : "bg-foreground/10"
              }`}
            ></div>
          ))}
        </div>

        {/* Form Container */}
        <form onSubmit={handleSubmit} className="space-y-10">
          <div className="min-h-[240px] animate-in fade-in slide-in-from-bottom-4 duration-700">
            {step === 1 && (
              <div className="space-y-8">
                <div className="space-y-1">
                  <label className="text-[10px] font-black text-foreground/40 uppercase tracking-[0.2em]">
                    01. Capital Repositories
                  </label>
                  <h2 className="text-xl font-black uppercase tracking-tight">
                    Where is your capital held?
                  </h2>
```

```
                </div>

                <div className="space-y-4">
                  {accounts.map((acc, idx) => (
                    <div
                      key={idx}
                           className="flex flex-col md:flex-row gap-4 p-4 bg-foreground/5 rounded-2xl relative
group"
                    >
                      <div className="flex-1 space-y-3">
                        <input
                          type="text"
                          placeholder="Account Name (e.g. M-Pesa, KCB)"
                          value={acc.account_name}
                          onChange={(e) => {
                            const newAccs = [...accounts];
                            newAccs[idx].account_name = e.target.value;
                            setAccounts(newAccs);
                          }}
                          className="w-full bg-transparent border-b border-foreground/10 py-2 font-bold text-sm
focus:outline-none focus:border-foreground"
                        />
                        <div className="flex gap-4">
                          <select
                            value={acc.account_type}
                            onChange={(e) => {
                              const newAccs = [...accounts];
                              newAccs[idx].account_type = e.target.value;
                              setAccounts(newAccs);
                            }}
                            className="bg-transparent border-b border-foreground/10 py-2 text-[10px] font-black
uppercase tracking-widest focus:outline-none"
                          >
                            {ACCOUNT_TYPES.map((t) => (
                              <option key={t.value} value={t.value}>
                                {t.label}
                              </option>
                            ))}
                          </select>
                          <input
                            type="number"
                            placeholder="Balance"
                            value={acc.current_balance}
                            onChange={(e) => {
                              const newAccs = [...accounts];
                              newAccs[idx].current_balance = e.target.value;
                              setAccounts(newAccs);
                            }}
                                 className="flex-1 bg-transparent border-b border-foreground/10 py-2 font-black
tabular-nums focus:outline-none focus:border-foreground"
                          />
                        </div>
                      </div>
                      {accounts.length > 1 && (
                        <button
                          type="button"
                          onClick={() => removeAccount(idx)}
                          className="p-2 text-foreground/20 hover:text-red-500 transition-colors self-center"
                        >
                          <svg
                            width="16"
                            height="16"
                            viewBox="0 0 24 24"
                            fill="none"
```

```
                          stroke="currentColor"
                          strokeWidth="3"
                        >
                           <path d="M3 6h18M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0 1 2-2h4a2 2
0 0 1 2 2v2" />
                        </svg>
                      </button>
                    )}
                  </div>
                ))}
              </div>
              <button
                type="button"
                onClick={addAccount}
                  className="w-full py-4 border-2 border-dashed border-foreground/10 rounded-2xl text-[10px]
font-black uppercase tracking-widest text-foreground/40 hover:border-foreground/20 hover:text-foreground
transition-all"
              >
                + Add Another Repository
              </button>
            </div>
          )}

          {step === 2 && (
            <div className="space-y-8">
              <div className="space-y-1">
                <label className="text-[10px] font-black text-foreground/40 uppercase tracking-[0.2em]">
                  02. Inflow Velocity
                </label>
                <h2 className="text-xl font-black uppercase tracking-tight">
                  How does capital arrive?
                </h2>
              </div>

              <div className="space-y-4">
                {incomes.length === 0 ? (
                  <div className="p-8 border-2 border-dashed border-foreground/10 rounded-3xl text-center">
                    <p className="text-[10px] font-black text-foreground/30 uppercase tracking-widest">
                      No regular inflow streams reported.
                    </p>
                  </div>
                ) : (
                  incomes.map((inc, idx) => (
                    <div
                      key={idx}
                      className="flex flex-col md:flex-row gap-4 p-4 bg-foreground/5 rounded-2xl"
                    >
                      <div className="flex-1 space-y-3">
                        <div className="relative group">
                          <input
                            type="text"
                            placeholder="Source (e.g. Salary, Shop Profits)"
                            value={inc.income_name}
                            onChange={(e) => {
                              const newIncs = [...incomes];
                              newIncs[idx].income_name = e.target.value;
                              setIncomes(newIncs);
                            }}
                               className="w-full bg-transparent border-b border-foreground/10 py-2 font-bold
text-sm focus:outline-none focus:border-foreground pr-8"
                          />
                                    <div className="absolute right-0 top-1/2 -translate-y-1/2 opacity-20
group-hover:opacity-100 transition-opacity">
                            <svg
```

```
                            width="12"
                            height="12"
                            viewBox="0 0 24 24"
                            fill="none"
                            stroke="currentColor"
                            strokeWidth="3"
                          >
                            <path d="M11 4H4a2 2 0 0 0-2 2v14a2 2 0 0 0 2 2h14a2 2 0 0 0 2-2v-7" />
                            <path d="M18.5 2.5a2.121 2.121 0 0 1 3 3L12 15l-4 1 1-4 9.5-9.5z" />
                          </svg>
                        </div>
                      </div>
                      <div className="flex flex-wrap gap-4">
                        <select
                          value={inc.income_type}
                          onChange={(e) => {
                            const newIncs = [...incomes];
                            newIncs[idx].income_type = e.target.value;
                            setIncomes(newIncs);
                          }}
                                  className="bg-transparent border-b border-foreground/10 py-2 text-[10px]
font-black uppercase tracking-widest focus:outline-none"
                        >
                          {INCOME_TYPES.map((t) => (
                            <option key={t.value} value={t.value}>
                              {t.label}
                            </option>
                          ))}
                        </select>
                        <select
                          value={inc.cadence}
                          onChange={(e) => {
                            const newIncs = [...incomes];
                            newIncs[idx].cadence = e.target.value;
                            setIncomes(newIncs);
                          }}
                                  className="bg-transparent border-b border-foreground/10 py-2 text-[10px]
font-black uppercase tracking-widest focus:outline-none"
                        >
                          {CADENCES.map((c) => (
                            <option key={c.value} value={c.value}>
                              {c.label}
                            </option>
                          ))}
                        </select>
                        <input
                          type="number"
                          placeholder="Amount"
                          value={inc.amount}
                          onChange={(e) => {
                            const newIncs = [...incomes];
                            newIncs[idx].amount = e.target.value;
                            setIncomes(newIncs);
                          }}
                          className="flex-1 min-w-[100px] bg-transparent border-b border-foreground/10 py-2
font-black tabular-nums focus:outline-none focus:border-foreground"
                        />
                      </div>
                    </div>
                    <button
                      type="button"
                      onClick={() => removeIncome(idx)}
                      className="p-2 text-foreground/20 hover:text-red-500 transition-colors self-center"
                    >
```

```
                        <svg
                          width="16"
                          height="16"
                          viewBox="0 0 24 24"
                          fill="none"
                          stroke="currentColor"
                          strokeWidth="3"
                        >
                           <path d="M3 6h18M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0 1 2-2h4a2 2 0 0 1 2 2v2" />
                        </svg>
                      </button>
                    </div>
                  ))
                )}
              </div>
              <button
                type="button"
                onClick={addIncome}
                  className="w-full py-4 border-2 border-dashed border-foreground/10 rounded-2xl text-[10px] font-black uppercase tracking-widest text-foreground/40 hover:border-foreground/20 hover:text-foreground transition-all"
              >
                + Add Inflow Source
              </button>
            </div>
          )}

          {step === 3 && (
            <div className="space-y-8">
              <div className="space-y-1">
                <label className="text-[10px] font-black text-foreground/40 uppercase tracking-[0.2em]">
                  03. Survival Baseline
                </label>
                <h2 className="text-xl font-black uppercase tracking-tight">
                  What is the cost of existence?
                </h2>
              </div>
              <div className="space-y-6 bg-foreground/5 p-8 rounded-3xl">
                <div className="relative">
                            <span  className="absolute  left-0  top-1/2  -translate-y-1/2  text-3xl  font-black text-foreground/20">
                    KSh
                  </span>
                  <input
                    autoFocus
                    type="number"
                    required
                    placeholder="0.00"
                    value={baseline.amount}
                    onChange={(e) =>
                      setBaseline({ ...baseline, amount: e.target.value })
                    }
                        className="w-full bg-transparent border-b-4 border-foreground/10 py-4 pl-16 text-5xl font-black        tabular-nums        focus:outline-none        focus:border-foreground        transition-colors placeholder:text-foreground/5"
                    />
                </div>
                <div className="flex flex-wrap items-center gap-6">
                  <span className="text-[10px] font-black uppercase tracking-widest text-foreground/40">
                    Measured on a
                  </span>
                  <div className="flex flex-wrap gap-2">
                    {BASELINE_CADENCES.map((c) => (
```

```
                        <button
                          key={c.value}
                          type="button"
                          onClick={() =>
                            setBaseline({ ...baseline, cadence: c.value })
                          }
                             className={`px-4 py-2 rounded-xl text-[10px] font-black uppercase tracking-widest
transition-all ${
                            baseline.cadence === c.value
                              ? "bg-foreground text-background"
                              : "bg-foreground/5 text-foreground/40 hover:bg-foreground/10"
                          }`}
                        >
                          {c.label}
                        </button>
                      ))}
                    </div>
                    <span className="text-[10px] font-black uppercase tracking-widest text-foreground/40">
                      basis.
                    </span>
                  </div>
                </div>
                          <p  className="text-[10px]  font-bold  text-foreground/30  uppercase  tracking-widest
leading-relaxed">
                  Enter your core survival cost (Rent, Food, Utilities). This
                  allows Axiom to calculate your true survival window.
                </p>
              </div>
            )}

            {step === 4 && (
              <div className="space-y-8">
                <div className="space-y-1">
                  <label className="text-[10px] font-black text-foreground/40 uppercase tracking-[0.2em]">
                    04. Immediate Obligations
                  </label>
                  <h2 className="text-xl font-black uppercase tracking-tight">
                    What does the system owe?
                  </h2>
                </div>

                <div className="space-y-4">
                  {liabilities.length === 0 ? (
                    <div className="p-8 border-2 border-dashed border-foreground/10 rounded-3xl text-center">
                      <p className="text-[10px] font-black text-foreground/30 uppercase tracking-widest">
                        No outstanding obligations reported.
                      </p>
                    </div>
                  ) : (
                    liabilities.map((liab, idx) => (
                      <div
                        key={idx}
                        className="flex flex-col md:flex-row gap-4 p-4 bg-foreground/5 rounded-2xl"
                      >
                        <div className="flex-1 space-y-3">
                          <div className="relative group">
                            <input
                              type="text"
                              placeholder="Obligation (e.g. Fuliza, Loan)"
                              value={liab.liability_name}
                              onChange={(e) => {
                                const newLiabs = [...liabilities];
                                newLiabs[idx].liability_name = e.target.value;
                                setLiabilities(newLiabs);
```

```
                            }}
                               className="w-full bg-transparent border-b border-foreground/10 py-2 font-bold
text-sm focus:outline-none focus:border-foreground pr-8"
                            />
                                        <div className="absolute right-0 top-1/2 -translate-y-1/2 opacity-20
group-hover:opacity-100 transition-opacity">
                              <svg
                                width="12"
                                height="12"
                                viewBox="0 0 24 24"
                                fill="none"
                                stroke="currentColor"
                                strokeWidth="3"
                              >
                                <path d="M11 4H4a2 2 0 0 0-2 2v14a2 2 0 0 0 2 2h14a2 2 0 0 0 2-2v-7" />
                                <path d="M18.5 2.5a2.121 2.121 0 0 1 3 3L12 15l-4 1 1-4 9.5-9.5z" />
                              </svg>
                            </div>
                          </div>
                          <input
                            type="number"
                            placeholder="Outstanding Balance"
                            value={liab.outstanding_balance}
                            onChange={(e) => {
                              const newLiabs = [...liabilities];
                              newLiabs[idx].outstanding_balance =
                                e.target.value;
                              setLiabilities(newLiabs);
                            }}
                               className="w-full bg-transparent border-b border-foreground/10 py-2 font-black
tabular-nums focus:outline-none focus:border-foreground"
                            />
                        </div>
                        <button
                          type="button"
                          onClick={() => removeLiability(idx)}
                          className="p-2 text-foreground/20 hover:text-red-500 transition-colors self-center"
                        >
                          <svg
                            width="16"
                            height="16"
                            viewBox="0 0 24 24"
                            fill="none"
                            stroke="currentColor"
                            strokeWidth="3"
                          >
                             <path d="M3 6h18M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0 1 2-2h4a2 2
0 0 1 2 2v2" />
                          </svg>
                        </button>
                      </div>
                    ))
                  )}
                </div>
                <button
                  type="button"
                  onClick={addLiability}
                    className="w-full py-4 border-2 border-dashed border-foreground/10 rounded-2xl text-[10px]
font-black uppercase tracking-widest text-foreground/40 hover:border-foreground/20 hover:text-foreground
transition-all"
                >
                  + Add Obligation
                </button>
              </div>
```

```
            )}
          </div>

          {error && (
               <div className="p-4 bg-red-500/10 border border-red-500/20 rounded-xl flex items-center gap-3
animate-shake">
              <svg
                width="16"
                height="16"
                viewBox="0 0 24 24"
                fill="none"
                stroke="currentColor"
                strokeWidth="3"
                className="text-red-500"
              >
                <circle cx="12" cy="12" r="10" />
                <path d="M12 8v4M12 16h.01" />
              </svg>
              <p className="text-[10px] font-black text-red-600 uppercase tracking-widest">
                {error}
              </p>
            </div>
          )}

          <div className="flex items-center justify-between pt-6">
            {step > 1 ? (
              <button
                type="button"
                onClick={() => setStep(step - 1)}
                            className="text-[10px] font-black uppercase tracking-[0.3em] text-foreground/30
hover:text-foreground transition-colors"
              >
                Go Back
              </button>
            ) : (
              <div></div>
            )}

            <button
              type="submit"
              disabled={isLoading || !isStepValid()}
                      className="px-10 py-5 bg-foreground text-background rounded-2xl font-black uppercase
tracking-[0.2em] hover:scale-105 active:scale-95 transition-all shadow-2xl shadow-foreground/20
disabled:opacity-20 disabled:scale-100"
            >
              {isLoading
                ? "CALCULATING..."
                : step === 4
                  ? "Initialize System"
                  : "Next Phase"}
            </button>
          </div>
        </form>

        {/* System Status Footer */}
              <div  className="pt-12  border-t  border-foreground/5  flex  flex-col  md:flex-row  md:items-center
justify-between gap-4">
          <div className="flex items-center gap-2">
            <div className="w-1.5 h-1.5 bg-green-500 rounded-full animate-pulse"></div>
            <span className="text-[9px] font-black text-foreground/30 uppercase tracking-widest">
              Analytical Precision Active
            </span>
          </div>
                 <p  className="text-[8px]  font-bold  text-foreground/20  uppercase  tracking-tighter  max-w-xs
```

```
md:text-right">
            Verification required for system activation. Provided telemetry
            remains encrypted.
          </p>
        </div>
      </div>
    </div>
  );
}
```

File: src/components/dashboard/TelemetryDashboard.tsx

```
"use client";

import { useEffect, useState, useCallback } from "react";
import {
  fetchTelemetryLogsAction,
  type AuditLog,
  type TelemetrySummary,
} from "@/app/dashboard/actions";

/**
 * TELEMETRY DASHBOARD (Phase 2)
 * High-density forensic UI for the Kairos Insight Engine.
 * Aesthetic: Quiet System (monochromatic, high-precision).
 */
export function TelemetryDashboard() {
  const [telemetry, setTelemetry] = useState<TelemetrySummary | null>(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  const loadTelemetry = useCallback(async (isManualRefresh = false) => {
    if (isManualRefresh) setIsLoading(true);
    setError(null);
    try {
      const summary = await fetchTelemetryLogsAction();
      setTelemetry(summary);
    } catch (err) {
      setError("Failed to retrieve forensic logs.");
      console.error(err);
    } finally {
      setIsLoading(false);
    }
  }, []);

  useEffect(() => {
    // eslint-disable-next-line react-hooks/set-state-in-effect
    loadTelemetry();
  }, [loadTelemetry]);

  if (isLoading) {
    return (
      <div className="p-8 space-y-6 animate-pulse">
        <div className="h-4 w-32 bg-foreground/5 rounded"></div>
        <div className="grid grid-cols-3 gap-6">
          <div className="h-24 bg-foreground/5 rounded-xl"></div>
          <div className="h-24 bg-foreground/5 rounded-xl"></div>
          <div className="h-24 bg-foreground/5 rounded-xl"></div>
        </div>
        <div className="h-64 bg-foreground/5 rounded-xl"></div>
      </div>
    );
  }
```

```
if (error) {
  return (
    <div className="p-8 text-center border-2 border-dashed border-red-500/10 rounded-3xl">
      <p className="text-[10px] font-black text-red-500 uppercase tracking-widest">
        {error}
      </p>
    </div>
  );
}

return (
  <div className="space-y-12">
    {/* PERFORMANCE SUMMARY */}
    <div className="grid grid-cols-1 md:grid-cols-4 gap-6">
      <div className="p-6 border border-foreground/10 rounded-2xl bg-foreground/[0.02]">
        <span className="text-[9px] font-black text-foreground/40 uppercase tracking-[0.2em] block mb-2">
          Total Cycles
        </span>
        <span className="text-2xl font-black tabular-nums text-foreground">
          {telemetry?.totalCycles || 0}
        </span>
      </div>
      <div className="p-6 border border-foreground/10 rounded-2xl bg-foreground/[0.02]">
        <span className="text-[9px] font-black text-foreground/40 uppercase tracking-[0.2em] block mb-2">
          Avg. Latency
        </span>
        <span className="text-2xl font-black tabular-nums text-foreground">
          {telemetry?.averageLatency || 0}
          <span className="text-xs ml-1 text-foreground/40 font-bold uppercase">
            ms
          </span>
        </span>
      </div>
      <div className="p-6 border border-foreground/10 rounded-2xl bg-foreground/[0.02]">
        <span className="text-[9px] font-black text-foreground/40 uppercase tracking-[0.2em] block mb-2">
          Match Rate
        </span>
        <span className="text-2xl font-black tabular-nums text-foreground">
          {telemetry?.matchRate || 0}%
        </span>
      </div>
      <div className="p-6 border border-foreground/10 rounded-2xl bg-foreground/[0.02]">
        <span className="text-[9px] font-black text-foreground/40 uppercase tracking-[0.2em] block mb-2">
          Active Rules
        </span>
        <span className="text-2xl font-black tabular-nums text-foreground">
          {Object.keys(telemetry?.ruleHits || {}).length}
        </span>
      </div>
    </div>

    {/* RULE FREQUENCY */}
    {telemetry && Object.keys(telemetry.ruleHits).length > 0 && (
      <div className="space-y-4">
        <h3 className="text-[10px] font-black text-foreground/30 uppercase tracking-[0.3em] ml-1">
          Rule Trigger Frequency
        </h3>
        <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4">
          {Object.entries(telemetry.ruleHits)
            .sort(([, a], [, b]) => b - a)
            .map(([ruleId, count]) => (
              <div
                key={ruleId}
```

```
                  className="p-4 border border-foreground/5 rounded-xl flex items-center justify-between"
                >
                  <span className="text-[10px] font-bold text-foreground/60 uppercase tracking-tighter truncate
max-w-[180px]">
                    {ruleId}
                  </span>
                                <span  className="text-[10px]  font-black  bg-foreground/5  px-2  py-0.5  rounded
text-foreground/80">
                    {count}
                  </span>
                </div>
              ))}
          </div>
        </div>
      )}

      {/* FORENSIC LOG TABLE */}
      <div className="space-y-4">
        <div className="flex items-center justify-between ml-1">
          <h3 className="text-[10px] font-black text-foreground/30 uppercase tracking-[0.3em]">
            Evaluation Audit Trail
          </h3>
          <button
            onClick={() => loadTelemetry(true)}
            className="text-[9px] font-black text-foreground/40 uppercase tracking-widest hover:text-foreground
transition-colors"
          >
            Refresh Log
          </button>
        </div>
        <div className="overflow-hidden border border-foreground/10 rounded-2xl">
          <table className="w-full text-left border-collapse">
            <thead>
              <tr className="bg-foreground/[0.02] border-b border-foreground/10">
                <th className="p-4 text-[9px] font-black text-foreground/40 uppercase tracking-widest">
                  Timestamp
                </th>
                <th className="p-4 text-[9px] font-black text-foreground/40 uppercase tracking-widest">
                  Rule ID
                </th>
                <th className="p-4 text-[9px] font-black text-foreground/40 uppercase tracking-widest">
                  Severity
                </th>
                      <th  className="p-4  text-[9px]  font-black  text-foreground/40  uppercase  tracking-widest
text-right">
                  Latency
                </th>
              </tr>
            </thead>
            <tbody className="divide-y divide-foreground/5">
              {telemetry?.logs.map((log) => (
                <tr
                  key={log.id}
                  className="hover:bg-foreground/[0.01] transition-colors group"
                >
                  <td className="p-4 text-[10px] font-bold text-foreground/40 tabular-nums">
                    {new Date(log.created_at).toLocaleTimeString([], {
                      hour12: false,
                      hour: "2-digit",
                      minute: "2-digit",
                      second: "2-digit",
                    })}
                  </td>
                  <td className="p-4 text-[10px] font-black text-foreground/80 uppercase tracking-tighter">
```

```
                  {log.rule_id}
                </td>
                <td className="p-4">
                  <span
                    className={`text-[9px] font-black uppercase tracking-widest px-2 py-0.5 rounded ${
                      log.severity === "critical"
                        ? "bg-red-500/10 text-red-500"
                        : log.severity === "high"
                          ? "bg-orange-500/10 text-orange-500"
                          : log.severity === "medium"
                            ? "bg-yellow-500/10 text-yellow-500"
                            : log.severity === "none"
                              ? "text-foreground/20"
                              : "bg-foreground/5 text-foreground/60"
                    }`}
                  >
                    {log.severity}
                  </span>
                </td>
                <td className="p-4 text-[10px] font-bold text-foreground/60 tabular-nums text-right">
                  {(log.evaluation_time_ms || 0).toFixed(2)}ms
                </td>
              </tr>
            ))}
            {(!telemetry || telemetry.logs.length === 0) && (
              <tr>
                <td
                  colSpan={4}
                            className="p-12 text-center text-[10px] font-black text-foreground/20 uppercase
tracking-[0.2em]"
                >
                  No evaluation data recorded in current window.
                </td>
              </tr>
            )}
          </tbody>
        </table>
      </div>
    </div>
  </div>
  );
}
```

File: src/components/landing/LandingTerminal.tsx

```
"use client";

import { useState, useEffect, useRef } from "react";

const LOGS = [
  { text: "> INITIALIZING KAIROS ENGINE...", type: "info" },
  { text: "> SYNCING OPERATIONAL BASELINE...", type: "info" },
  { text: "> ANALYZING TRANSACTION TAXONOMY...", type: "info" },
  { text: "> CALCULATING STRUCTURAL RUNWAY...", type: "info" },
  { text: "> [WARNING] INFLOW CONCENTRATION RISK DETECTED.", type: "warning" },
  { text: "> 85% OF YIELD RELIES ON A SINGLE FLOW.", type: "warning" },
  { text: "> STRUCTURAL SOLVENCY: OPTIMAL", type: "success" },
  { text: "> DEPLOYMENT STRATEGY: CALCULATED", type: "info" },
];

export function LandingTerminal() {
  const [visibleLogs, setVisibleLogs] = useState<number[]>([]);
  const [isTyping, setIsTyping] = useState(true);
```

```
  const containerRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    let timeoutId: ReturnType<typeof setTimeout>;

    const showNextLog = (index: number) => {
      if (index >= LOGS.length) {
        setIsTyping(false);
        // Reset after a delay to loop the animation
        timeoutId = setTimeout(() => {
          setVisibleLogs([]);
          setIsTyping(true);
          showNextLog(0);
        }, 5000);
        return;
      }

      timeoutId = setTimeout(
        () => {
          setVisibleLogs((prev) => [...prev, index]);
          showNextLog(index + 1);
        },
        index === 0 ? 500 : 1200,
      );
    };

    showNextLog(0);

    return () => clearTimeout(timeoutId);
  }, []);

  return (
    <div className="w-full max-w-2xl mx-auto font-mono text-sm">
              <div  className="bg-[#0a0a0a]  border  border-white/10  rounded-lg  overflow-hidden  shadow-2xl
shadow-white/5">
        {/* Terminal Header */}
        <div className="bg-white/5 px-4 py-2 border-b border-white/10 flex items-center justify-between">
          <div className="flex gap-1.5">
            <div className="w-2.5 h-2.5 rounded-full bg-white/10" />
            <div className="w-2.5 h-2.5 rounded-full bg-white/10" />
            <div className="w-2.5 h-2.5 rounded-full bg-white/10" />
          </div>
          <div className="text-[10px] uppercase tracking-widest text-white/30">
            Kairos // Audit Terminal
          </div>
        </div>

        {/* Terminal Body */}
        <div
          ref={containerRef}
          className="p-6 h-64 overflow-y-auto space-y-2 scrollbar-hide"
        >
          {visibleLogs.map((logIndex) => {
            const log = LOGS[logIndex];
            return (
              <div
                key={logIndex}
                className={`
                  ${log.type === "warning" ? "text-orange-500" : ""}
                  ${log.type === "success" ? "text-emerald-500" : ""}
                  ${log.type === "info" ? "text-white/70" : ""}
                  animate-in fade-in slide-in-from-left-2 duration-300
                `}
              >
```

```
                {log.text}
              </div>
            );
          })}
          {isTyping && (
            <div className="w-2 h-4 bg-white/40 animate-pulse inline-block align-middle ml-1" />
          )}
        </div>
      </div>

      {/* Decorative footer for the terminal */}
          <div  className="mt-4  flex  justify-between  items-center  px-4  text-[10px]  text-white/20  uppercase
tracking-[0.2em]">
        <div>Status: Running Clinical Audit</div>
        <div>Memory: 0.124ms / 256MB</div>
      </div>
    </div>
  );
}
```

File: src/components/landing/RunwaySimulator.tsx

```
"use client";

import { useState, useMemo } from "react";

export function RunwaySimulator() {
  const [liquidity, setLiquidity] = useState(50000);
  const [monthlyBurn, setMonthlyBurn] = useState(30000);

  const runwayResult = useMemo(() => {
    if (monthlyBurn <= 0) return { months: 99, days: 0, isInfinite: true };

    const totalMonths = liquidity / monthlyBurn;
    const months = Math.floor(totalMonths);
    const days = Math.round((totalMonths - months) * 30);

    return { months, days, isInfinite: false };
  }, [liquidity, monthlyBurn]);

  const getVerdict = () => {
    const totalMonths = runwayResult.isInfinite
      ? 99
      : runwayResult.months + runwayResult.days / 30;
    if (totalMonths < 3) {
      return {
        label: "CRITICAL",
        text: "Severe solvency risk. Capital depletion imminent.",
        color: "text-orange-500",
      };
    }
    if (totalMonths >= 3 && totalMonths < 6) {
      return {
        label: "ADVISORY",
        text: "Fragile runway. One systemic shock away from illiquidity.",
        color: "text-white/60",
      };
    }
    return {
      label: "OBSERVATION",
      text: "Runway stable. Capital ready for strategic deployment.",
      color: "text-emerald-500/80",
    };
```

```
  };

  const verdict = getVerdict();

  const formatCurrency = (val: number) => {
    return new Intl.NumberFormat("en-KE", {
      style: "currency",
      currency: "KES",
      maximumFractionDigits: 0,
    }).format(val);
  };

  return (
      <div  className="w-full  max-w-2xl  mx-auto  space-y-12  p-8  md:p-12  border  border-white/10  bg-[#050505]
rounded-2xl relative overflow-hidden group">
      {/* Background decoration */}
      <div className="absolute top-0 right-0 w-32 h-32 bg-white/[0.02] blur-3xl rounded-full" />

      <div className="space-y-8 relative z-10">
        <div className="flex justify-between items-end">
          <div className="space-y-1">
            <div className="text-[10px] font-mono uppercase tracking-[0.3em] text-white/40">
              Operational Variable 01
            </div>
            <div className="text-sm font-black uppercase tracking-widest">
              Available Liquidity
            </div>
          </div>
          <div className="font-mono text-2xl font-bold">
            {formatCurrency(liquidity)}
          </div>
        </div>
        <input
          type="range"
          min="0"
          max="1000000"
          step="5000"
          value={liquidity}
          onChange={(e) => setLiquidity(Number(e.target.value))}
          className="w-full h-[1px] bg-white/20 appearance-none cursor-crosshair accent-white"
        />
      </div>

      <div className="space-y-8 relative z-10">
        <div className="flex justify-between items-end">
          <div className="space-y-1">
            <div className="text-[10px] font-mono uppercase tracking-[0.3em] text-white/40">
              Operational Variable 02
            </div>
            <div className="text-sm font-black uppercase tracking-widest">
              Monthly Burn Rate
            </div>
          </div>
          <div className="font-mono text-2xl font-bold">
            {formatCurrency(monthlyBurn)}
          </div>
        </div>
        <input
          type="range"
          min="1000"
          max="500000"
          step="1000"
          value={monthlyBurn}
          onChange={(e) => setMonthlyBurn(Number(e.target.value))}
```

```
          className="w-full h-[1px] bg-white/20 appearance-none cursor-crosshair accent-white"
        />
      </div>

      <div className="pt-8 border-t border-white/5 space-y-6">
        <div className="flex justify-between items-baseline">
          <div className="text-[10px] font-mono uppercase tracking-[0.3em] text-white/40">
            Calculated Outcome
          </div>
          <div className="text-4xl font-black font-mono tracking-tighter flex items-baseline gap-2">
            {runwayResult.isInfinite ? (
              "?"
            ) : (
              <>
                <span>{runwayResult.months}</span>
                <span className="text-xs uppercase tracking-widest text-white/40">
                  Months
                </span>
                {runwayResult.days > 0 && (
                  <>
                    <span className="ml-2">{runwayResult.days}</span>
                    <span className="text-xs uppercase tracking-widest text-white/40">
                      Days
                    </span>
                  </>
                )}
              </>
            )}
          </div>
        </div>

        <div
            className={`p-6 border border-white/5 bg-white/[0.02] font-mono text-xs leading-relaxed space-y-2
animate-in fade-in slide-in-from-top-1 duration-500`}
        >
          <div className="flex items-center gap-2">
            <div
                        className={`w-1.5 h-1.5 rounded-full animate-pulse ${verdict.label === "CRITICAL" ?
"bg-orange-500" : verdict.label === "ADVISORY" ? "bg-white/40" : "bg-emerald-500"}`}
            />
            <span className={`font-black tracking-[0.2em] ${verdict.color}`}>
              {verdict.label}
            </span>
          </div>
          <div className="text-white/60">{verdict.text}</div>
        </div>
      </div>

      <div className="absolute bottom-4 right-8 text-[8px] font-mono uppercase tracking-[0.4em] text-white/10">
        Deterministic Solver v0.4
      </div>
    </div>
  );
}
```

File: src/lib/ai/kairos.test.ts

```
import assert from "node:assert/strict";
import test from "node:test";
import { generateSystemInsights } from "./kairos";
import { AnalyticsSummary } from "../analytics/types";
import { buildBehavioralContext } from "../context/buildBehavioralContext";
import { evaluateInsights } from "../insights/generateInsights";
```

```
const MOCK_METADATA_QUALITY = {
  averageScore: 1,
  lowQualityCount: 0,
  lowQualityRatio: 0,
  confidenceMultiplier: 1,
};

const MOCK_ANALYTICS: AnalyticsSummary = {
  totalDeployed: 1000,
  averageDeployment: 100,
  dailyBurnRate: 10,
  runwayDays: 100,
  categoryBreakdown: { Leakage: 500, Asset: 500 },
  deploymentCount: 10,
  metadataQuality: MOCK_METADATA_QUALITY,
  totalLiabilities: 0,
  totalAssets: 2000,
  netWorth: 2000,
  liquidity: 1000,
  totalMonthlyIncome: 300,
  recurringIncome: 300,
  irregularIncome: 0,
  incomeConcentration: {},
  maxIncomeConcentrationRatio: 0,
  adjustedDailyBurn: 0,
  totalStrategicTargets: 1000,
  totalCurrentProgress: 500,
  averageGoalProgress: 50,
  criticalGoalCount: 0,
  fundingGap: 500,
  // Strategic Objectives v1
  strategicAlignment: {
    fundingRatios: {},
    alignmentPressure: 0,
    liquiditySufficiency: {
      isSufficient: true,
      message:
        "Current liquidity structure sufficiently supports short-term obligations.",
      shortfall: 0,
    },
    objectiveStarvationSignals: [],
    strategicAllocationSignals: [],
    velocity: {},
  },
  // Operational Baseline v1
  totalStructuralMonthlyBurn: 0,
  totalSystemicMonthlyAllocation: 0,
};

test("Kairos Insights: correctly invokes rule engine and returns primary insight", async () => {
  const context = buildBehavioralContext(
    { currentAnalytics: MOCK_ANALYTICS },
    [100, 200, 300],
  );
  const { primaryInsight } = await evaluateInsights(context);

  assert.ok(primaryInsight.message);
  assert.ok(primaryInsight.severity);
  assert.ok(primaryInsight.category);
});

test("Kairos Insights: critical runway trigger", async () => {
  const analytics = {
```

```
    ...MOCK_ANALYTICS,
    runwayDays: 10,
  };
  const context = buildBehavioralContext(
    { currentAnalytics: analytics },
    [100, 200, 300],
  );
  const { primaryInsight } = await evaluateInsights(context);

  assert.equal(primaryInsight.severity, "critical");
  assert.equal(primaryInsight.category, "solvency_pressure");
  assert.match(primaryInsight.message, /runway has contracted/);
});

test("Kairos Insights: efficiency crisis trigger", async () => {
  const analytics = {
    ...MOCK_ANALYTICS,
    categoryBreakdown: { Leakage: 1000, Asset: 0 },
    runwayDays: 10, // Force low runway to plummet efficiency score < 40
  };
  const context = buildBehavioralContext(
    { currentAnalytics: analytics },
    [100, 2000, 50, 3000],
  );

  // Rule condition: capitalEfficiencyScore < 40
  // buildBehavioralContext for runway < 90 gives -40 penalty.
  // Volatility penalty is vol * 60.

  const { primaryInsight } = await evaluateInsights(context);

  // Solvency pressure (runway) usually has higher priority than efficiency crisis
  // if both are triggered, since runway is status: 'critical'.
  // However, in registry.ts, both have priority: 'high'.
  // evaluateInsights sorts by priority and then maintains order.
  // runway_critical is ID 1, efficiency_crisis is ID 2.

  assert.equal(primaryInsight.severity, "critical");
  assert.match(
    primaryInsight.message,
    /(runway has contracted|efficiency crisis)/,
  );
});

test("Kairos Insights: unclassified data advisory", async () => {
  const analytics = {
    ...MOCK_ANALYTICS,
    categoryBreakdown: { Unknown: 500, Asset: 500 },
  };
  const context = buildBehavioralContext(
    { currentAnalytics: analytics },
    [500, 500],
  );
  const { allInsights } = await evaluateInsights(context);

  const unclassifiedInsight = allInsights.find((i) =>
    i.message.includes("Unclassified"),
  );
  assert.ok(unclassifiedInsight);
  assert.equal(unclassifiedInsight.severity, "advisory");
});

test("Kairos Insights: high discipline pattern", async () => {
  // Healthy state with stable spending
```

```
  const context = buildBehavioralContext(
    { currentAnalytics: MOCK_ANALYTICS },
    [100, 100, 100],
  );
  const { primaryInsight } = await evaluateInsights(context);

  assert.equal(primaryInsight.severity, "observation");
  assert.match(primaryInsight.message, /High operational discipline/);
  assert.equal(primaryInsight.isSilent, true);
});

test("Kairos Insights: silence state default pattern", async () => {
  // State that doesn't trigger High Discipline but falls into Default Pattern
  const analytics = {
    ...MOCK_ANALYTICS,
    deploymentCount: 1, // High Discipline requires count > 1
  };
  const context = buildBehavioralContext(
    { currentAnalytics: analytics },
    [100],
  );
  const { primaryInsight } = await evaluateInsights(context);

  assert.equal(primaryInsight.severity, "observation");
  assert.match(primaryInsight.message, /No material behavioral shifts/);
  assert.equal(primaryInsight.isSilent, true);
});
```

File: src/lib/ai/kairos.ts

```
import { createClient } from "../supabase/server";
import { getDeployments } from "../db/deployments";
import { getAccounts } from "../db/accounts";
import { getLiabilities } from "../db/liabilities";
import { getIncomeStreams } from "../db/income";
import { getGoals } from "../db/goals";
import { getStrategicObjectives } from "../db/objectives";
import { getOperationalBaseline } from "../db/baseline";
import { getUserSettings } from "../db/settings";
import { generateSummary } from "../analytics/engine";
import {
  Deployment,
  Account,
  Liability,
  IncomeStream,
  FinancialGoal,
  StrategicObjective,
  OperationalBaseline,
} from "../analytics/types";
import { MetadataQualitySummary } from "../finance/metadataQuality";
import { buildBehavioralContext } from "../context/buildBehavioralContext";
import { evaluateInsights } from "../insights/generateInsights";

export type InsightSeverity =
  | "observation"
  | "advisory"
  | "warning"
  | "critical";

export interface KairosInsight {
  type: "warning" | "info" | "pattern" | "opportunity";
  severity: InsightSeverity;
  category:
```

```
    | "capital_efficiency"
    | "solvency_pressure"
    | "strategic_alignment"
    | "objective_starvation";
  message: string;
  supportingSignals: string[];
  timestamp: string;
  runway: number | null;
  capitalEfficiency: number;
  isSilent: boolean;
  metadataQuality?: MetadataQualitySummary;
  is_new_signal?: boolean;
}

export interface KairosTelemetry {
  deployments?: Deployment[];
  liquidity?: number;
  accounts?: Account[];
  liabilities?: Liability[];
  incomeStreams?: IncomeStream[];
  goals?: FinancialGoal[];
  objectives?: StrategicObjective[];
  baseline?: OperationalBaseline[];
  previousInsight?: KairosInsight | null;
}

/**
 * SERVER-SIDE ORCHESTRATOR: generateSystemInsights
 * Safely hydrates the Kairos AI layer with analytical context.
 */
export async function generateSystemInsights(
  telemetry: KairosTelemetry = {},
): Promise<KairosInsight> {
  const supabase = await createClient();
  const {
    data: { user },
  } = await supabase.auth.getUser();

  if (!user) {
    throw new Error("Authentication required for Kairos hydration.");
  }

  // 1. Resolve Canonical Context (Hydrate missing pieces only)
  const deps =
    (telemetry.deployments ?? (await getDeployments(supabase))) || [];
  const accounts = (telemetry.accounts ?? (await getAccounts(supabase))) || [];
  const liabilities =
    (telemetry.liabilities ?? (await getLiabilities(supabase))) || [];
  const income =
    (telemetry.incomeStreams ?? (await getIncomeStreams(supabase))) || [];
  const goals = (telemetry.goals ?? (await getGoals(supabase))) || [];
  const objectives =
    (telemetry.objectives ?? (await getStrategicObjectives(supabase))) || [];
  const baseline =
    (telemetry.baseline ?? (await getOperationalBaseline(supabase))) || [];

  let liquidity = telemetry.liquidity;
  if (liquidity === undefined) {
    const settings = await getUserSettings(supabase, user.id);
    liquidity = Number(
      (settings as { total_liquidity: number | string }).total_liquidity,
    );
  }
```

```
  // 2. Build Deterministic Analytics
  const analytics = generateSummary(
    deps,
    liquidity,
    accounts,
    liabilities,
    income,
    goals,
    objectives,
    baseline,
  );

  // 3. Build Behavioral Context for Insight Engine
  const context = buildBehavioralContext(
    { currentAnalytics: analytics },
    deps.map((d) => Number(d.amount)),
  );

  // 4. Evaluate Insights via Rule Registry (V1.1)
  console.log("KAIROS TELEMETRY:", {
    deploymentsCount: deps.length,
    runwayDays: analytics.runwayDays,
    alignmentPressure: analytics.strategicAlignment.alignmentPressure,
    netWorth: analytics.netWorth,
  });

  const { primaryInsight } = await evaluateInsights(context);

  const previousInsight = telemetry.previousInsight;
  const isMessageChanged = primaryInsight.message !== previousInsight?.message;
  const isSeverityChanged =
    primaryInsight.severity !== previousInsight?.severity;
  const isSilentChanged = primaryInsight.isSilent !== previousInsight?.isSilent;

  const isNewSignal = isMessageChanged || isSeverityChanged || isSilentChanged;

  return {
    ...primaryInsight,
    is_new_signal: isNewSignal,
  };
}

/**
 * Bridge support for streamed telemetry from Dashboard Actions.
 */
export async function generateKairosAIInsight(
  telemetry: KairosTelemetry,
): Promise<KairosInsight> {
  return generateSystemInsights(telemetry);
}
```

File: src/lib/analytics/engine.test.ts

```
import assert from "node:assert/strict";
import test from "node:test";
import {
  projectRunway,
  calculateBurnRate,
  calculateIncomeMetrics,
} from "./engine";
import type { Deployment, IncomeStream } from "./types";

test("projectRunway handles dimensional math correctly", () => {
```

```
  // Scenario: 1000 balance, 10 daily burn, 0 income
  // Runway = 1000 / 10 = 100 days
  assert.equal(projectRunway(1000, 10, 0), 100);

  // Scenario: 1000 balance, 10 daily burn, 150 monthly income
  // adjustedDailyBurn = 10 - (150 / 30) = 5
  // Runway = 1000 / 5 = 200 days
  assert.equal(projectRunway(1000, 10, 150), 200);
});

test("projectRunway returns null when state is stable (income >= burn)", () => {
  // Scenario: 10 daily burn, 300 monthly income
  // adjustedDailyBurn = 10 - (300 / 30) = 0
  assert.equal(projectRunway(1000, 10, 300), null);

  // Scenario: 10 daily burn, 450 monthly income
  // adjustedDailyBurn = 10 - (450 / 30) = -5
  assert.equal(projectRunway(1000, 10, 450), null);
});

test("projectRunway returns 0 when balance is 0 or negative (and burn > income)", () => {
  assert.equal(projectRunway(0, 10, 0), 0);
  assert.equal(projectRunway(-100, 10, 0), 0);
});

test("projectRunway returns null when dailyBurnRate is 0 or negative (unstable state but no burn)", () => {
  assert.equal(projectRunway(1000, 0, 0), null);
  assert.equal(projectRunway(1000, -5, 0), null);
});

test("calculateBurnRate remains deterministic", () => {
  const deployments: Deployment[] = [
    {
      id: "1",
      amount: 300,
      title: "Rent",
      category: "Leakage",
      created_at: new Date().toISOString(),
    },
    {
      id: "2",
      amount: 150,
      title: "Cloud",
      category: "Leverage",
      created_at: new Date().toISOString(),
    },
  ];

  // 450 total / 30 days = 15/day
  assert.equal(calculateBurnRate(deployments, 30), 15);
});

test("calculateMaxIncomeConcentration: 90% dependency", () => {
  const streams: Partial<IncomeStream>[] = [
    {
      income_name: "Source A",
      amount: 900,
      cadence: "monthly",
      income_type: "salary",
      is_recurring: true,
    },
    {
      income_name: "Source B",
      amount: 100,
```

```
      cadence: "monthly",
      income_type: "freelance",
      is_recurring: true,
    },
  ];

  // total = 1000
  // max = 900
  // ratio = 0.9
  const metrics = calculateIncomeMetrics(streams as IncomeStream[]);
  assert.equal(metrics.maxRatio, 0.9);
});

test("calculateMaxIncomeConcentration: balanced split", () => {
  const streams: Partial<IncomeStream>[] = [
    {
      income_name: "Source A",
      amount: 100,
      cadence: "monthly",
      income_type: "salary",
      is_recurring: true,
    },
    {
      income_name: "Source B",
      amount: 100,
      cadence: "monthly",
      income_type: "salary",
      is_recurring: true,
    },
    {
      income_name: "Source C",
      amount: 100,
      cadence: "monthly",
      income_type: "salary",
      is_recurring: true,
    },
  ];

  // total = 300
  // max = 100
  // ratio = 0.333...
  const metrics = calculateIncomeMetrics(streams as IncomeStream[]);
  assert.ok(Math.abs(metrics.maxRatio - 0.3333333333333333) < 0.0001);
});

test("calculateMaxIncomeConcentration: zero income", () => {
  const metrics = calculateIncomeMetrics([]);
  assert.equal(metrics.maxRatio, 0);
});
```

File: src/lib/analytics/engine.ts

```
import {
  Deployment,
  AnalyticsSummary,
  Account,
  Liability,
  IncomeStream,
  FinancialGoal,
  StrategicObjective,
  StrategicAlignment,
  OperationalBaseline,
} from "./types";
```

```
/**
 * Normalizes cadences into monthly values.
 */
export const normalizeToMonthly = (amount: number, cadence: string): number => {
  switch (cadence) {
    case "daily":
      return amount * 30.4375;
    case "weekly":
      return amount * 4.345;
    case "biweekly":
      return amount * 2.1725;
    case "monthly":
      return amount;
    case "quarterly":
      return amount / 3;
    case "yearly":
      return amount / 12;
    default:
      return amount;
  }
};

/**
 * Calculates monthly structural burn and systemic allocation metrics.
 */
export const calculateBaselineMetrics = (baseline: OperationalBaseline[]) => {
  return baseline.reduce(
    (acc, item) => {
      if (!item.is_active) return acc;
      const monthlyAmount = normalizeToMonthly(item.amount, item.cadence);

      if (item.baseline_type === "expense") {
        acc.monthlyBurn += monthlyAmount;
      } else {
        acc.monthlyAllocation += monthlyAmount;
      }
      return acc;
    },
    { monthlyBurn: 0, monthlyAllocation: 0 },
  );
};
import { summarizeMetadataQuality } from "../finance/metadataQuality";
import { calculateMonthlyInflow } from "../finance/income";
import { calculateGoalProgressPercentage } from "../finance/goals";
import { calculateObjectiveFundingRatio } from "../finance/objectives";

/**
 * Pure, deterministic engine for financial intelligence.
 * Rule: The database is truth. This engine only interprets.
 */

export const calculateTotal = (deployments: Deployment[]): number => {
  return deployments.reduce((sum, d) => sum + Number(d.amount), 0);
};

export const calculateAccountTotal = (accounts: Account[]): number => {
  return accounts.reduce((sum, a) => sum + Number(a.current_balance), 0);
};

export const calculateLiabilityTotal = (liabilities: Liability[]): number => {
  return liabilities.reduce((sum, l) => sum + Number(l.outstanding_balance), 0);
};
```

```
export const calculateIncomeMetrics = (streams: IncomeStream[]) => {
  const result = streams.reduce(
    (acc, stream) => {
      const monthly = calculateMonthlyInflow(stream);
      acc.total += monthly;
      if (stream.is_recurring) {
        acc.recurring += monthly;
      } else {
        acc.irregular += monthly;
      }
      acc.concentration[stream.income_type] =
        (acc.concentration[stream.income_type] || 0) + monthly;

      // Group by origin source (income_name) for concentration risk analysis
      acc.sourceTotals[stream.income_name] =
        (acc.sourceTotals[stream.income_name] || 0) + monthly;

      return acc;
    },
    {
      total: 0,
      recurring: 0,
      irregular: 0,
      concentration: {} as Record<string, number>,
      sourceTotals: {} as Record<string, number>,
    },
  );

  const highestSourceSum = Math.max(0, ...Object.values(result.sourceTotals));
  const maxRatio = result.total > 0 ? highestSourceSum / result.total : 0;

  return {
    total: result.total,
    recurring: result.recurring,
    irregular: result.irregular,
    concentration: result.concentration,
    maxRatio,
  };
};

export const calculateGoalMetrics = (
  goals: FinancialGoal[],
  objectives: StrategicObjective[] = [],
) => {
  const goalStats = goals.reduce(
    (acc, goal) => {
      acc.totalTargets += Number(goal.target_amount);
      acc.totalProgress += Number(goal.current_progress);
      if (goal.priority === "critical" && goal.status === "active") {
        acc.criticalCount += 1;
      }
      acc.progressSum += calculateGoalProgressPercentage(goal);
      return acc;
    },
    {
      totalTargets: 0,
      totalProgress: 0,
      criticalCount: 0,
      progressSum: 0,
    },
  );

  return objectives.reduce((acc, obj) => {
    acc.totalTargets += Number(obj.target_amount);
```

```
    acc.totalProgress += Number(obj.current_amount);
    if (obj.priority_level === "critical" && obj.status === "active") {
      acc.criticalCount += 1;
    }
    acc.progressSum += calculateObjectiveFundingRatio(obj);
    return acc;
  }, goalStats);
};

export const calculateAverage = (deployments: Deployment[]): number => {
  if (deployments.length === 0) return 0;
  return calculateTotal(deployments) / deployments.length;
};

/**
 * Calculates burn rate over a specific window (in days).
 * Defaulting to a 30-day window for normalized metrics.
 */
export const calculateBurnRate = (
  deployments: Deployment[],
  windowDays: number = 30,
): number => {
  if (windowDays <= 0) return 0;
  const total = calculateTotal(deployments);
  return total / windowDays;
};

/**
 * Projects runway based on deterministic liquidity and income offset.
 * Correct Formula: Runway = balance / (dailyBurnRate - (monthlyIncome / 30))
 * Note: Returns days. Returns null if adjusted burn is <= 0 (stable state).
 */
export const projectRunway = (
  balance: number,
  dailyBurnRate: number,
  monthlyIncome: number = 0,
  netWorth: number = 0,
): number | null => {
  const adjustedDailyBurn = dailyBurnRate - monthlyIncome / 30;

  // If net worth is negative, the structural foundation is critical regardless of cash flow
  if (netWorth < 0) return 0;

  if (adjustedDailyBurn <= 0) return null;
  if (balance <= 0) return 0;

  return balance / adjustedDailyBurn;
};

/**
 * Aggregates deployments into categories.
 */
export const getCategoryBreakdown = (
  deployments: Deployment[],
): Record<string, number> => {
  return deployments.reduce(
    (acc, d) => {
      const cat = d.category || "Unclassified";
      acc[cat] = (acc[cat] || 0) + Number(d.amount);
      return acc;
    },
    {} as Record<string, number>,
  );
};
```

```
/**
 * Deterministic Strategic Alignment Logic
 */

export function calculateStrategicAlignment(
  objectives: StrategicObjective[],
  deployments: Deployment[],
  liquidity: number,
  incomeTotal: number,
  burnRate: number,
  liabilities: Liability[],
  baseline: OperationalBaseline[] = [],
): StrategicAlignment {
  const fundingRatios: Record<string, number> = {};
  const velocity: Record<
    string,
    "stable" | "accelerating" | "stagnant" | "regressing"
  > = {};
  const objectiveStarvationSignals: string[] = [];
  const strategicAllocationSignals: string[] = [];
  const now = new Date();

  // 1. Funding Ratios & Starvation
  objectives.forEach((obj) => {
    const ratio = calculateObjectiveFundingRatio(obj);
    fundingRatios[obj.id] = ratio;

    const ageInDays = Math.max(
      1,
      (now.getTime() - new Date(obj.created_at).getTime()) /
        (1000 * 60 * 60 * 24),
    );

    // Starvation detection
    if (obj.status === "active" && ratio < 10 && ageInDays > 90) {
      objectiveStarvationSignals.push(
        `${obj.objective_name} objective has remained below 10% funded for ${Math.floor(ageInDays)} days.`,
      );
    }

    // Velocity (Simplified deterministic approach based on age)
    const dailyRate = obj.current_amount / ageInDays;
    if (dailyRate === 0) {
      velocity[obj.id] = "stagnant";
    } else if (ratio >= 100) {
      velocity[obj.id] = "stable"; // Completed
    } else {
      velocity[obj.id] = "stable"; // Default for now as we don't have historical delta
    }
  });

  // 2. Strategic Allocation Awareness
  const categories = getCategoryBreakdown(deployments);
  const leakage = categories["Leakage"] || 0;
  const assets = categories["Asset"] || 0;
  const hasActiveAccumulation = objectives.some(
    (o) =>
      o.status === "active" &&
      (o.objective_type === "liquidity" ||
        o.objective_type === "emergency_reserve"),
  );

  if (hasActiveAccumulation && leakage > assets) {
```

```
    strategicAllocationSignals.push(
      "Leakage deployments exceeded Asset deployments during an active liquidity accumulation objective.",
    );
  }

  // 3. Liquidity Sufficiency
  const criticalObjectives = objectives.filter(
    (o) => o.priority_level === "critical" && o.status === "active",
  );
  const totalCriticalTarget = criticalObjectives.reduce(
    (sum, o) => sum + (o.target_amount - o.current_amount),
    0,
  );
  const isSufficient = liquidity >= totalCriticalTarget;
  const shortfall = Math.max(0, totalCriticalTarget - liquidity);

  const liquiditySufficiency = {
    isSufficient,
    message: isSufficient
      ? "Current liquidity structure sufficiently supports short-term obligations."
      : `Emergency reserve target exceeds current liquidity capacity by ${shortfall.toLocaleString()} KSh.`,
    shortfall,
  };

  // 4. Alignment Pressure (Weighted Scoring 0-100)
  let pressureScore = 0;

  // Weight: Leakage vs Assets (up to 25 points)
  if (leakage > assets) pressureScore += 25;
  else if (leakage > 0) pressureScore += 10;

  // Weight: Liquidity Sufficiency (up to 30 points)
  if (!isSufficient) pressureScore += 30;

  // Weight: Starvation (up to 20 points)
  if (objectiveStarvationSignals.length > 0) pressureScore += 20;

  // Weight: High Liabilities (up to 25 points)
  const totalLiabilities = liabilities.reduce(
    (sum, l) => sum + Number(l.outstanding_balance),
    0,
  );
  if (totalLiabilities > liquidity * 2) pressureScore += 25;

  return {
    fundingRatios,
    velocity,
    alignmentPressure: Math.min(100, pressureScore),
    liquiditySufficiency,
    objectiveStarvationSignals,
    strategicAllocationSignals,
  };
}

/**
 * Main entry point for generating a full analytics context.
 */
export const generateSummary = (
  deployments: Deployment[],
  liquidity: number = 0,
  accounts: Account[] = [],
  liabilities: Liability[] = [],
  incomeStreams: IncomeStream[] = [],
  goals: FinancialGoal[] = [],
```

```
  objectives: StrategicObjective[] = [],
  baseline: OperationalBaseline[] = [],
): AnalyticsSummary => {
  const total = calculateTotal(deployments);
  const baselineMetrics = calculateBaselineMetrics(baseline);

  // Total daily burn = (Deployment Burn) + (Baseline Burn / 30)
  const deploymentBurn = calculateBurnRate(deployments);
  const baselineDailyBurn = baselineMetrics.monthlyBurn / 30;
  const burnRate = deploymentBurn + baselineDailyBurn;

  const totalAssets = calculateAccountTotal(accounts);
  const totalLiabilities = calculateLiabilityTotal(liabilities);
  const netWorth = totalAssets - totalLiabilities;

  const income = calculateIncomeMetrics(incomeStreams);
  const goalMetrics = calculateGoalMetrics(goals, objectives);

  const strategicAlignment = calculateStrategicAlignment(
    objectives,
    deployments,
    liquidity,
    income.total,
    burnRate,
    liabilities,
    baseline,
  );

  const totalIntentionsCount = goals.length + objectives.length;

  return {
    totalDeployed: total,
    averageDeployment: calculateAverage(deployments),
    dailyBurnRate: burnRate,
    runwayDays: projectRunway(liquidity, burnRate, income.total, netWorth),
    categoryBreakdown: getCategoryBreakdown(deployments),
    deploymentCount: deployments.length,
    metadataQuality: summarizeMetadataQuality(deployments),
    // Liability System v1
    totalLiabilities,
    totalAssets,
    netWorth,
    liquidity,
    // Income Engine v1
    totalMonthlyIncome: income.total,
    recurringIncome: income.recurring,
    irregularIncome: income.irregular,
    incomeConcentration: income.concentration,
    maxIncomeConcentrationRatio: income.maxRatio,
    adjustedDailyBurn: Math.max(0, burnRate - income.total / 30),
    // Goal System v1
    totalStrategicTargets: goalMetrics.totalTargets,
    totalCurrentProgress: goalMetrics.totalProgress,
    averageGoalProgress:
      totalIntentionsCount > 0
        ? goalMetrics.progressSum / totalIntentionsCount
        : 0,
    criticalGoalCount: goalMetrics.criticalCount,
    fundingGap: Math.max(
      0,
      goalMetrics.totalTargets - goalMetrics.totalProgress,
    ),
    // Strategic Objectives v1
    strategicAlignment,
```

```
    // Operational Baseline v1
    totalStructuralMonthlyBurn: baselineMetrics.monthlyBurn,
    totalSystemicMonthlyAllocation: baselineMetrics.monthlyAllocation,
  };
};
```

File: src/lib/analytics/index.ts

```
export * from "./types";
export * from "./engine";
```

File: src/lib/analytics/strategic.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
import { calculateStrategicAlignment } from "./engine";
import type { Deployment, StrategicObjective, Liability } from "./types";

test("strategic alignment funding ratios are deterministic", () => {
  const objectives: StrategicObjective[] = [
    {
      id: "obj1",
      user_id: "u1",
      objective_name: "Emergency Fund",
      objective_type: "emergency_reserve",
      target_amount: 1000,
      current_amount: 500,
      target_date: null,
      priority_level: "critical",
      status: "active",
      created_at: new Date().toISOString(),
      updated_at: new Date().toISOString(),
    },
  ];

  const alignment = calculateStrategicAlignment(objectives, [], 0, 0, 0, []);
  assert.strictEqual(alignment.fundingRatios["obj1"], 50);
});

test("alignment pressure scoring reflects structural conflict", () => {
  const objectives: StrategicObjective[] = [
    {
      id: "obj1",
      user_id: "u1",
      objective_name: "Liquidity",
      objective_type: "liquidity",
      target_amount: 1000,
      current_amount: 0,
      target_date: null,
      priority_level: "critical",
      status: "active",
      created_at: new Date().toISOString(),
      updated_at: new Date().toISOString(),
    },
  ];

  const deployments: Deployment[] = [
    {
      id: "d1",
      amount: 500,
      title: "Bad Spending",
      category: "Leakage",
```

```
      created_at: new Date().toISOString(),
    },
  ];

  const liabilities: Liability[] = [
    {
      id: "l1",
      user_id: "u1",
      liability_name: "Debt",
      liability_type: "personal_loan",
      outstanding_balance: 10000,
      created_at: "",
      updated_at: "",
      interest_rate: 0,
      minimum_payment: 0,
      due_date: null,
      currency: "KSh",
    },
  ];

  // Scenario: High Leakage, No Assets, Insufficient Liquidity, High Liabilities
  const alignment = calculateStrategicAlignment(
    objectives,
    deployments,
    1000,
    0,
    0,
    liabilities,
  );

  // Leakage > Assets (+25)
  // !isSufficient (+30) - totalCriticalTarget is 1000, liquidity is 1000. Wait, 1000 >= 1000 is true.
  // totalLiabilities (10000) > liquidity * 2 (2000) (+25)
  // Score should be at least 50.

  assert.ok(alignment.alignmentPressure >= 50);
});

test("liquidity sufficiency detects shortfalls", () => {
  const objectives: StrategicObjective[] = [
    {
      id: "obj1",
      user_id: "u1",
      objective_name: "Reserve",
      objective_type: "emergency_reserve",
      target_amount: 5000,
      current_amount: 1000,
      target_date: null,
      priority_level: "critical",
      status: "active",
      created_at: new Date().toISOString(),
      updated_at: new Date().toISOString(),
    },
  ];

  const alignment = calculateStrategicAlignment(objectives, [], 2000, 0, 0, []);

  assert.strictEqual(alignment.liquiditySufficiency.isSufficient, false);
  assert.strictEqual(alignment.liquiditySufficiency.shortfall, 2000); // 4000 remaining target - 2000 liquidity
});

test("objective starvation detection works for old stagnant goals", () => {
  const sixMonthsAgo = new Date();
  sixMonthsAgo.setMonth(sixMonthsAgo.getMonth() - 6);
```

```
  const objectives: StrategicObjective[] = [
    {
      id: "obj1",
      user_id: "u1",
      objective_name: "Starved Goal",
      objective_type: "investment",
      target_amount: 10000,
      current_amount: 100, // 1%
      target_date: null,
      priority_level: "high",
      status: "active",
      created_at: sixMonthsAgo.toISOString(),
      updated_at: sixMonthsAgo.toISOString(),
    },
  ];

  const alignment = calculateStrategicAlignment(objectives, [], 0, 0, 0, []);

  assert.ok(alignment.objectiveStarvationSignals.length > 0);
  assert.ok(alignment.objectiveStarvationSignals[0].includes("Starved Goal"));
});

test("strategic allocation awareness detects leakage conflict", () => {
  const objectives: StrategicObjective[] = [
    {
      id: "obj1",
      user_id: "u1",
      objective_name: "Build Cash",
      objective_type: "liquidity",
      target_amount: 1000,
      current_amount: 0,
      target_date: null,
      priority_level: "high",
      status: "active",
      created_at: new Date().toISOString(),
      updated_at: new Date().toISOString(),
    },
  ];

  const deployments: Deployment[] = [
    {
      id: "d1",
      amount: 1000,
      title: "Leakage",
      category: "Leakage",
      created_at: new Date().toISOString(),
    },
    {
      id: "d2",
      amount: 500,
      title: "Asset",
      category: "Asset",
      created_at: new Date().toISOString(),
    },
  ];

  const alignment = calculateStrategicAlignment(
    objectives,
    deployments,
    0,
    0,
    0,
    [],
```

```
  );

  assert.ok(alignment.strategicAllocationSignals.length > 0);
  assert.ok(
    alignment.strategicAllocationSignals[0].includes(
      "Leakage deployments exceeded Asset deployments",
    ),
  );
});
```

File: src/lib/analytics/types.ts

```
import { MetadataQualitySummary } from "../finance/metadataQuality";
import { DeploymentAdvancedContext } from "../finance/deploymentContext";
import { Account } from "../finance/accounts";
import { Liability } from "../finance/liabilities";
import { IncomeStream } from "../finance/income";
import { FinancialGoal } from "../finance/goals";
import { StrategicObjective } from "../finance/objectives";

export type { Account, AccountType } from "../finance/accounts";
export type { Liability, LiabilityType } from "../finance/liabilities";
export type { IncomeStream, IncomeType } from "../finance/income";
export type {
  FinancialGoal,
  GoalType,
  GoalPriority,
  GoalStatus,
} from "../finance/goals";
export type {
  StrategicObjective,
  ObjectiveType,
  ObjectivePriority,
  ObjectiveStatus,
} from "../finance/objectives";

export type BaselineCadence =
  | "daily"
  | "weekly"
  | "biweekly"
  | "monthly"
  | "quarterly"
  | "yearly";

export type BaselineType = "expense" | "allocation";

export interface OperationalBaseline {
  id: string;
  user_id: string;
  title: string;
  amount: number;
  category: string;
  cadence: BaselineCadence;
  baseline_type: BaselineType;
  is_active: boolean;
  notes?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export interface Deployment {
  id: string;
```

```
  amount: number;
  created_at: string;
  category?: string | null;
  title: string;
  advanced_context?: DeploymentAdvancedContext | null;
  account_id?: string | null;
  deleted_at?: string | null;
}

export interface StrategicAlignment {
  fundingRatios: Record<string, number>; // objective_id -> ratio (0-100)
  alignmentPressure: number; // 0-100 deterministic score
  liquiditySufficiency: {
    isSufficient: boolean;
    message: string;
    shortfall: number;
  };
  objectiveStarvationSignals: string[];
  strategicAllocationSignals: string[];
  velocity: Record<
    string,
    "stable" | "accelerating" | "stagnant" | "regressing"
  >;
}

export interface AnalyticsSummary {
  totalDeployed: number;
  averageDeployment: number;
  dailyBurnRate: number;
  runwayDays: number | null;
  categoryBreakdown: Record<string, number>;
  deploymentCount: number;
  metadataQuality: MetadataQualitySummary;
  // Liability System v1
  totalLiabilities: number;
  netWorth: number;
  totalAssets: number;
  liquidity: number;
  // Income Engine v1
  totalMonthlyIncome: number;
  recurringIncome: number;
  irregularIncome: number;
  incomeConcentration: Record<string, number>;
  maxIncomeConcentrationRatio: number;
  adjustedDailyBurn: number;
  // Goal System v1
  totalStrategicTargets: number;
  totalCurrentProgress: number;
  averageGoalProgress: number;
  criticalGoalCount: number;
  fundingGap: number;
  // Strategic Objectives v1
  strategicAlignment: StrategicAlignment;
  // Operational Baseline v1
  totalStructuralMonthlyBurn: number;
  totalSystemicMonthlyAllocation: number;
}
```

File: src/lib/context/behavioralFlags.ts

```
import { TrendState } from "./contextTypes";

/**
```

```
 * Identifies discrete behavioral signals for AI interpretation.
 * Objective: High signal, low token usage.
 */

export const generateBehavioralFlags = (params: {
  burnTrend: TrendState;
  volatility: number;
  concentration: number;
  runwayDays: number | null;
  dominantCategory: string;
}): string[] => {
  const flags: string[] = [];

  // 1. Burn Signals
  if (params.burnTrend === "increasing") flags.push("accelerating_burn");
  if (params.burnTrend === "decreasing") flags.push("burn_optimization_detected");

  // 2. Consistency Signals
  if (params.volatility > 0.6) flags.push("erratic_spending_pattern");
  if (params.volatility < 0.2 && params.volatility > 0) flags.push("high_operational_discipline");

  // 3. Allocation Signals
  if (params.concentration > 0.7) flags.push(`heavy_concentration:${params.dominantCategory.toLowerCase()}`);
  if (params.concentration < 0.3) flags.push("fragmented_capital_allocation");

  // 4. Criticality Signals
  if (params.runwayDays !== null && params.runwayDays < 30) flags.push("critical_runway_warning");
  if (params.runwayDays !== null && params.runwayDays > 180) flags.push("extended_operational_security");

  return flags;
};

/**
 * Analyzes category concentration.
 * Returns a score between 0 (even spread) and 1 (all in one category).
 */
export const calculateConcentrationScore = (distribution: Record<string, number>): number => {
  const values = Object.values(distribution);
  if (values.length === 0) return 0;
  if (values.length === 1) return 1;

  // Use Herfindahl-Hirschman Index (HHI) approach for concentration
  // Sum of squares of shares
  const hhi = values.reduce((sum, val) => sum + Math.pow(val, 2), 0);
  return hhi;
};
```

File: src/lib/context/buildBehavioralContext.ts

```
import { BehavioralContext, ContextInput } from "./contextTypes";

import {
  calculateTrendState,
  calculateVolatility,
  inferDiscipline,
} from "./trendAnalysis";
import {
  generateBehavioralFlags,
  calculateConcentrationScore,
} from "./behavioralFlags";
import { isValidCategory } from "../finance/taxonomy";

/**
```

```
 * Axiom Behavioral Context Builder
 * Transforms analytics signals into a stable behavioral state for Kairos.
 */
export const buildBehavioralContext = (
  input: ContextInput,
  deploymentAmounts: number[],
): BehavioralContext => {
  const { currentAnalytics, historicalMetrics } = input;

  // 1. Calculate Trends
  const burnTrend = calculateTrendState(
    currentAnalytics.dailyBurnRate,
    historicalMetrics?.previousBurnRate || 0,
  );

  const runwayDelta =
    currentAnalytics.runwayDays !== null &&
    historicalMetrics?.previousRunwayDays !== undefined
      ? currentAnalytics.runwayDays -
        (historicalMetrics.previousRunwayDays || 0)
      : 0;

  // 2. Allocation Analysis
  // 2. Allocation Analysis
  const total = currentAnalytics.totalDeployed || 1;
  const distribution: Record<string, number> = {};
  let dominantCategory = "None";
  let maxAmt = -1;

  let unclassifiedTotal = 0;

  Object.entries(currentAnalytics.categoryBreakdown).forEach(([cat, amt]) => {
    if (!isValidCategory(cat)) {
      unclassifiedTotal += amt;
    } else {
      distribution[cat] = amt / total;
      if (amt > maxAmt) {
        maxAmt = amt;
        dominantCategory = cat;
      }
    }
  });

  // Add the merged 'Unclassified' group
  if (unclassifiedTotal > 0) {
    distribution["Unclassified"] = unclassifiedTotal / total;
    if (unclassifiedTotal > maxAmt) {
      dominantCategory = "Unclassified";
    }
  }

  const concentrationScore = calculateConcentrationScore(distribution);
  const volatility = calculateVolatility(deploymentAmounts);

  // 3. Efficiency Calculation (Heuristic)
  // Starts at 100, penalized by volatility, accelerating burn, and low runway.
  let efficiency = 100;
  efficiency -= volatility * 60; // Much more sensitive to erratic spending
  if (burnTrend === "increasing") efficiency -= 15;
  if (currentAnalytics.runwayDays && currentAnalytics.runwayDays < 90)
    efficiency -= 40; // Aggressive penalty for low runway

  // 4. Status Determination
```

```
  // 4. Status Determination
  let runwayStatus: "healthy" | "concerning" | "critical" = "healthy";
  if (currentAnalytics.runwayDays !== null) {
    if (currentAnalytics.runwayDays < 30) runwayStatus = "critical";
    else if (currentAnalytics.runwayDays < 90) runwayStatus = "concerning";
  }

  return {
    generatedAt: new Date().toISOString(),
    deploymentCount: currentAnalytics.deploymentCount,
    capitalEfficiencyScore: Math.round(Math.max(efficiency, 0)),

    netWorth: currentAnalytics.netWorth,
    liquidity: currentAnalytics.liquidity,
    totalMonthlyIncome: currentAnalytics.totalMonthlyIncome,
    maxIncomeConcentrationRatio: currentAnalytics.maxIncomeConcentrationRatio,

    strategicAlignment: currentAnalytics.strategicAlignment,

    burnRate: {
      current: currentAnalytics.dailyBurnRate,
      trend: burnTrend,
      monthlyProjection: currentAnalytics.dailyBurnRate * 30,
    },

    runway: {
      currentDays: currentAnalytics.runwayDays,
      deltaDays: runwayDelta,
      status: runwayStatus,
    },

    allocation: {
      dominantCategory,
      categoryDistribution: distribution,
      concentrationScore,
    },

    volatilityScore: volatility,

    metadataQuality: currentAnalytics.metadataQuality,

    disciplineTrend: inferDiscipline(burnTrend, runwayDelta),

    behavioralFlags: generateBehavioralFlags({
      burnTrend,
      volatility,
      concentration: concentrationScore,
      runwayDays: currentAnalytics.runwayDays,
      dominantCategory,
    }),
  };
};

/**
 * Context Compression
 * Strips metadata to optimize for LLM token usage while preserving signal.
 */
export const compressContextForAI = (context: BehavioralContext): string => {
  return JSON.stringify({
    score: context.capitalEfficiencyScore,
    burn: `${context.burnRate.current}/day (${context.burnRate.trend})`,
    runway: `${context.runway.currentDays}d (${context.runway.status})`,
                                    allocation:         `${context.allocation.dominantCategory}         (conc:
${context.allocation.concentrationScore.toFixed(2)})`,
```

```
    flags: context.behavioralFlags.join(","),
    discipline: context.disciplineTrend,
    metadata: `${context.metadataQuality.lowQualityCount}/${context.deploymentCount} low-quality labels`,
  });
};
```

File: src/lib/context/contextTypes.ts

```
import { StrategicAlignment } from "../analytics/types";
import { MetadataQualitySummary } from "../finance/metadataQuality";

export type TrendState = "increasing" | "stable" | "decreasing";
export type DisciplineState = "improving" | "stable" | "declining";

export interface AllocationState {
  dominantCategory: string;
  categoryDistribution: Record<string, number>; // Percentage based (0-1)
  concentrationScore: number; // 0 to 1, where 1 is total concentration in one category
}

export interface BehavioralContext {
  generatedAt: string;
  deploymentCount: number;

  // High-level aggregate metric (0-100)
  capitalEfficiencyScore: number;

  netWorth: number;
  liquidity: number;
  totalMonthlyIncome: number;
  maxIncomeConcentrationRatio: number;

  strategicAlignment: StrategicAlignment;

  burnRate: {
    current: number;
    trend: TrendState;
    monthlyProjection: number;
  };

  runway: {
    currentDays: number | null;
    deltaDays: number; // Change from previous context
    status: "healthy" | "concerning" | "critical";
  };

  allocation: AllocationState;

  // Discrete behavioral markers
  behavioralFlags: string[];

  // Measure of spending consistency
  volatilityScore: number; // 0 to 1, higher is more erratic

  metadataQuality: MetadataQualitySummary;

  disciplineTrend: DisciplineState;
}

/**
 * Input required for the context builder.
 * Represents the current state vs a previous snapshot.
 */
```

```
export interface ContextInput {
  currentAnalytics: {
    totalDeployed: number;
    dailyBurnRate: number;
    runwayDays: number | null;
    categoryBreakdown: Record<string, number>;
    deploymentCount: number;
    metadataQuality: MetadataQualitySummary;
    netWorth: number;
    liquidity: number;
    totalMonthlyIncome: number;
    maxIncomeConcentrationRatio: number;
    strategicAlignment: StrategicAlignment;
  };
  historicalMetrics?: {
    previousBurnRate: number;
    previousRunwayDays: number | null;
    previousTotalDeployed: number;
  };
}
```

File: src/lib/context/trendAnalysis.ts

```
import { TrendState, DisciplineState } from "./contextTypes";

/**
 * Determines the trend direction between two values.
 * Threshold (0.05) prevents noise from triggering trend shifts.
 */
export const calculateTrendState = (
  current: number,
  previous: number,
  threshold: number = 0.05
): TrendState => {
  if (!previous || previous === 0) return "stable";

  const change = (current - previous) / previous;

  if (change > threshold) return "increasing";
  if (change < -threshold) return "decreasing";
  return "stable";
};

/**
 * Calculates a volatility score (0-1) based on the variance of deployment sizes.
 * High variance = High volatility (erratic spending).
 */
export const calculateVolatility = (amounts: number[]): number => {
  if (amounts.length < 2) return 0;

  const mean = amounts.reduce((a, b) => a + b, 0) / amounts.length;
  const variance = amounts.reduce((a, b) => a + Math.pow(b - mean, 2), 0) / amounts.length;
  const stdDev = Math.sqrt(variance);

  // Normalize volatility relative to the mean.
  // A standard deviation equal to the mean results in a score of 0.5.
  const score = stdDev / (mean + stdDev);
  return Math.min(Math.max(score, 0), 1);
};

/**
 * Infers discipline trend by observing burn rate vs total deployment.
 */
```

```
export const inferDiscipline = (
  burnTrend: TrendState,
  runwayDelta: number
): DisciplineState => {
  if (burnTrend === "decreasing" && runwayDelta > 0) return "improving";
  if (burnTrend === "increasing" && runwayDelta < 0) return "declining";
  return "stable";
};
```

File: src/lib/db/accounts.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateAccount, type AccountType } from "../finance/accounts";

export async function getAccounts(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("accounts")
    .select("*")
    .is("deleted_at", null)
    .order("account_name", { ascending: true });

  if (error) throw error;
  return data;
}

export async function createAccount(
  supabase: SupabaseClient,
  userId: string,
  data: {
    account_name: string;
    account_type: string;
    current_balance: number;
    institution?: string;
    currency?: string;
    deleted_at?: string | null;
  },
) {
  const validated = validateAccount({
    account_name: data.account_name,
    account_type: data.account_type,
    current_balance: data.current_balance,
  });

  const { data: account, error } = await supabase
    .from("accounts")
    .insert({
      user_id: userId,
      account_name: validated.account_name,
      account_type: validated.account_type,
      current_balance: validated.current_balance,
      institution: data.institution,
      currency: data.currency || "KSh",
    })
    .select()
    .single();

  if (error) throw error;
  return account;
}

export async function updateAccount(
  supabase: SupabaseClient,
  id: string,
```

```
  userId: string,
  updates: {
    account_name?: string;
    account_type?: string;
    current_balance?: number;
    institution?: string;
    deleted_at?: string | null;
  },
) {
  const dbUpdates: Record<string, unknown> = { ...updates };

  if (
    updates.account_name !== undefined ||
    updates.account_type !== undefined ||
    updates.current_balance !== undefined
  ) {
    // Basic validation for what's provided
    if (
      updates.account_name !== undefined &&
      updates.account_name.trim().length === 0
    ) {
      throw new Error("Account name is required.");
    }
    // Note: We don't use full validateAccount here to allow partial updates
  }

  const { data, error } = await supabase
    .from("accounts")
    .update(dbUpdates)
    .eq("id", id)
    .eq("user_id", userId)
    .select()
    .single();

  if (error) throw error;
  return data;
}

export async function deleteAccount(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("accounts")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

File: src/lib/db/audit.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";

export type AuditRecordType = "deployment" | "income" | "liability";

export interface AuditRecord {
  id: string;
  type: AuditRecordType;
  name: string;
```

```
  amount: number;
  cadence?: string;
  deleted_at: string;
  original_created_at: string;
}

/**
 * Retrieves all soft-deleted records from the ledger tables (deployments, income_streams, liabilities).
 * Returns a unified, chronologically sorted timeline of deletions.
 */
export async function getDeletedLedgerRecords(
  supabase: SupabaseClient,
  userId: string,
): Promise<AuditRecord[]> {
  const [deployments, incomeStreams, liabilities] = await Promise.all([
    supabase
      .from("deployments")
      .select("id, title, amount, deleted_at, created_at")
      .eq("user_id", userId)
      .not("deleted_at", "is", null),
    supabase
      .from("income_streams")
      .select("id, income_name, amount, cadence, deleted_at, created_at")
      .eq("user_id", userId)
      .not("deleted_at", "is", null),
    supabase
      .from("liabilities")
      .select("id, liability_name, outstanding_balance, deleted_at, created_at")
      .eq("user_id", userId)
      .not("deleted_at", "is", null),
  ]);

  if (deployments.error) throw deployments.error;
  if (incomeStreams.error) throw incomeStreams.error;
  if (liabilities.error) throw liabilities.error;

  const normalizedDeployments: AuditRecord[] = (deployments.data || []).map(
    (d) => ({
      id: d.id,
      type: "deployment",
      name: d.title,
      amount: d.amount,
      deleted_at: d.deleted_at!,
      original_created_at: d.created_at,
    }),
  );

  const normalizedIncome: AuditRecord[] = (incomeStreams.data || []).map(
    (i) => ({
      id: i.id,
      type: "income",
      name: i.income_name,
      amount: i.amount,
      cadence: i.cadence,
      deleted_at: i.deleted_at!,
      original_created_at: i.created_at,
    }),
  );

  const normalizedLiabilities: AuditRecord[] = (liabilities.data || []).map(
    (l) => ({
      id: l.id,
      type: "liability",
      name: l.liability_name,
```

```
      amount: l.outstanding_balance,
      deleted_at: l.deleted_at!,
      original_created_at: l.created_at,
    }),
  );

  return [
    ...normalizedDeployments,
    ...normalizedIncome,
    ...normalizedLiabilities,
  ].sort(
    (a, b) =>
      new Date(b.deleted_at).getTime() - new Date(a.deleted_at).getTime(),
  );
}
```

File: src/lib/db/baseline.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { OperationalBaseline } from "../analytics/types";

export async function getOperationalBaseline(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("operational_baseline")
    .select("*")
    .is("deleted_at", null)
    .order("created_at", { ascending: true });

  if (error) throw error;
  return data as OperationalBaseline[];
}

export async function createOperationalBaseline(
  supabase: SupabaseClient,
  userId: string,
  data: Omit<
    OperationalBaseline,
    "id" | "user_id" | "created_at" | "updated_at" | "deleted_at"
  >,
) {
  const { data: baseline, error } = await supabase
    .from("operational_baseline")
    .insert({
      user_id: userId,
      ...data,
    })
    .select()
    .single();

  if (error) throw error;
  return baseline as OperationalBaseline;
}

export async function updateOperationalBaseline(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: Partial<
    Omit<
      OperationalBaseline,
      "id" | "user_id" | "created_at" | "updated_at" | "deleted_at"
    >
  >,
```

```
) {
  const { data: baseline, error } = await supabase
    .from("operational_baseline")
    .update(updates)
    .eq("id", id)
    .eq("user_id", userId)
    .select()
    .single();

  if (error) throw error;
  return baseline as OperationalBaseline;
}

export async function deleteOperationalBaseline(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("operational_baseline")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

File: src/lib/db/deployments.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateDeployment } from "../finance/validateDeployment";
import {
  hasDeploymentAdvancedContext,
  type DeploymentAdvancedContextInput,
} from "../finance/deploymentContext";

function isMissingAdvancedContextColumn(error: { message?: string }) {
  return Boolean(error.message?.toLowerCase().includes("advanced_context"));
}

export async function getDeployments(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("deployments")
    .select("*")
    .is("deleted_at", null)
    .order("created_at", { ascending: false });

  if (error) throw error;
  return data;
}

/**
 * Creates a new deployment record.
 * Guaranteed to pass through the Taxonomy Enforcement Gate.
 */
export async function createDeployment(
  supabase: SupabaseClient,
  title: string,
  amount: number,
  userId: string,
  category: string,
  impactScore: number = 0,
```

```
  advancedContext: DeploymentAdvancedContextInput = {},
  accountId?: string,
) {
  // 1. Pass through Taxonomy Enforcement Gate
  const validated = validateDeployment({
    title,
    amount,
    category,
    impactScore,
    advancedContext,
  });

  // 2. Database Write (Verified Truth)
  const deploymentRecord = {
    title: validated.title,
    amount: validated.amount,
    user_id: userId,
    category: validated.category,
    impact_score: validated.impactScore,
    account_id: accountId || null,
    ...(hasDeploymentAdvancedContext(validated.advancedContext)
      ? { advanced_context: validated.advancedContext }
      : {}),
  };

  const { data, error } = await supabase
    .from("deployments")
    .insert(deploymentRecord);

  if (error) {
    if (
      hasDeploymentAdvancedContext(validated.advancedContext) &&
      isMissingAdvancedContextColumn(error)
    ) {
      throw new Error(
           "Advanced Context schema unavailable. Apply supabase-setup.sql before saving advanced deployment
metadata.",
      );
    }

    throw error;
  }
  return data;
}

/**
 * Updates an existing deployment record.
 * Enforces partial taxonomy validation for updated fields.
 */
export async function updateDeployment(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: {
    title?: string;
    amount?: number;
    category?: string;
    impact_score?: number;
    advancedContext?: DeploymentAdvancedContextInput;
    accountId?: string;
  },
) {
  // 1. Enforcement for provided fields
  const dbUpdates: Record<string, unknown> = {};
```

```
  if (updates.accountId !== undefined) {
    dbUpdates.account_id = updates.accountId || null;
  }

  const shouldValidate =
    updates.title !== undefined ||
    updates.amount !== undefined ||
    updates.category !== undefined ||
    updates.advancedContext !== undefined ||
    updates.impact_score !== undefined;

  if (shouldValidate) {
    // We simulate a full input to reuse the Gate logic,
    // but only use the validated fields for the update.
    const validated = validateDeployment({
      title: updates.title ?? "Temporary Title",
      amount: updates.amount ?? 1,
      category: updates.category ?? "Leakage", // Explicit default for gate reuse
      impactScore: updates.impact_score,
      advancedContext: updates.advancedContext,
    });

    if (updates.title !== undefined) dbUpdates.title = validated.title;
    if (updates.amount !== undefined) dbUpdates.amount = validated.amount;
    if (updates.category !== undefined) dbUpdates.category = validated.category;
    if (updates.impact_score !== undefined)
      dbUpdates.impact_score = validated.impactScore;
    if (updates.advancedContext !== undefined)
      dbUpdates.advanced_context = validated.advancedContext;
  }

  // 2. Database Update
  const { data, error } = await supabase
    .from("deployments")
    .update(dbUpdates)
    .eq("id", id)
    .eq("user_id", userId);

  if (error) {
    if (
      updates.advancedContext !== undefined &&
      isMissingAdvancedContextColumn(error)
    ) {
      throw new Error(
           "Advanced Context schema unavailable. Apply supabase-setup.sql before saving advanced deployment
metadata.",
      );
    }

    throw error;
  }
  return data;
}

export async function deleteDeployment(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("deployments")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
```

```
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

File: src/lib/db/goals.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateGoal } from "../finance/goals";

export async function getGoals(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("financial_goals")
    .select("*")
    .is("deleted_at", null)
    .order("priority", { ascending: false });

  if (error) throw error;
  return data;
}

export async function createGoal(
  supabase: SupabaseClient,
  userId: string,
  data: {
    goal_name: string;
    goal_type: string;
    target_amount: number;
    current_progress?: number;
    target_date?: string | null;
    priority: string;
    status: string;
    notes?: string;
    deleted_at?: string | null;
  },
) {
  const validated = validateGoal({
    goal_name: data.goal_name,
    goal_type: data.goal_type,
    target_amount: data.target_amount,
    current_progress: data.current_progress,
    priority: data.priority,
    status: data.status,
  });

  const { data: goal, error } = await supabase
    .from("financial_goals")
    .insert({
      user_id: userId,
      goal_name: validated.goal_name,
      goal_type: validated.goal_type,
      target_amount: validated.target_amount,
      current_progress: validated.current_progress,
      target_date: data.target_date,
      priority: validated.priority,
      status: validated.status,
      notes: data.notes,
    })
    .select()
    .single();

  if (error) throw error;
```

```
  return goal;
}

export async function updateGoal(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: {
    goal_name?: string;
    goal_type?: string;
    target_amount?: number;
    current_progress?: number;
    target_date?: string | null;
    priority?: string;
    status?: string;
    notes?: string | null;
    deleted_at?: string | null;
  },
) {
  const dbUpdates: Record<string, unknown> = { ...updates };

  if (
    updates.goal_name !== undefined &&
    updates.goal_name.trim().length === 0
  ) {
    throw new Error("Goal name is required.");
  }

  const { data, error } = await supabase
    .from("financial_goals")
    .update(dbUpdates)
    .eq("id", id)
    .eq("user_id", userId)
    .select()
    .single();

  if (error) throw error;
  return data;
}

export async function deleteGoal(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("financial_goals")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

File: src/lib/db/income.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateIncomeStream } from "../finance/income";

export async function getIncomeStreams(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("income_streams")
```

```
    .select("*")
    .is("deleted_at", null)
    .order("income_name", { ascending: true });

  if (error) throw error;
  return data;
}

export async function createIncomeStream(
  supabase: SupabaseClient,
  userId: string,
  data: {
    income_name: string;
    income_type: string;
    amount: number;
    cadence: string;
    execution_day?: number | null;
    is_recurring?: boolean;
    source?: string;
    currency?: string;
    start_date?: string;
    end_date?: string | null;
    deleted_at?: string | null;
  },
) {
  const validated = validateIncomeStream({
    income_name: data.income_name,
    income_type: data.income_type,
    amount: data.amount,
    cadence: data.cadence,
    execution_day: data.execution_day,
    is_recurring: data.is_recurring,
  });

  const { data: stream, error } = await supabase
    .from("income_streams")
    .insert({
      user_id: userId,
      income_name: validated.income_name,
      income_type: validated.income_type,
      amount: validated.amount,
      cadence: validated.cadence,
      execution_day: validated.execution_day,
      is_recurring: validated.is_recurring,
      source: data.source,
      currency: data.currency || "KSh",
      start_date: data.start_date || new Date().toISOString().split("T")[0],
      end_date: data.end_date,
    })
    .select()
    .single();

  if (error) throw error;
  return stream;
}

export async function createIncomeStreams(
  supabase: SupabaseClient,
  userId: string,
  inputs: {
    income_name: string;
    income_type: string;
    amount: number;
    cadence: string;
```

```
    execution_day?: number | null;
    is_recurring?: boolean;
    source?: string;
    currency?: string;
    start_date?: string;
    end_date?: string | null;
    deleted_at?: string | null;
  }[],
) {
  const validatedInputs = inputs.map((data) => {
    const validated = validateIncomeStream({
      income_name: data.income_name,
      income_type: data.income_type,
      amount: data.amount,
      cadence: data.cadence,
      execution_day: data.execution_day,
      is_recurring: data.is_recurring,
    });

    return {
      user_id: userId,
      income_name: validated.income_name,
      income_type: validated.income_type,
      amount: validated.amount,
      cadence: validated.cadence,
      execution_day: validated.execution_day,
      is_recurring: validated.is_recurring,
      source: data.source,
      currency: data.currency || "KSh",
      start_date: data.start_date || new Date().toISOString().split("T")[0],
      end_date: data.end_date,
    };
  });

  const { data: streams, error } = await supabase
    .from("income_streams")
    .insert(validatedInputs)
    .select();

  if (error) throw error;
  return streams;
}

export async function updateIncomeStream(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: {
    income_name?: string;
    income_type?: string;
    amount?: number;
    cadence?: string;
    execution_day?: number | null;
    last_executed_at?: string | null;
    is_recurring?: boolean;
    source?: string;
    start_date?: string;
    end_date?: string | null;
    deleted_at?: string | null;
  },
) {
  const dbUpdates: Record<string, unknown> = { ...updates };

  if (
```

```
    updates.income_name !== undefined &&
    updates.income_name.trim().length === 0
  ) {
    throw new Error("Income name is required.");
  }

  const { data, error } = await supabase
    .from("income_streams")
    .update(dbUpdates)
    .eq("id", id)
    .eq("user_id", userId)
    .select()
    .single();

  if (error) throw error;
  return data;
}

export async function deleteIncomeStream(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("income_streams")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

File: src/lib/db/insights.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import type { KairosInsight } from "@/lib/ai/kairos";

export async function saveInsight(
  supabase: SupabaseClient,
  insight: KairosInsight,
  userId: string,
) {
  const { data, error } = await supabase.from("kairos_insights").insert({
    user_id: userId,
    type: insight.type,
    category: insight.category,
    confidence: 1.0, // Deterministic logic has absolute confidence
    message: insight.message,
    metadata: {
      related_ids: [],
      metadata_quality: insight.metadataQuality || null,
      severity: insight.severity,
      supporting_signals: insight.supportingSignals,
      runway: insight.runway,
      capital_efficiency: insight.capitalEfficiency,
      is_silent: insight.isSilent,
      timestamp: insight.timestamp,
      source: "server_orchestrator_v5e",
    },
  });

  if (error) {
```

```
    console.error("Error saving insight:", error);
    throw error;
  }
  return data;
}

export async function getInsights(supabase: SupabaseClient, limit: number = 5) {
  const { data, error } = await supabase
    .from("kairos_insights")
    .select("*")
    .order("created_at", { ascending: false })
    .limit(limit);

  if (error) {
    console.error("Error fetching insights:", error);
    throw error;
  }
  return data;
}
```

File: src/lib/db/liabilities.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateLiability } from "../finance/liabilities";

export async function getLiabilities(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("liabilities")
    .select("*")
    .is("deleted_at", null)
    .order("liability_name", { ascending: true });

  if (error) throw error;
  return data;
}

export async function createLiability(
  supabase: SupabaseClient,
  userId: string,
  data: {
    liability_name: string;
    liability_type: string;
    outstanding_balance: number;
    interest_rate?: number;
    minimum_payment?: number;
    due_date?: string | null;
    institution?: string;
    currency?: string;
    deleted_at?: string | null;
  },
) {
  const validated = validateLiability({
    liability_name: data.liability_name,
    liability_type: data.liability_type,
    outstanding_balance: data.outstanding_balance,
    interest_rate: data.interest_rate,
    minimum_payment: data.minimum_payment,
  });

  const { data: liability, error } = await supabase
    .from("liabilities")
    .insert({
      user_id: userId,
```

```
      liability_name: validated.liability_name,
      liability_type: validated.liability_type,
      outstanding_balance: validated.outstanding_balance,
      interest_rate: validated.interest_rate,
      minimum_payment: validated.minimum_payment,
      due_date: data.due_date,
      institution: data.institution,
      currency: data.currency || "KSh",
    })
    .select()
    .single();

  if (error) throw error;
  return liability;
}

export async function updateLiability(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: {
    liability_name?: string;
    liability_type?: string;
    outstanding_balance?: number;
    interest_rate?: number;
    minimum_payment?: number;
    due_date?: string | null;
    institution?: string;
    deleted_at?: string | null;
  },
) {
  const dbUpdates: Record<string, unknown> = { ...updates };

  if (
    updates.liability_name !== undefined &&
    updates.liability_name.trim().length === 0
  ) {
    throw new Error("Liability name is required.");
  }

  const { data, error } = await supabase
    .from("liabilities")
    .update(dbUpdates)
    .eq("id", id)
    .eq("user_id", userId)
    .select()
    .single();

  if (error) throw error;
  return data;
}

export async function deleteLiability(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("liabilities")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
```

```
  return true;
}
```

File: src/lib/db/objectives.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";
import { validateObjective } from "../finance/objectives";

export async function getStrategicObjectives(supabase: SupabaseClient) {
  const { data, error } = await supabase
    .from("strategic_objectives")
    .select("*")
    .is("deleted_at", null)
    .order("priority_level", { ascending: false })
    .order("created_at", { ascending: false });

  if (error) throw error;
  return data;
}

export async function createStrategicObjective(
  supabase: SupabaseClient,
  userId: string,
  data: {
    objective_name: string;
    objective_type: string;
    target_amount: number;
    current_amount?: number;
    target_date?: string | null;
    priority_level: string;
    status: string;
    notes?: string;
    deleted_at?: string | null;
  },
) {
  const validated = validateObjective({
    objective_name: data.objective_name,
    objective_type: data.objective_type,
    target_amount: data.target_amount,
    current_amount: data.current_amount,
    priority_level: data.priority_level,
    status: data.status,
  });

  const { data: objective, error } = await supabase
    .from("strategic_objectives")
    .insert({
      user_id: userId,
      objective_name: validated.objective_name,
      objective_type: validated.objective_type,
      target_amount: validated.target_amount,
      current_amount: validated.current_amount,
      target_date: data.target_date,
      priority_level: validated.priority_level,
      status: validated.status,
      notes: data.notes,
    })
    .select()
    .single();

  if (error) throw error;
  return objective;
}
```

```
export async function updateStrategicObjective(
  supabase: SupabaseClient,
  id: string,
  userId: string,
  updates: {
    objective_name?: string;
    objective_type?: string;
    target_amount?: number;
    current_amount?: number;
    target_date?: string | null;
    priority_level?: string;
    status?: string;
    notes?: string | null;
    deleted_at?: string | null;
  },
) {
  // If objective_name is provided, validate it's not empty
  if (
    updates.objective_name !== undefined &&
    updates.objective_name.trim().length === 0
  ) {
    throw new Error("Objective name is required.");
  }

  const { data, error } = await supabase
    .from("strategic_objectives")
    .update(updates)
    .eq("id", id)
    .eq("user_id", userId)
    .select()
    .single();

  if (error) throw error;
  return data;
}

export async function deleteStrategicObjective(
  supabase: SupabaseClient,
  id: string,
  userId: string,
) {
  const { error } = await supabase
    .from("strategic_objectives")
    .update({ deleted_at: new Date().toISOString() })
    .eq("id", id)
    .eq("user_id", userId);

  if (error) throw error;
  return true;
}
```

File: src/lib/db/settings.ts

```
import type { SupabaseClient } from "@supabase/supabase-js";

export async function getUserSettings(
  supabase: SupabaseClient,
  userId: string,
) {
  const { data, error } = await supabase
    .from("user_settings")
    .select("*")
```

```
    .eq("user_id", userId)
    .maybeSingle();

  if (error) throw error;

  // If no settings exist, create default
  if (!data) {
    const { data: newData, error: createError } = await supabase
      .from("user_settings")
      .insert({ user_id: userId, total_liquidity: 0 })
      .select()
      .single();

    if (createError) throw createError;
    return newData;
  }

  return data;
}

export async function updateLiquidity(
  supabase: SupabaseClient,
  userId: string,
  amount: number,
) {
  const { data, error } = await supabase
    .from("user_settings")
    .upsert({
      user_id: userId,
      total_liquidity: amount,
      updated_at: new Date().toISOString(),
    })
    .select()
    .single();

  if (error) throw error;
  return data;
}
```

File: src/lib/db/telemetry.ts

```
import { createClient } from "@/lib/supabase/server";

/**
 * TELEMETRY SERVICE (Phase 1)
 * Records insight evaluation events for observability.
 */
export async function logKairosEvaluation(data: {
  ruleId: string;
  severity: string;
  durationMs: number;
}) {
  try {
    let supabase;
    try {
      supabase = await createClient();
    } catch (e) {
      // If we're in a test environment or outside a request scope, cookies() will fail.
      // We silenty exit as telemetry is non-critical for these contexts.
      return;
    }

    const {
```

```
      data: { user },
    } = await supabase.auth.getUser();
    if (!user) return;

    // Record evaluation event to kairos_insights table.
    // This phase uses the rule registry ID and performance metadata.
    const { error } = await supabase.from("kairos_insights").insert({
      user_id: user.id,
      rule_id: data.ruleId,
      severity: data.severity,
      evaluation_time_ms: data.durationMs,
    });

    if (error) {
      console.error("[Telemetry] Error recording evaluation:", error.message);
    }
  } catch (err) {
    // Observability must be non-blocking; we swallow errors to protect primary flows.
    console.error("[Telemetry] Critical failure in logging:", err);
  }
}
```

File: src/lib/finance/accounts.ts

```
export const ACCOUNT_TYPES = [
  { value: "checking", label: "Checking" },
  { value: "savings", label: "Savings" },
  { value: "mobile_money", label: "Mobile Money" },
  { value: "brokerage", label: "Brokerage" },
  { value: "crypto", label: "Crypto" },
  { value: "cash", label: "Cash" },
] as const;

export type AccountType = (typeof ACCOUNT_TYPES)[number]["value"];

export interface Account {
  id: string;
  user_id: string;
  account_name: string;
  account_type: AccountType;
  current_balance: number;
  currency: string;
  institution?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export function isValidAccountType(type: string): type is AccountType {
  return ACCOUNT_TYPES.some((t) => t.value === type);
}

export function validateAccount(data: {
  account_name: string;
  account_type: string;
  current_balance: number;
}) {
  if (!data.account_name || data.account_name.trim().length === 0) {
    throw new Error("Account name is required.");
  }

  if (!isValidAccountType(data.account_type)) {
    throw new Error(`Invalid account type: ${data.account_type}`);
```

```
  }

  if (isNaN(data.current_balance)) {
    throw new Error("Initial balance must be a number.");
  }

  return {
    account_name: data.account_name.trim(),
    account_type: data.account_type as AccountType,
    current_balance: data.current_balance,
  };
}
```

File: src/lib/finance/deploymentContext.ts

```
export const EXPECTED_RETURN_HORIZONS = [
  { value: "short-term", label: "Short-term" },
  { value: "medium-term", label: "Medium-term" },
  { value: "long-term", label: "Long-term" },
] as const;

export type ExpectedReturnHorizon =
  (typeof EXPECTED_RETURN_HORIZONS)[number]["value"];

export interface DeploymentAdvancedContext {
  associatedAccount?: string;
  expectedReturnHorizon?: ExpectedReturnHorizon;
  tags?: string[];
}

export interface DeploymentAdvancedContextInput {
  associatedAccount?: string | null;
  expectedReturnHorizon?: string | null;
  tags?: string | string[] | null;
}

const EXPECTED_RETURN_HORIZON_VALUES = new Set(
  EXPECTED_RETURN_HORIZONS.map((horizon) => horizon.value),
);

export function isExpectedReturnHorizon(
  value: string,
): value is ExpectedReturnHorizon {
  return EXPECTED_RETURN_HORIZON_VALUES.has(value as ExpectedReturnHorizon);
}

function normalizeTags(tags: DeploymentAdvancedContextInput["tags"]) {
  const source = Array.isArray(tags) ? tags.join(",") : tags || "";
  const normalizedTags = source
    .split(",")
    .map((tag) => tag.trim().toLowerCase())
    .filter(Boolean);

  return Array.from(new Set(normalizedTags));
}

export function normalizeDeploymentAdvancedContext(
  input: DeploymentAdvancedContextInput | null = {},
): DeploymentAdvancedContext {
  const safeInput = input || {};
  const associatedAccount = (safeInput.associatedAccount || "").trim();
  const expectedReturnHorizon = (
    safeInput.expectedReturnHorizon || ""
```

```
  ).trim();
  const tags = normalizeTags(safeInput.tags);

  if (
    expectedReturnHorizon &&
    !isExpectedReturnHorizon(expectedReturnHorizon)
  ) {
    throw new Error(
      "Context Gate: Unknown expected return horizon. Use short-term, medium-term, or long-term.",
    );
  }

  return {
    ...(associatedAccount ? { associatedAccount } : {}),
    ...(expectedReturnHorizon
      ? { expectedReturnHorizon: expectedReturnHorizon as ExpectedReturnHorizon }
      : {}),
    ...(tags.length > 0 ? { tags } : {}),
  };
}

export function hasDeploymentAdvancedContext(
  context: DeploymentAdvancedContext,
) {
  return Boolean(
    context.associatedAccount ||
      context.expectedReturnHorizon ||
      (context.tags && context.tags.length > 0),
  );
}
```

File: src/lib/finance/financeRules.test.ts

```
import assert from "node:assert/strict";
import test from "node:test";

import {
  evaluateMetadataQuality,
  summarizeMetadataQuality,
  weightConfidenceForMetadataQuality,
} from "./metadataQuality";
import {
  normalizeDeploymentAdvancedContext,
} from "./deploymentContext";
import { isValidCategory } from "./taxonomy";
import { validateDeployment } from "./validateDeployment";

test("taxonomy validation accepts only canonical return categories", () => {
  assert.equal(isValidCategory("Asset"), true);
  assert.equal(isValidCategory("Leakage"), true);
  assert.equal(isValidCategory("Unclassified"), false);
  assert.equal(isValidCategory("Unknown"), false);
});

test("metadata quality scoring is deterministic for generic labels", () => {
  assert.deepEqual(evaluateMetadataQuality(" misc ").reasons, [
    "generic_label",
  ]);
  assert.equal(evaluateMetadataQuality("stuff").isLowQuality, true);
  assert.equal(evaluateMetadataQuality("Revenue engine setup").isLowQuality, false);
});

test("metadata quality summary produces stable confidence weighting", () => {
```

```
  const summary = summarizeMetadataQuality([
    { title: "misc" },
    { title: "Brokerage allocation" },
  ]);

  assert.equal(summary.lowQualityCount, 1);
  assert.equal(summary.lowQualityRatio, 0.5);
  assert.equal(summary.confidenceMultiplier, 0.93);
  assert.equal(weightConfidenceForMetadataQuality(0.9, summary), 0.84);
});

test("advanced context normalization preserves optional structured metadata", () => {
  assert.deepEqual(
    normalizeDeploymentAdvancedContext({
      associatedAccount: " Brokerage ",
      expectedReturnHorizon: "long-term",
      tags: "Ops, recurring, ops",
    }),
    {
      associatedAccount: "Brokerage",
      expectedReturnHorizon: "long-term",
      tags: ["ops", "recurring"],
    },
  );
});

test("advanced context rejects unknown return horizons", () => {
  assert.throws(
    () =>
      normalizeDeploymentAdvancedContext({
        expectedReturnHorizon: "eventually",
      }),
    /Unknown expected return horizon/,
  );
});

test("deployment validation remains the write gate", () => {
  assert.throws(
    () =>
      validateDeployment({
        title: "Capital",
        amount: 100,
        category: "Unclassified",
      }),
    /Strategic classification is mandatory/,
  );

  assert.deepEqual(
    validateDeployment({
      title: " Cloud infra ",
      amount: 500,
      category: "Leverage",
      advancedContext: {
        expectedReturnHorizon: "medium-term",
        tags: "ops, infra",
      },
    }).advancedContext,
    {
      expectedReturnHorizon: "medium-term",
      tags: ["ops", "infra"],
    },
  );
});
```

File: src/lib/finance/goals.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
import { validateGoal, calculateGoalProgressPercentage } from "./goals";
import { generateSummary } from "../analytics/engine";
import type { FinancialGoal } from "../analytics/types";

test("goal validation enforces structural integrity", () => {
  const data = {
    goal_name: "Emergency Fund",
    goal_type: "emergency_fund",
    target_amount: 1000000,
    current_progress: 100000,
    priority: "critical",
    status: "active",
  };

  const validated = validateGoal(data);
  assert.strictEqual(validated.goal_name, "Emergency Fund");
  assert.strictEqual(validated.goal_type, "emergency_fund");
  assert.strictEqual(validated.target_amount, 1000000);
  assert.strictEqual(validated.current_progress, 100000);
});

test("goal progress calculation is deterministic", () => {
  assert.strictEqual(calculateGoalProgressPercentage({ target_amount: 1000, current_progress: 500 }), 50);
  assert.strictEqual(calculateGoalProgressPercentage({ target_amount: 1000, current_progress: 1000 }), 100);
  assert.strictEqual(calculateGoalProgressPercentage({ target_amount: 1000, current_progress: 1500 }), 100);
  assert.strictEqual(calculateGoalProgressPercentage({ target_amount: 0, current_progress: 500 }), 0);
});

test("strategic metrics are accurately aggregated", () => {
  const goals: FinancialGoal[] = [
    {
      id: "g1",
      user_id: "u1",
      goal_name: "G1",
      goal_type: "emergency_fund",
      target_amount: 1000,
      current_progress: 200,
      priority: "critical",
      status: "active",
      linked_account_ids: [],
      created_at: "",
      updated_at: "",
      target_date: null,
    },
    {
      id: "g2",
      user_id: "u1",
      goal_name: "G2",
      goal_type: "retirement",
      target_amount: 1000,
      current_progress: 800,
      priority: "medium",
      status: "active",
      linked_account_ids: [],
      created_at: "",
      updated_at: "",
      target_date: null,
    },
  ];
```

```
  const summary = generateSummary([], 0, [], [], [], goals);

  assert.strictEqual(summary.totalStrategicTargets, 2000);
  assert.strictEqual(summary.totalCurrentProgress, 1000);
  assert.strictEqual(summary.averageGoalProgress, 50); // (20% + 80%) / 2
  assert.strictEqual(summary.criticalGoalCount, 1);
  assert.strictEqual(summary.fundingGap, 1000);
});
```

File: src/lib/finance/goals.ts

```
export const GOAL_TYPES = [
  { value: "emergency_fund", label: "Emergency Fund" },
  { value: "retirement", label: "Retirement" },
  { value: "home_purchase", label: "Home Purchase" },
  { value: "investment_target", label: "Investment Target" },
  { value: "business_runway", label: "Business Runway" },
  { value: "education", label: "Education" },
  { value: "debt_payoff", label: "Debt Payoff" },
  { value: "wealth_preservation", label: "Wealth Preservation" },
  { value: "other", label: "Other" },
] as const;

export type GoalType = (typeof GOAL_TYPES)[number]["value"];

export const GOAL_PRIORITIES = [
  { value: "critical", label: "Critical" },
  { value: "high", label: "High" },
  { value: "medium", label: "Medium" },
  { value: "low", label: "Low" },
] as const;

export type GoalPriority = (typeof GOAL_PRIORITIES)[number]["value"];

export const GOAL_STATUSES = [
  { value: "active", label: "Active" },
  { value: "paused", label: "Paused" },
  { value: "achieved", label: "Achieved" },
  { value: "archived", label: "Archived" },
] as const;

export type GoalStatus = (typeof GOAL_STATUSES)[number]["value"];

export interface FinancialGoal {
  id: string;
  user_id: string;
  goal_name: string;
  goal_type: GoalType;
  target_amount: number;
  current_progress: number;
  target_date: string | null;
  priority: GoalPriority;
  status: GoalStatus;
  linked_account_ids: string[];
  notes?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export function isValidGoalType(type: string): type is GoalType {
  return GOAL_TYPES.some((t) => t.value === type);
```

```
}

export function isValidPriority(priority: string): priority is GoalPriority {
  return GOAL_PRIORITIES.some((p) => p.value === priority);
}

export function isValidStatus(status: string): status is GoalStatus {
  return GOAL_STATUSES.some((s) => s.value === status);
}

export function validateGoal(data: {
  goal_name: string;
  goal_type: string;
  target_amount: number;
  current_progress?: number;
  priority: string;
  status: string;
}) {
  if (!data.goal_name || data.goal_name.trim().length === 0) {
    throw new Error("Goal name is required.");
  }

  if (!isValidGoalType(data.goal_type)) {
    throw new Error(`Invalid goal type: ${data.goal_type}`);
  }

  if (isNaN(data.target_amount) || data.target_amount <= 0) {
    throw new Error("Target amount must be a positive number.");
  }

  if (data.current_progress !== undefined && isNaN(data.current_progress)) {
    throw new Error("Current progress must be a number.");
  }

  if (!isValidPriority(data.priority)) {
    throw new Error(`Invalid priority: ${data.priority}`);
  }

  if (!isValidStatus(data.status)) {
    throw new Error(`Invalid status: ${data.status}`);
  }

  return {
    goal_name: data.goal_name.trim(),
    goal_type: data.goal_type as GoalType,
    target_amount: data.target_amount,
    current_progress: data.current_progress || 0,
    priority: data.priority as GoalPriority,
    status: data.status as GoalStatus,
  };
}

/**
 * Calculates deterministic goal progress.
 */
export function calculateGoalProgressPercentage(
  goal: Pick<FinancialGoal, "current_progress" | "target_amount">,
): number {
  if (goal.target_amount <= 0) return 0;
  return Math.min(
    100,
    Math.max(0, (goal.current_progress / goal.target_amount) * 100),
  );
}
```

File: src/lib/finance/income.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
import { validateIncomeStream, calculateMonthlyInflow } from "./income";
import { generateSummary } from "../analytics/engine";
import type { Deployment, IncomeStream } from "../analytics/types";

test("income validation enforces structural integrity", () => {
  const data = {
    income_name: "Salary",
    income_type: "salary",
    amount: 100000,
    cadence: "monthly",
  };

  const validated = validateIncomeStream(data);
  assert.strictEqual(validated.income_name, "Salary");
  assert.strictEqual(validated.income_type, "salary");
  assert.strictEqual(validated.amount, 100000);
  assert.strictEqual(validated.cadence, "monthly");
});

test("income normalization is accurate", () => {
  assert.strictEqual(
    calculateMonthlyInflow({ amount: 1000, cadence: "weekly" }),
    1000 * (52 / 12),
  );
  assert.strictEqual(
    calculateMonthlyInflow({ amount: 120000, cadence: "annually" }),
    10000,
  );
  assert.strictEqual(
    calculateMonthlyInflow({ amount: 5000, cadence: "monthly" }),
    5000,
  );
});

test("runway logic incorporates income offset correctly", () => {
  const deployments: Deployment[] = [
    {
      id: "1",
      amount: 3000,
      created_at: new Date().toISOString(),
      title: "Rent",
      category: "Fixed",
    },
  ]; // 100/day burn over 30 days

  const liquidity = 1000;
  const incomeStreams: IncomeStream[] = [
    {
      id: "s1",
      user_id: "u1",
      income_name: "Salary",
      amount: 1500,
      cadence: "monthly",
      is_recurring: true,
      income_type: "salary",
      currency: "KSh",
      start_date: "",
      created_at: "",
      updated_at: "",
```

```
    },
  ]; // 50/day offset

  // Correct Formula: Liquidity / (Daily Burn - (Monthly Income / 30))
  // 1000 / (100 - (1500 / 30)) = 1000 / 50 = 20 days
  const summary = generateSummary(
    deployments,
    liquidity,
    [],
    [],
    incomeStreams,
  );

  assert.strictEqual(summary.dailyBurnRate, 100);
  assert.strictEqual(summary.totalMonthlyIncome, 1500);
  assert.strictEqual(summary.runwayDays, 20);
  assert.strictEqual(summary.adjustedDailyBurn, 50);
});
```

File: src/lib/finance/income.ts

```
export const INCOME_TYPES = [
  { value: "salary", label: "Salary" },
  { value: "freelance", label: "Freelance" },
  { value: "business", label: "Business" },
  { value: "dividends", label: "Dividends" },
  { value: "rental", label: "Rental" },
  { value: "contract", label: "Contract" },
  { value: "other", label: "Other" },
] as const;

export type IncomeType = (typeof INCOME_TYPES)[number]["value"];

export const CADENCES = [
  { value: "weekly", label: "Weekly", factor: 52 / 12 },
  { value: "biweekly", label: "Biweekly", factor: 26 / 12 },
  { value: "monthly", label: "Monthly", factor: 1 },
  { value: "quarterly", label: "Quarterly", factor: 1 / 3 },
  { value: "annually", label: "Annually", factor: 1 / 12 },
  { value: "irregular", label: "Irregular", factor: 0 },
] as const;

export type Cadence = (typeof CADENCES)[number]["value"];

export interface IncomeStream {
  id: string;
  user_id: string;
  income_name: string;
  income_type: IncomeType;
  amount: number;
  cadence: Cadence;
  execution_day?: number | null;
  last_executed_at?: string | null;
  is_recurring: boolean;
  currency: string;
  source?: string | null;
  start_date: string;
  end_date?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}
```

```
export function isValidIncomeType(type: string): type is IncomeType {
  return INCOME_TYPES.some((t) => t.value === type);
}

export function isValidCadence(cadence: string): cadence is Cadence {
  return CADENCES.some((c) => c.value === cadence);
}

export function validateIncomeStream(data: {
  income_name: string;
  income_type: string;
  amount: number;
  cadence: string;
  execution_day?: number | null;
  is_recurring?: boolean;
}) {
  if (!data.income_name || data.income_name.trim().length === 0) {
    throw new Error("Income name is required.");
  }

  if (!isValidIncomeType(data.income_type)) {
    throw new Error(`Invalid income type: ${data.income_type}`);
  }

  if (!isValidCadence(data.cadence)) {
    throw new Error(`Invalid cadence: ${data.cadence}`);
  }

  if (isNaN(data.amount) || data.amount < 0) {
    throw new Error("Amount must be a non-negative number.");
  }

  return {
    income_name: data.income_name.trim(),
    income_type: data.income_type as IncomeType,
    amount: data.amount,
    cadence: data.cadence as Cadence,
    execution_day: data.execution_day,
    is_recurring: data.is_recurring ?? true,
  };
}

/**
 * Normalizes an income stream amount to a monthly value.
 */
export function calculateMonthlyInflow(
  stream: Pick<IncomeStream, "amount" | "cadence">,
): number {
  const cadenceConfig = CADENCES.find((c) => c.value === stream.cadence);
  if (!cadenceConfig) return 0;
  return stream.amount * cadenceConfig.factor;
}
```

File: src/lib/finance/liabilities.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
import { validateLiability } from "./liabilities";
import { generateSummary } from "../analytics/engine";
import type { Account, Liability } from "../analytics/types";

test("liability validation enforces structural integrity", () => {
  const data = {
```

```
    liability_name: "Credit Card",
    liability_type: "credit_card",
    outstanding_balance: 50000,
  };

  const validated = validateLiability(data);
  assert.strictEqual(validated.liability_name, "Credit Card");
  assert.strictEqual(validated.liability_type, "credit_card");
  assert.strictEqual(validated.outstanding_balance, 50000);
});

test("liability validation rejects empty names", () => {
  assert.throws(() => {
    validateLiability({
      liability_name: "",
      liability_type: "credit_card",
      outstanding_balance: 1000,
    });
  }, /Liability name is required/);
});

test("liability validation rejects invalid types", () => {
  assert.throws(() => {
    validateLiability({
      liability_name: "Test",
      liability_type: "gambling_debt",
      outstanding_balance: 1000,
    });
  }, /Invalid liability type/);
});

test("net worth calculation is deterministic", () => {
  const accounts: Account[] = [
    {
      id: "1",
      user_id: "u1",
      account_name: "Checking",
      account_type: "checking",
      current_balance: 100000,
      currency: "KSh",
      created_at: "",
      updated_at: "",
      institution: null,
    },
  ];

  const liabilities: Liability[] = [
    {
      id: "l1",
      user_id: "u1",
      liability_name: "Loan",
      liability_type: "personal_loan",
      outstanding_balance: 30000,
      interest_rate: 0,
      minimum_payment: 0,
      currency: "KSh",
      created_at: "",
      updated_at: "",
      institution: null,
      due_date: null,
    },
  ];

  const summary = generateSummary([], 100000, accounts, liabilities);
```

```
  assert.strictEqual(summary.totalAssets, 100000);
  assert.strictEqual(summary.totalLiabilities, 30000);
  assert.strictEqual(summary.netWorth, 70000);
});
```

File: src/lib/finance/liabilities.ts

```
export const LIABILITY_TYPES = [
  { value: "credit_card", label: "Credit Card" },
  { value: "mortgage", label: "Mortgage" },
  { value: "personal_loan", label: "Personal Loan" },
  { value: "student_loan", label: "Student Loan" },
  { value: "business_loan", label: "Business Loan" },
  { value: "line_of_credit", label: "Line of Credit" },
  { value: "other", label: "Other" },
] as const;

export type LiabilityType = (typeof LIABILITY_TYPES)[number]["value"];

export interface Liability {
  id: string;
  user_id: string;
  liability_name: string;
  liability_type: LiabilityType;
  outstanding_balance: number;
  interest_rate: number;
  minimum_payment: number;
  due_date: string | null;
  currency: string;
  institution?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export function isValidLiabilityType(type: string): type is LiabilityType {
  return LIABILITY_TYPES.some((t) => t.value === type);
}

export function validateLiability(data: {
  liability_name: string;
  liability_type: string;
  outstanding_balance: number;
  interest_rate?: number;
  minimum_payment?: number;
}) {
  if (!data.liability_name || data.liability_name.trim().length === 0) {
    throw new Error("Liability name is required.");
  }

  if (!isValidLiabilityType(data.liability_type)) {
    throw new Error(`Invalid liability type: ${data.liability_type}`);
  }

  if (isNaN(data.outstanding_balance)) {
    throw new Error("Outstanding balance must be a number.");
  }

  return {
    liability_name: data.liability_name.trim(),
    liability_type: data.liability_type as LiabilityType,
    outstanding_balance: data.outstanding_balance,
```

```
    interest_rate: data.interest_rate || 0,
    minimum_payment: data.minimum_payment || 0,
  };
}
```

File: src/lib/finance/metadataQuality.ts

```
export type MetadataQualityLevel = "specific" | "low";

export interface MetadataQualityEvaluation {
  level: MetadataQualityLevel;
  score: number;
  isLowQuality: boolean;
  normalizedLabel: string;
  reasons: string[];
}

export interface MetadataQualitySummary {
  averageScore: number;
  lowQualityCount: number;
  lowQualityRatio: number;
  confidenceMultiplier: number;
}

type LabelBearingInput = {
  title?: string | null;
};

const LOW_QUALITY_EXACT_LABELS = new Set([
  "misc",
  "stuff",
  "other",
  "random",
  "generic",
  "unclear",
  "unknown",
  "untitled",
  "no title",
  "none",
  "nil",
  "na",
  "n/a",
  "tbd",
  "test",
]);

const EMPTY_LIKE_PATTERNS = [/^-+$/, /^\.+$/, /^\?+$/, /^_+$/];

const roundQualityScore = (value: number) => Math.round(value * 100) / 100;

export function normalizeMetadataLabel(label: string) {
  return label.trim().toLowerCase().replace(/\s+/g, " ");
}

export function evaluateMetadataQuality(
  label: string | null | undefined,
): MetadataQualityEvaluation {
  const normalizedLabel = normalizeMetadataLabel(label || "");
  const semanticLabel = normalizedLabel
    .replace(/[^a-z0-9/?._-]+/g, " ")
    .replace(/\s+/g, " ")
    .trim();
  const comparableLabel = semanticLabel.replace(
```

```
    /^[^a-z0-9]+|[^a-z0-9]+$/g,
    "",
  );

  const reasons: string[] = [];

  if (
    !semanticLabel ||
    EMPTY_LIKE_PATTERNS.some((pattern) => pattern.test(semanticLabel))
  ) {
    reasons.push("empty_like_label");
  }

  if (
    LOW_QUALITY_EXACT_LABELS.has(semanticLabel) ||
    LOW_QUALITY_EXACT_LABELS.has(comparableLabel)
  ) {
    reasons.push("generic_label");
  }

  const isLowQuality = reasons.length > 0;

  return {
    level: isLowQuality ? "low" : "specific",
    score: isLowQuality ? 0.35 : 1,
    isLowQuality,
    normalizedLabel,
    reasons,
  };
}

export function summarizeMetadataQuality(
  deployments: LabelBearingInput[],
): MetadataQualitySummary {
  if (deployments.length === 0) {
    return {
      averageScore: 1,
      lowQualityCount: 0,
      lowQualityRatio: 0,
      confidenceMultiplier: 1,
    };
  }

  const evaluations = deployments.map((deployment) =>
    evaluateMetadataQuality(deployment.title),
  );
  const lowQualityCount = evaluations.filter(
    (evaluation) => evaluation.isLowQuality,
  ).length;
  const averageScore =
    evaluations.reduce((sum, evaluation) => sum + evaluation.score, 0) /
    evaluations.length;
  const lowQualityRatio = lowQualityCount / evaluations.length;

  return {
    averageScore: roundQualityScore(averageScore),
    lowQualityCount,
    lowQualityRatio: roundQualityScore(lowQualityRatio),
    confidenceMultiplier: roundQualityScore(
      Math.max(0.85, 1 - lowQualityRatio * 0.15),
    ),
  };
}
```

```
export function weightConfidenceForMetadataQuality(
  confidence: number,
  metadataQuality: MetadataQualitySummary,
) {
  return roundQualityScore(
    Math.min(1, Math.max(0, confidence * metadataQuality.confidenceMultiplier)),
  );
}
```

File: src/lib/finance/objectives.test.ts

```
import { test } from "node:test";
import assert from "node:assert";
import { validateObjective } from "./objectives";

test("strategic objective validation enforces structural integrity", () => {
  const data = {
    objective_name: "Safety Net",
    objective_type: "emergency_reserve",
    target_amount: 500000,
    current_amount: 50000,
    priority_level: "critical",
    status: "active",
  };

  const validated = validateObjective(data);
  assert.strictEqual(validated.objective_name, "Safety Net");
  assert.strictEqual(validated.objective_type, "emergency_reserve");
  assert.strictEqual(validated.target_amount, 500000);
  assert.strictEqual(validated.current_amount, 50000);
  assert.strictEqual(validated.priority_level, "critical");
  assert.strictEqual(validated.status, "active");
});

test("objective validation fails on invalid types", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "invalid_type",
      target_amount: 1000,
      priority_level: "high",
      status: "active",
    });
  }, /Invalid objective type/);
});

test("objective validation fails on invalid priority", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 1000,
      priority_level: "very_high",
      status: "active",
    });
  }, /Invalid priority level/);
});

test("objective validation fails on invalid status", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
```

```
      target_amount: 1000,
      priority_level: "high",
      status: "unknown",
    });
  }, /Invalid status/);
});

test("objective validation fails on zero or negative target amount", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 0,
      priority_level: "high",
      status: "active",
    });
  }, /Target amount must be a positive number/);

  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: -1,
      priority_level: "high",
      status: "active",
    });
  }, /Target amount must be a positive number/);
});

test("objective validation fails on overfunded state", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 1000,
      current_amount: 1001,
      priority_level: "high",
      status: "active",
    });
  }, /Current amount cannot exceed target amount/);
});

test("objective validation handles date normalization", () => {
  const validated = validateObjective({
    objective_name: "Test",
    objective_type: "liquidity",
    target_amount: 1000,
    priority_level: "high",
    status: "active",
    target_date: "2026-12-31",
  });
  assert.strictEqual(validated.target_date, "2026-12-31");
});

test("objective validation fails on malformed dates", () => {
  assert.throws(() => {
    validateObjective({
      objective_name: "Test",
      objective_type: "liquidity",
      target_amount: 1000,
      priority_level: "high",
      status: "active",
      target_date: "not-a-date",
    });
```

```
  }, /Invalid target date format/);
});

test("objective normalization trims whitespace and handles empty notes", () => {
  const validated = validateObjective({
    objective_name: "  Safety Net  ",
    objective_type: "emergency_reserve",
    target_amount: 1000,
    priority_level: "critical",
    status: "active",
    notes: "   Need this for safety   ",
  });
  assert.strictEqual(validated.objective_name, "Safety Net");
  assert.strictEqual(validated.notes, "Need this for safety");

  const validatedEmptyNotes = validateObjective({
    objective_name: "Test",
    objective_type: "liquidity",
    target_amount: 1000,
    priority_level: "high",
    status: "active",
    notes: "    ",
  });
  assert.strictEqual(validatedEmptyNotes.notes, null);
});
```

File: src/lib/finance/objectives.ts

```
export const OBJECTIVE_TYPES = [
  { value: "emergency_reserve", label: "Emergency Reserve" },
  { value: "liquidity", label: "Liquidity" },
  { value: "debt_elimination", label: "Debt Elimination" },
  { value: "investment", label: "Investment" },
  { value: "retirement", label: "Retirement" },
  { value: "education", label: "Education" },
  { value: "acquisition", label: "Acquisition" },
  { value: "custom", label: "Custom" },
] as const;

export type ObjectiveType = (typeof OBJECTIVE_TYPES)[number]["value"];

export const OBJECTIVE_PRIORITIES = [
  { value: "critical", label: "Critical" },
  { value: "high", label: "High" },
  { value: "moderate", label: "Moderate" },
  { value: "low", label: "Low" },
] as const;

export type ObjectivePriority = (typeof OBJECTIVE_PRIORITIES)[number]["value"];

export const OBJECTIVE_STATUSES = [
  { value: "active", label: "Active" },
  { value: "paused", label: "Paused" },
  { value: "completed", label: "Completed" },
  { value: "abandoned", label: "Abandoned" },
] as const;

export type ObjectiveStatus = (typeof OBJECTIVE_STATUSES)[number]["value"];

export interface StrategicObjective {
  id: string;
  user_id: string;
  objective_name: string;
```

```
  objective_type: ObjectiveType;
  target_amount: number;
  current_amount: number;
  target_date: string | null;
  priority_level: ObjectivePriority;
  status: ObjectiveStatus;
  notes?: string | null;
  created_at: string;
  updated_at: string;
  deleted_at?: string | null;
}

export function isValidObjectiveType(type: string): type is ObjectiveType {
  return OBJECTIVE_TYPES.some((t) => t.value === type);
}

export function isValidPriority(
  priority: string,
): priority is ObjectivePriority {
  return OBJECTIVE_PRIORITIES.some((p) => p.value === priority);
}

export function isValidStatus(status: string): status is ObjectiveStatus {
  return OBJECTIVE_STATUSES.some((s) => s.value === status);
}

/**
 * Validates and normalizes strategic objective data.
 * This is the canonical authority for objective correctness.
 */
/**
 * Calculates deterministic objective progress percentage.
 */
export function calculateObjectiveProgressPercentage(
  objective: Pick<StrategicObjective, "current_amount" | "target_amount">,
): number {
  if (objective.target_amount <= 0) return 0;
  return Math.min(
    100,
    Math.max(0, (objective.current_amount / objective.target_amount) * 100),
  );
}

/**
 * Calculates the funding ratio (alias for progress for semantic clarity in UI).
 */
export function calculateObjectiveFundingRatio(
  objective: Pick<StrategicObjective, "current_amount" | "target_amount">,
): number {
  return calculateObjectiveProgressPercentage(objective);
}

export function validateObjective(data: {
  objective_name: string;
  objective_type: string;
  target_amount: number;
  current_amount?: number;
  priority_level: string;
  status: string;
  target_date?: string | null;
  notes?: string | null;
}) {
  // 1. Required Fields & Basic Types
  if (!data.objective_name || data.objective_name.trim().length === 0) {
```

```
    throw new Error("Objective name is required.");
  }

  // 2. Enum Validations
  if (!isValidObjectiveType(data.objective_type)) {
    throw new Error(`Invalid objective type: ${data.objective_type}`);
  }

  if (!isValidPriority(data.priority_level)) {
    throw new Error(`Invalid priority level: ${data.priority_level}`);
  }

  if (!isValidStatus(data.status)) {
    throw new Error(`Invalid status: ${data.status}`);
  }

  // 3. Numeric Validations
  const targetAmount = Number(data.target_amount);
  const currentAmount = Number(data.current_amount || 0);

  if (isNaN(targetAmount) || targetAmount <= 0) {
    throw new Error("Target amount must be a positive number.");
  }

  if (isNaN(currentAmount) || currentAmount < 0) {
    throw new Error("Current amount must be a non-negative number.");
  }

  if (currentAmount > targetAmount) {
    throw new Error(
      "Current amount cannot exceed target amount (overfunding requires manual strategic adjustment).",
    );
  }

  // 4. Date Validation
  let normalizedDate: string | null = null;
  if (data.target_date) {
    const d = new Date(data.target_date);
    if (isNaN(d.getTime())) {
      throw new Error("Invalid target date format.");
    }
    normalizedDate = d.toISOString().split("T")[0]; // Store as YYYY-MM-DD
  }

  // 5. Normalization
  const normalizedNotes = data.notes?.trim() || null;
  const finalNotes = normalizedNotes === "" ? null : normalizedNotes;

  return {
    objective_name: data.objective_name.trim(),
    objective_type: data.objective_type as ObjectiveType,
    target_amount: targetAmount,
    current_amount: currentAmount,
    priority_level: data.priority_level as ObjectivePriority,
    status: data.status as ObjectiveStatus,
    target_date: normalizedDate,
    notes: finalNotes,
  };
}
```

File: src/lib/finance/taxonomy.ts

```
/**
```

```
 * AXIOM RETURN-BASED TAXONOMY
 * Centralized definitions for the financial intelligence system.
 */

export const VALID_CATEGORIES = [
  "Asset",
  "Skill",
  "Leverage",
  "Experience",
  "Maintenance",
  "Leakage",
] as const;
export type ValidCategory = (typeof VALID_CATEGORIES)[number];

export interface TaxonomyCategory {
  value: ValidCategory;
  label: string;
  definition: string;
  interpretation: string;
}

export const TAXONOMY_CATEGORIES: TaxonomyCategory[] = [
  {
    value: "Asset",
    label: "Asset",
    definition: "Expected future value generation",
    interpretation:
      "Strategic capital foundation. These deployments are expected to produce tangible appreciation or yield
over time.",
  },
  {
    value: "Skill",
    label: "Skill",
    definition: "Improves future earning capability",
    interpretation:
      "Human capital investment. Deployments into skills increase your intrinsic market value and future inflow
capacity.",
  },
  {
    value: "Leverage",
    label: "Leverage",
    definition: "Multiplies output or preserves time",
    interpretation:
      "Operational efficiency. Leverage deployments are designed to buy back time or multiply the results of
your effort.",
  },
  {
    value: "Experience",
    label: "Experience",
    definition: "Intentional quality-of-life deployment",
    interpretation:
          "Strategic utility. Non-yielding deployments that provide intentional lived-value and mental
sustainability.",
  },
  {
    value: "Maintenance",
    label: "Maintenance",
    definition: "Structural survival requirements",
    interpretation:
      "Operational core. Survival-layer capital needed to maintain your current system (Rent, utilities, basic
nourishment).",
  },
  {
    value: "Leakage",
```

```
    label: "Leakage",
    definition: "Non-strategic capital drift",
    interpretation:
      "Systemic inefficiency. Tracked separately to identify recurring capital drift. Excess leakage compresses
operational runway.",
  },
];

export const isValidCategory = (value: string): boolean => {
  return VALID_CATEGORIES.includes(value as ValidCategory);
};

export const formatTaxonomyCategoryList = (): string => {
  return VALID_CATEGORIES.join(", ");
};

export const getTaxonomyDefinition = (value: string) => {
  return TAXONOMY_CATEGORIES.find((c) => c.value === value)?.definition || "";
};

export const getTaxonomyInterpretation = (value: string) => {
  return (
    TAXONOMY_CATEGORIES.find((c) => c.value === value)?.interpretation || ""
  );
};
```

File: src/lib/finance/validateDeployment.ts

```
/**
 * TAXONOMY ENFORCEMENT GATE
 * Ensures every deployment written to the ledger follows the strict return-based taxonomy.
 */

import {
  formatTaxonomyCategoryList,
  isValidCategory,
} from "./taxonomy";
import {
  DeploymentAdvancedContextInput,
  normalizeDeploymentAdvancedContext,
} from "./deploymentContext";
import { evaluateMetadataQuality } from "./metadataQuality";

export { VALID_CATEGORIES, type ValidCategory } from "./taxonomy";

export interface DeploymentInput {
  title: string;
  amount: number;
  category: string;
  impactScore?: number;
  advancedContext?: DeploymentAdvancedContextInput;
}

/**
 * Pre-write Validation Layer
 * Rejects any input that violates taxonomy integrity or data truth.
 */
export function validateDeployment(input: DeploymentInput) {
  const { title, amount, category, impactScore, advancedContext } = input;

  // 1. Structural Integrity
  if (!title || !title.trim()) {
    throw new Error("Taxonomy Gate: Title is required for deployment verification");
```

```
  }

  if (amount === undefined || amount === null || isNaN(amount)) {
    throw new Error("Taxonomy Gate: Numeric amount is required");
  }

  if (amount <= 0) {
    throw new Error("Taxonomy Gate: Capital deployment must be a positive value (> 0)");
  }

  // 2. Taxonomy Enforcement
  if (!isValidCategory(category)) {
    // Specifically reject 'Unclassified' and legacy categories
    if (category === "Unclassified") {
      throw new Error("Taxonomy Gate: Strategic classification is mandatory. Entry rejected.");
    }

    throw new Error(`Taxonomy Gate: Unknown category '${category}'. Use ${formatTaxonomyCategoryList()}.`);
  }

  // 3. Normalization
  return {
    title: title.trim(),
    amount: Number(amount),
    category,
    advancedContext: normalizeDeploymentAdvancedContext(advancedContext),
    metadataQuality: evaluateMetadataQuality(title),
    impactScore: Math.min(Math.max(impactScore || 0, 0), 10) // Scale 0-10
  };
}
```

File: src/lib/insights/generateInsights.ts

```
import { BehavioralContext } from "../context/contextTypes";
import { InsightEngineResult, InsightPriority } from "./types";
import { KairosInsight } from "../ai/kairos";
import { rules } from "./rules/registry";
import { logKairosEvaluation } from "../db/telemetry";

/**
 * Axiom Insight Engine Orchestrator
 * Evaluates behavioral context against the rule registry to generate
 * prioritized strategic insights.
 *
 * V1.2: Added asynchronous telemetry logging for performance audit.
 */
export const evaluateInsights = async (
  context: BehavioralContext,
): Promise<InsightEngineResult> => {
  // 1. Process all rules and capture telemetry
  const evaluationResults = await Promise.all(
    rules.map(async (rule) => {
      const start = Date.now();
      const isMatched = rule.condition(context);

      let insight: KairosInsight | null = null;
      if (isMatched) {
        insight = rule.generate(context);
      }

      const duration = Date.now() - start;

      // Persist forensic telemetry (Non-blocking but awaited for database integrity)
```

```
      await logKairosEvaluation({
        ruleId: rule.id,
        severity: insight?.severity || "none",
        durationMs: duration,
      });

      if (isMatched && insight) {
        return {
          priority: rule.priority,
          insight,
        };
      }
      return null;
    }),
  );

  const generatedInsights = evaluationResults.filter(
    (result): result is { priority: InsightPriority; insight: KairosInsight } =>
      result !== null,
  );

  // Axiom Standard: Gracefully handle the "Quiet" state
  if (generatedInsights.length === 0) {
    return {
      primaryInsight: {
        type: "info",
        severity: "observation",
        category: "strategic_alignment",
        message:
          "No material structural deterioration detected since previous evaluation.",
        supportingSignals: [],
        timestamp: new Date().toISOString(),
        runway: null,
        capitalEfficiency: context.capitalEfficiencyScore ?? 0,
        isSilent: true, // This is the "Quiet System" output
      },
      allInsights: [],
    };
  }

  // 2. Prioritize
  // Sort by priority (High > Medium > Low) and then by confidence
  const sortedInsights = [...generatedInsights].sort((a, b) => {
    const priorityScore: Record<InsightPriority, number> = {
      critical: 4,
      high: 3,
      medium: 2,
      low: 1,
    };

    if (priorityScore[a.priority] !== priorityScore[b.priority]) {
      return priorityScore[b.priority] - priorityScore[a.priority];
    }

    return 0; // Maintain order within same priority
  });

  return {
    primaryInsight: sortedInsights[0].insight,
    allInsights: generatedInsights.map((result) => result.insight),
  };
};
```

File: src/lib/insights/rules/registry.ts

```
import { InsightRule } from "../types";

/**
 * KAIROS RULE REGISTRY (V1.1)
 * Behavioral Presence implementation.
 * Identity: Restrained Operational Analyst.
 */

export const rules: InsightRule[] = [
  // 1. SOLVENCY CRISIS (Critical Severity)
  {
    id: "solvency_crisis",
    priority: "critical",
    condition: (ctx) => ctx.netWorth < 0,
    generate: (ctx) => ({
      type: "warning",
      severity: "critical",
      category: "solvency_pressure",
      timestamp: new Date().toISOString(),
      message:
        "Severe solvency crisis detected. Total credit facilities (e.g., Fuliza, M-Shwari, SACCO Loans) exceed
combined capital assets, resulting in a negative net worth state.",
      supportingSignals: [
        `Net Worth: KSh ${Math.round(ctx.netWorth).toLocaleString()}`,
        `Liquid Reserves (MMF/SACCO): KSh ${Math.round(ctx.liquidity).toLocaleString()}`,
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    }),
  },

  // 2. STRUCTURAL DEFICIT (Critical Severity)
  {
    id: "structural_deficit",
    priority: "high",
    condition: (ctx) => {
      const monthlyBurn = ctx.burnRate.monthlyProjection;
      const monthlyReplenishment = ctx.totalMonthlyIncome;

      // Trigger if structural burn exceeds income (Structural Deficit)
      return monthlyBurn > monthlyReplenishment && monthlyBurn > 0;
    },
    generate: (ctx) => {
      const deficit = ctx.burnRate.monthlyProjection - ctx.totalMonthlyIncome;
      return {
        type: "warning",
        severity: "critical",
        category: "solvency_pressure",
        timestamp: new Date().toISOString(),
        message:
          "Structural deficit detected. Baseline monthly outflow (including M-Pesa leakage) currently outpaces
aggregate inbound yield (M-Pesa flows / Hustle & Salary).",
        supportingSignals: [
          `Monthly Deficit: KSh ${Math.round(deficit).toLocaleString()}`,
            `Inbound Yield Coverage: ${Math.round((ctx.totalMonthlyIncome / ctx.burnRate.monthlyProjection) *
100)}% of monthly burn.`,
            `Survival Window: ${ctx.runway.currentDays ? Math.round(ctx.runway.currentDays) : "Unknown"} days
remaining.`,
        ],
        runway: ctx.runway.currentDays,
```

```
        capitalEfficiency: ctx.capitalEfficiencyScore,
        isSilent: false,
      };
    },
  },
  // 3. SOLVENCY PRESSURE (Critical/Warning Severity)
  {
    id: "solvency_pressure",
    priority: "high",
    condition: (ctx) =>
      !ctx.strategicAlignment.liquiditySufficiency.isSufficient,
    generate: (ctx) => {
      const { shortfall, message } =
        ctx.strategicAlignment.liquiditySufficiency;

      const isCritical =
        ctx.liquidity === 0 ? shortfall > 0 : shortfall / ctx.liquidity > 0.5;

      return {
        type: "warning",
        severity: isCritical ? "critical" : "warning",
        category: "solvency_pressure",
        timestamp: new Date().toISOString(),
        message:
                "Liquid reserves (MMF, SACCO Deposits) are insufficient to satisfy all critical strategic
obligations.",
        supportingSignals: [
          message,
          `Combined Available Liquid Reserves: KSh ${Math.round(ctx.liquidity).toLocaleString()}`,
        ],
        runway: ctx.runway.currentDays,
        capitalEfficiency: ctx.capitalEfficiencyScore,
        isSilent: false,
      };
    },
  },

  // 4. OBJECTIVE STARVATION (Advisory Severity)
  {
    id: "objective_starvation",
    priority: "medium",
    condition: (ctx) =>
      ctx.strategicAlignment.objectiveStarvationSignals.length > 0,
    generate: (ctx) => ({
      type: "info",
      severity: "advisory",
      category: "objective_starvation",
      timestamp: new Date().toISOString(),
      message: `Strategic starvation detected: ${ctx.strategicAlignment.objectiveStarvationSignals[0]}`,
      supportingSignals:
        ctx.strategicAlignment.objectiveStarvationSignals.slice(1, 3),
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    }),
  },

  // 5. STRATEGIC CONFLICT (Warning Severity)
  {
    id: "strategic_conflict",
    priority: "medium",
    condition: (ctx) =>
      ctx.strategicAlignment.strategicAllocationSignals.length > 0,
    generate: (ctx) => ({
```

```
      type: "warning",
      severity: "warning",
      category: "strategic_alignment",
      timestamp: new Date().toISOString(),
      message: ctx.strategicAlignment.strategicAllocationSignals[0],
      supportingSignals: [
        `Alignment Pressure: ${ctx.strategicAlignment.alignmentPressure}/100`,
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    }),
  },

  // 6. RUNWAY DEPLETION (Critical Severity)
  {
    id: "runway_critical",
    priority: "critical",
    condition: (ctx) => ctx.runway.status === "critical",
    generate: (ctx) => ({
      type: "warning",
      severity: "critical",
      category: "solvency_pressure",
      timestamp: new Date().toISOString(),
      message: `Operational runway has contracted to ${Math.round(ctx.runway.currentDays || 0)} days. Immediate
capital preservation (minimizing M-Pesa leakage) required.`,
      supportingSignals: [
          `Runway Delta: ${ctx.runway.deltaDays >= 0 ? "+" : ""}${Math.round(ctx.runway.deltaDays)} days since
last evaluation.`,
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    }),
  },

  // 7. LEAKAGE CRISIS (Critical Severity)
  {
    id: "efficiency_crisis",
    priority: "high",
    condition: (ctx) => ctx.capitalEfficiencyScore < 40,
    generate: (ctx) => {
      const isLeakage = ctx.allocation.dominantCategory === "Leakage";
      return {
        type: "warning",
        severity: "critical",
        category: "capital_efficiency",
        timestamp: new Date().toISOString(),
        message: isLeakage
            ? "Capital efficiency crisis detected. Excessive 'Mobile Money (M-Pesa) Leakage' is compromising
long-term sustainability."
            : "Capital efficiency has reached a critical threshold. Current deployment patterns (Chama, MMF,
Shamba) are suboptimal.",
        supportingSignals: [
           `Efficiency Index: ${ctx.capitalEfficiencyScore}/100 ? Dominant: ${ctx.allocation.dominantCategory}
(${(ctx.allocation.categoryDistribution[ctx.allocation.dominantCategory] * 100).toFixed(1)}%).`,
        ],
        runway: ctx.runway.currentDays,
        capitalEfficiency: ctx.capitalEfficiencyScore,
        isSilent: false,
      };
    },
  },
```

```
  // 8. UNCLASSIFIED DATA (Advisory Severity)
  {
    id: "unclassified_data",
    priority: "medium",
    condition: (ctx) =>
      (ctx.allocation.categoryDistribution["Unclassified"] || 0) > 0.1, // Only alert if > 10%
    generate: (ctx) => ({
      type: "info",
      severity: "advisory",
      category: "strategic_alignment",
      timestamp: new Date().toISOString(),
      message:
           "Unclassified outflows detected. Reclassify recent M-Pesa flows to restore the accuracy of the
efficiency index.",
      supportingSignals: [
         `Metadata Integrity: ${Math.round((ctx.allocation.categoryDistribution["Unclassified"] || 0) * 100)}%
of flows lack strategic classification.`,
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    }),
  },

  // 9. ERRATIC VOLATILITY (Warning Severity)
  {
    id: "erratic_volatility",
    priority: "medium",
    condition: (ctx) => ctx.volatilityScore > 0.4,
    generate: (ctx) => ({
      type: "warning",
      severity: "warning",
      category: "capital_efficiency",
      timestamp: new Date().toISOString(),
      message:
          "High volatility in capital deployment detected. Irregular outbound M-Pesa flows may disrupt runway
projections.",
      supportingSignals: [
          `Irregularity Index: HIGH ? Volatility score of ${(ctx.volatilityScore * 100).toFixed(1)}% exceeds
variance threshold.`,
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    }),
  },

  // 10. HIGH DISCIPLINE (Observation Severity)
  {
    id: "high_discipline",
    priority: "medium",
    condition: (ctx) => ctx.volatilityScore < 0.2 && ctx.deploymentCount > 1,
    generate: (ctx) => {
      const isAsset = ctx.allocation.dominantCategory === "Asset";
      return {
        type: "info",
        severity: "observation",
        category: "capital_efficiency",
        timestamp: new Date().toISOString(),
        message: isAsset
             ? "High operational discipline in Capital Deployment (Chama, MMF, Shamba) detected. Consistent
deployment provides a stable foundation for scaling."
          : "High operational discipline detected. Consistent outflow patterns provide a stable foundation for
capital scaling.",
```

```
        supportingSignals: [
                  `Stability Metric: ${(100 - ctx.volatilityScore * 100).toFixed(1)}% consistency across
${ctx.deploymentCount} deployments.`,
        ],
        runway: ctx.runway.currentDays,
        capitalEfficiency: ctx.capitalEfficiencyScore,
        isSilent: true,
      };
    },
  },

  // 11. HEAVY CONCENTRATION (Advisory Severity)
  {
    id: "heavy_concentration",
    priority: "low",
    condition: (ctx) => ctx.allocation.concentrationScore > 0.7,
    generate: (ctx) => ({
      type: "info",
      severity: "advisory",
      category: "strategic_alignment",
      timestamp: new Date().toISOString(),
       message: `Significant capital concentration detected in ${ctx.allocation.dominantCategory}. Ensure this
deployment (Chama, MMF, Shamba) aligns with strategic intent.`,
      supportingSignals: [
                              `Concentration Score: ${ctx.allocation.concentrationScore.toFixed(2)} ?
${ctx.allocation.dominantCategory}                                                                    represents
${(ctx.allocation.categoryDistribution[ctx.allocation.dominantCategory] * 100).toFixed(1)}% of total
deployment.`,
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    }),
  },

  // 12. DEFAULT PATTERN (Legible Silence - Observation)
  {
    id: "default_pattern",
    priority: "low",
    condition: () => true,
    generate: (ctx) => ({
      type: "info",
      severity: "observation",
      category: "strategic_alignment",
      timestamp: new Date().toISOString(),
      message:
             "No material behavioral shifts detected in M-Pesa flows or capital deployment. Silence is
intentional.",
      supportingSignals: [
        "Observing capital behavior: State remains within normal variance parameters.",
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: true,
    }),
  },

  // 13. INCOME CONCENTRATION RISK (Warning Severity)
  {
    id: "income_concentration_risk",
    priority: "medium",
    condition: (ctx) => ctx.maxIncomeConcentrationRatio > 0.8,
    generate: (ctx) => ({
      type: "warning",
```

```
      severity: "warning",
      category: "capital_efficiency",
      timestamp: new Date().toISOString(),
      message:
         "High inbound concentration risk detected. Over 80% of total yield relies on a single primary flow,
creating severe structural fragility.",
      supportingSignals: [
        `Concentration Ratio: ${(ctx.maxIncomeConcentrationRatio * 100).toFixed(1)}% from primary source.`,
      ],
      runway: ctx.runway.currentDays,
      capitalEfficiency: ctx.capitalEfficiencyScore,
      isSilent: false,
    }),
  },
];
```

File: src/lib/insights/types.ts

```
import { BehavioralContext } from "../context/contextTypes";
import { KairosInsight } from "../ai/kairos";

export type InsightPriority = "critical" | "high" | "medium" | "low";

export interface InsightRule {
  id: string;
  priority: InsightPriority;
  condition: (context: BehavioralContext) => boolean;
  generate: (context: BehavioralContext) => KairosInsight;
}

export interface InsightEngineResult {
  primaryInsight: KairosInsight;
  allInsights: KairosInsight[];
}
```

File: src/lib/supabase.ts

```
import { createClient } from "./supabase/client";

export const supabase = createClient();
```

File: src/lib/supabase/client.ts

```
import { createBrowserClient } from '@supabase/ssr'

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  )
}
```

File: src/lib/supabase/middleware.ts

```
import { createServerClient } from "@supabase/ssr";
import { NextResponse, type NextRequest } from "next/server";

export async function updateSession(request: NextRequest) {
  let supabaseResponse = NextResponse.next({
    request,
  });
```

```
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll();
        },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value }) =>
            request.cookies.set(name, value),
          );
          supabaseResponse = NextResponse.next({
            request,
          });
          cookiesToSet.forEach(({ name, value, options }) =>
            supabaseResponse.cookies.set(name, value, options),
          );
        },
      },
    },
  );

  // refreshing the auth token
  await supabase.auth.getUser();

  return supabaseResponse;
}
```

File: src/lib/supabase/server.ts

```
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options),
            );
          } catch (error) {
            // This is expected if calling from a Server Component that is already streaming.
            // We log it only for debugging in dev mode.
            if (process.env.NODE_ENV === "development") {
              console.warn(
                "Supabase SSR: Unable to set cookies from Server Component. Ensure middleware is configured.",
                error,
              );
            }
          }
        },
      },
```

```
    },
  );
}
```

File: src/lib/utils/formatters.ts

```
/**
 * Formatting utilities for the Axiom presentation layer.
 * Localized for the Kenyan market (KES / en-KE).
 */

export const formatCurrency = (amount: number): string => {
  return new Intl.NumberFormat("en-KE", {
    style: "currency",
    currency: "KES",
    maximumFractionDigits: 0,
  }).format(amount);
};

export const formatNumber = (num: number): string => {
  return new Intl.NumberFormat("en-KE").format(num);
};

export const formatDate = (dateString: string): string => {
  return new Date(dateString).toLocaleDateString("en-KE", {
    year: "numeric",
    month: "short",
    day: "numeric",
  });
};
```

File: src/lib/utils/taxonomy.ts

```
import { AccountType } from "../finance/accounts";
import { LiabilityType } from "../finance/liabilities";
import { IncomeType } from "../finance/income";
import { ValidCategory } from "../finance/taxonomy";

/**
 * TAXONOMY MAPPING DICTIONARY
 * Localizes abstract database enums to Kenyan market realities.
 */

export const AccountMap: Record<AccountType, string> = {
  checking: "M-Pesa / Bank",
  savings: "Savings / MMF",
  mobile_money: "M-Pesa",
  brokerage: "Investment / CDS",
  crypto: "Digital Assets",
  cash: "Cash on Hand",
};

export const LiabilityMap: Record<LiabilityType, string> = {
  credit_card: "Credit Card / Digital Loan",
  mortgage: "Mortgage / Shamba Loan",
  personal_loan: "Fuliza / SACCO Loan",
  student_loan: "HELB / Education Loan",
  business_loan: "Business / SME Loan",
  line_of_credit: "Overdraft facility",
  other: "Other Credit Facilities",
};
```

```
export const IncomeMap: Record<IncomeType, string> = {
  salary: "Salary / Hustle",
  freelance: "Freelance / Gig Work",
  business: "Business Yield",
  dividends: "Dividends / Chama Payout",
  rental: "Rental Income",
  contract: "Contract Fee",
  other: "Other Inflows",
};

export const DeploymentMap: Record<ValidCategory, string> = {
  Asset: "MMF / Chama / Equities",
  Skill: "Upskilling / Certifications",
  Leverage: "Tools / Automation",
  Experience: "Lifestyle / Experiences",
  Maintenance: "Basic Survival (Rent/Food)",
  Leakage: "M-Pesa Leakage / Unclassified",
};
```

File: src/proxy.ts

```
import { type NextRequest, NextResponse } from "next/server";
import { updateSession } from "@/lib/supabase/middleware";
import { createServerClient } from "@supabase/ssr";

export async function proxy(request: NextRequest) {
  // Update the session (refreshes the token)
  const response = await updateSession(request);

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll();
        },
        setAll() {
          // No-op for read-only session check
        },
      },
    },
  );

  const {
    data: { user },
  } = await supabase.auth.getUser();

  // ROUTE PROTECTION

  // If user is not logged in and trying to access dashboard, redirect to home
  if (!user && request.nextUrl.pathname.startsWith("/dashboard")) {
    const url = request.nextUrl.clone();
    url.pathname = "/";
    const redirectResponse = NextResponse.redirect(url);

    // Copy all cookies from the session update response to the redirect response
    response.cookies.getAll().forEach((cookie) => {
      redirectResponse.cookies.set(cookie.name, cookie.value, cookie);
    });

    return redirectResponse;
  }
```

```
  // If user is logged in and trying to access home (login page), redirect to dashboard
  if (user && request.nextUrl.pathname === "/") {
    const url = request.nextUrl.clone();
    url.pathname = "/dashboard";
    const redirectResponse = NextResponse.redirect(url);

    // Copy all cookies from the session update response to the redirect response
    response.cookies.getAll().forEach((cookie) => {
      redirectResponse.cookies.set(cookie.name, cookie.value, cookie);
    });

    return redirectResponse;
  }

  return response;
}

export const config = {
  matcher: [
    /*
     * Match all request paths except for the ones starting with:
     * - _next/static (static files)
     * - _next/image (image optimization files)
     * - favicon.ico (favicon file)
     * Feel free to modify this pattern to include more paths.
     */
    "/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*)",
  ],
};
```

File: tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next",
      },
    ],
    "paths": {
      "@/*": ["./src/*"],
    },
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
```

```
    "**/*.mts",
  ],
  "exclude": ["node_modules", "mobile"],
}
```